Universität Bielefeld
Technische Fakultät
AG Technische Informatik

Bachelorarbeit

Realtime obstacle detection from optical flow in panoramic images

Andreas Stöckel

5. September 2013

Im Studiengang
Kognitive Informatik

Gutachter
M.Sc. Michael Horst
M.Sc. David Fleer

Danksagung

Danke an meine Betreuer Michael Horst und David Fleer für das widerstandslose Ertragen meiner Berichte über Probleme, die sich im Anschluss daran meist von selbst lösten, zündende Ideen, sowie das Korrekturlesen des Manuskripts. Dank gilt natürlich auch meiner Familie – insbesondere meinen Eltern – dafür, mein Exil fernab der Heimat so erträglich wie irgend möglich zu gestalten. Abschließend gilt mein besonderer Dank Herrn Prof. Möller für den *Earl Grey* Tee, dessen Superkräfte mich auch zu nachtschlafender Zeit noch arbeiten ließen.

Contents

Summary **5**

1 Introduction **7**
1.1 Motivation 7
1.2 Special requirements 9
1.3 Existing solutions 9
1.4 Algorithm overview 10
1.4.1 Limitations of the approaches presented above 10
1.4.2 General idea 11

2 Algorithm description **13**
2.1 Prerequisites 13
2.1.1 Input data: motion parameters and panoramic images . 13
2.1.2 Assumptions 17
2.2 Depth reconstruction 18
2.2.1 Short introduction to optical flow estimation 18
2.2.2 Motion field equation 19
2.2.3 Flow parametrization 19
2.2.4 Error stack calculation 20
2.2.5 Input image augmentation 21
2.2.6 Minimum error search 23
2.2.7 Minimum error refinement 24
2.3 Obstacle detection 25
2.3.1 Probabilistic landmark model 27
2.3.2 Distribution accumulation and obstacle detection 28

3 Algorithm extensions **31**
3.1 Coarse-to-fine refinement 31
3.1.1 Motivation: temporal aliasing 31
3.1.2 Working around temporal aliasing: coarse-to-fine refinement 32
3.1.3 Integration of coarse-to-fine refinement into the depth reconstruction framework 32

3.1.4 Implementation remarks . . . 34
3.2 Flow template calculation . . . 36
3.2.1 Motion field parameter space reduction . . . 37
3.2.2 Parameter selection, index to depth conversion and distance variance . . . 40

4 Implementation **41**
4.1 Short introduction to SSE . . . 41
4.2 Error stack calculation . . . 42
4.2.1 Snapshot memory layout . . . 42
4.2.2 Separability of the motion field equation . . . 44
4.2.3 Flow template calculation . . . 45
4.2.4 Snapshot warping and error stack calculation . . . 45
4.3 Index field calculation . . . 47
4.3.1 Propagation matrix and minimum error search . . . 47
4.3.2 Index field refinement . . . 48
4.4 Local environment map . . . 50
4.4.1 General optimizations . . . 50
4.4.2 Distribution accumulation optimization . . . 50

5 Experiments and results **53**
5.1 Environments . . . 53
5.1.1 Computer-generated environments . . . 53
5.1.2 Real-world tests . . . 56
5.2 Quality measures . . . 56
5.2.1 True and false positive obstacle points . . . 56
5.2.2 Obstacle detection error . . . 57
5.2.3 Depth reconstruction error . . . 58
5.3 Results . . . 58
5.3.1 Default parameters . . . 58
5.3.2 Simulated test run . . . 59
5.3.3 Real-world test run . . . 59
5.3.4 Error stack bias . . . 63
5.3.5 Coarse-to-fine search . . . 64
5.3.6 Noise robustness . . . 64

6 Discussion and conclusion **71**
6.1 Discussion . . . 71
6.1.1 Simulated test runs . . . 71
6.1.2 Real-world test . . . 72
6.1.3 Detailed experiments . . . 72
6.2 Conclusion . . . 73
6.3 Outlook . . . 73

A Derivations and proofs **75**
A.1 Probabilistic landmark model 75
A.1.1 Covariance matrix transformation 75
A.1.2 Derivation of σ_β for adjacent Gaussian distributions . . 77
A.2 Motion field equation 78
A.2.1 Substitution of r and d 78
A.2.2 Rotational motion elimination 78
A.2.3 Maximum horizontal shift 80

B Error propagation matrices **83**
B.1 Space variant propagation matrix 83
B.2 Propagation matrix performance 84

C Source code **87**
C.1 Error stack calculation 87
C.1.1 Horizontal interpolation 87
C.1.2 Vertical interpolation 88
C.2 Gauss template accumulation 89

Bibliography **91**

Summary

An important component of mobile robot navigation is obstacle detection. For this purpose, the target robot platform is equipped with a set of static laser diodes and an omnidirectional camera. The usage of only four diodes at a time in conjunction with slow temporal sampling results in a relatively coarse environmental coverage, which causes small and slim objects (like legs of tables or chairs) to be often missed.

A previously developed algorithm is capable of detecting obstacles by analyzing time series of spherical panoramic images using implicit optical flow estimation. The goal of this thesis is to lay the theoretical foundations of the algorithm and to present performance optimizations allowing real-time operation on an embedded system. Moreover, measures for the obstacle detection performance in simulated test runs are developed.

Performance optimization is achieved by a resolution pyramid with coarse-to-fine refinement in the optical flow estimation step, motion field precalculation, as well as the use of single instruction multiple data (SIMD) instruction set extensions in the implementation.

The final implementation fully satisfies the real-time constraint, typically requiring less than 30 ms for an obstacle detection step on the target system. While tests in simulated environments show that the algorithm is capable of mean obstacle detection errors smaller than 0.06 m, the real-world performance heavily depends on the quality of the input and especially on correct robot orientation estimates.

Chapter 1

Introduction

In the last three decades a vast number of techniques for obstacle detection and evasion in mobile robots have been developed. While classical approaches include active range sensors, advancements in imaging hardware and processing power led to approaches which solely rely on visual information.

This chapter provides an overview of the particular obstacle detection problem that had to be solved, as well as an outline and evaluation of already existing approaches, finally leading to the actual method presented in this thesis.

1.1 Motivation

Obstacle detection and avoidance are crucial to most autonomous mobile systems, both biological and robotic. While mandatory for systems moving at high velocities in order to avoid physical damage, long-range obstacle detection is still a necessity for efficient navigation and route planning in applications like consumer-grade cleaning robots.

The target robot platform — an in-house development inspired by such cleaning robots (see figure 1.1) — carries a set of eight static lasers diodes which are used for obstacle detection. The detection is performed by triangulating the laser points as they appear in an omnidirectional camera image. The usage of only four diodes at a time in conjunction with slow temporal sampling results in a relatively coarse environmental coverage, which likely causes small and slim objects to be missed.

Classical approaches to obstacle detection on mobile robots include sonar sensors and laser scanners, while more recent developments include time-of-flight cameras and structured light sensors (Siegwart et al., 2011, chapter 4.1.9 “Active ranging”, p. 125ff.). While laser scanners in particular provide a high spatial resolution, these solutions are specialized hardware components which might not be suitable for budget-constrained consumer devices. However, commercially available cleaning robots that employ laser scanner based

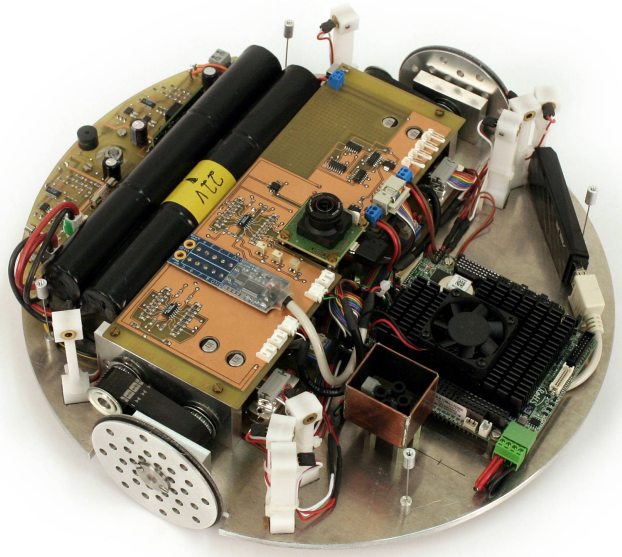


Figure 1.1: Photo of the robot platform (in-house development) as used in the experiments. One can see the eight laser diodes (mounted at the white holders) used for the coarse obstacle detection as well as the fisheye camera in the center of the platform and the on-board computer at the right side of the image. The radius of the entire system (including the cover not shown) is about 35 cm, the top of the camera lens is located about 10 cm above the ground.

obstacle detection exist[1]. Furthermore space, payload and electrical power restrictions can often force the hardware to be as simple and small as possible.

The availability of low-cost imaging hardware (partially caused by the widespread adaption of cameras in day-to-day electronic devices) and the emergence of small, powerful microprocessors with low energy footprint allow for cost-effective integration of image processing systems into mobile robots. Therefore a monochrome, upward directed camera equipped with a fisheye lens, allowing omnidirectional vision, is already built into the robot platform and — besides the coarse laser based obstacle detection — utilized for navigation using min-warping (Möller et al., 2010; Gerstmayr-Hillen et al., 2013).

The goal of this thesis is to deduce an obstacle detection algorithm from a fundamental optical flow equation and finally to provide a real-time solution

[1]According to the manufacturer, laser scanner based obstacle detection is employed in their *Neato* robot vacuum cleaner series (see `http://www.neatorobotics.com/`). Yet the laser scanner used here might not measure time of flight, but may instead be based on triangulation.

capable of exploiting the visual information provided by the visual sensor. Obstacle detection is thus turned from a hardware into a mere software problem.

1.2 Special requirements

In order to allow real-time operation of the entire robot, the obstacle detection algorithm must stay within a certain time budget. As the algorithm has to run along numerous other processes and a new image is captured approximately every 200 ms, the execution time of the obstacle detection method should stay well below 50 ms on an embedded Intel® Atom™ N2600 dual core CPU running at 1.60 GHz.

Another requirement concerns the vertical range in which obstacles need to be detected: The robot only has a height of about 10 cm, hence obstacles lying far above that height do not need to be detected. In fact, detecting such obstacles (at least without height information) would be counterproductive, as it might prevent the robot from driving beneath furniture.

The requirements are relaxed by the fact that the robot is aimed at domestic indoor use, which allows to assume a relatively static world and a mostly even ground.

1.3 Existing solutions

Vision based obstacle detection is an active research area. Binocular vision is an approach often used in applications for which reliable obstacle detection in non-static worlds or in conditions without ego-motion is mandatory (for example automotive applications as in Bertozzi and Broggi, 1998).

Fewer solutions exist for omnidirectional, monocular vision. The underlying idea of these approaches is to exploit the parallax information present in subsequent images captured under motion: Near objects show a relatively large displacement and can thus be detected.

The following list contains a selection of approaches suitable for the present camera setup:

Direct analysis of non-sparse optical flow: In this approach, non-sparse optical flow is estimated from a time series of images. The flow field is then directly fed into the motor controller in order to avoid obstacles. This approach can be biologically motivated, as for example insect flight is believed to be controlled by optical flow (Egelhaaf, 2009). Early experiments with optical flow and flying robots have shown that this direct form of motor control is indeed practical (Coombs and Roberts, 1992).

Free space detection: As above optical flow is estimated from a time series of images and compared to the expected movement of the ground surface.

Instead of direct motor control, free space regions with no obstacles are detected by analyzing the difference between the expected and measured flow vector field (Yoshizaki et al., 2008).

Feature-based optical flow and obstacle detection: A sparse flow field is created by matching corresponding image regions in a time series of input images. This matching may be performed using either sparse optical flow estimation techniques and/or feature detectors like SIFT (Lowe, 2004). Single high velocity flow vectors then correspond to near obstacles.

This method is for example described in a paper by Low and Wyeth (2005), where an one-dimensional polar obstacle map (comparable to those obtained from a laser scanner) is built from a "flow vector length to distance" estimate.

Obstacles from depth: Given the presence of a depth map which is reconstructed from the input images, classical approaches for robot navigation (Siegwart et al., 2011, chapters 5 and 6) can be employed.

Methods for depth reconstruction in monocular vision include *shape from texture* and *shape from shading*. Here depth information is reconstructed from still images under assumptions about texture distortion or lighting (Bülthoff and Yuille, 1991). Moreover, machine learning approaches can be applied to depth recognition in still images (Saxena et al., 2008).

Reconstructing depth from a series of images captured at slightly different positions is called *structure from motion*. Here a correspondence between feature points from at least two images is established. Besides performing depth reconstruction, the camera motion is estimated (see Siegwart et al., 2011, chapter 4.2.6 "Structure from motion", p. 181ff.).

An example for *structure from motion* in omnidirectional images on mobile robots can be found in Kawanishi et al. (2013), where parallel vertical lines are extracted as features in order to allow operation in textureless artificial environments.

1.4 Algorithm overview

In this section the approaches listed above are evaluated, followed by a brief summary of the solution presented in this thesis.

1.4.1 Limitations of the approaches presented above

The algorithm presented in this thesis follows the *obstacles from depth* design scheme:

An insect-like, direct control scheme is not suitable for efficient, preplanned trajectories. Free space detection as proposed by Yoshizaki et al.

cannot be used, as the omnidirectional camera on the robot platform does not cover the range below the image horizon and thus does not see the ground. In fact, it is noteworthy that many algorithms for obstacle detection in monocular, omnidirectional vision systems (including those mentioned above) are designed for a catadioptric camera system mounted on top of the robot, which allows to perceive the ground plane (Yasushi, 1999; Kim and Suga, 2007).

Feature-based approaches were also discarded, as preceding tests have shown that in usual domestic environments only few points of interest are present in the narrow ($\leq 10^\circ$) range above the image horizon, which is critical for obstacle detection.

Most *structure from motion* methods also use feature points for depth reconstruction and as mentioned above try to estimate the camera motion between the images. Yet as the robot knows its current velocity from the odometry data, this motion information is not required. In addition these techniques — including shape from shading and machine learning approaches — may not be suitable for the real-time constraints given on an embedded system.

1.4.2 General idea

Figure 1.2 shows a short summary of the general idea of the algorithm: The depth reconstruction is performed by implicitly estimating optical flow. This flow estimation is accelerated by utilizing as much prior knowledge as applicable, including the motion information returned from the odometry sensors. Once the depth has been reconstructed, the resulting landmark points are projected onto a local top-down environment map in which the probability distributions accompanying each reconstructed surface point are accumulated. If a certain threshold is exceeded, the corresponding map region is emitted as an obstacle point.

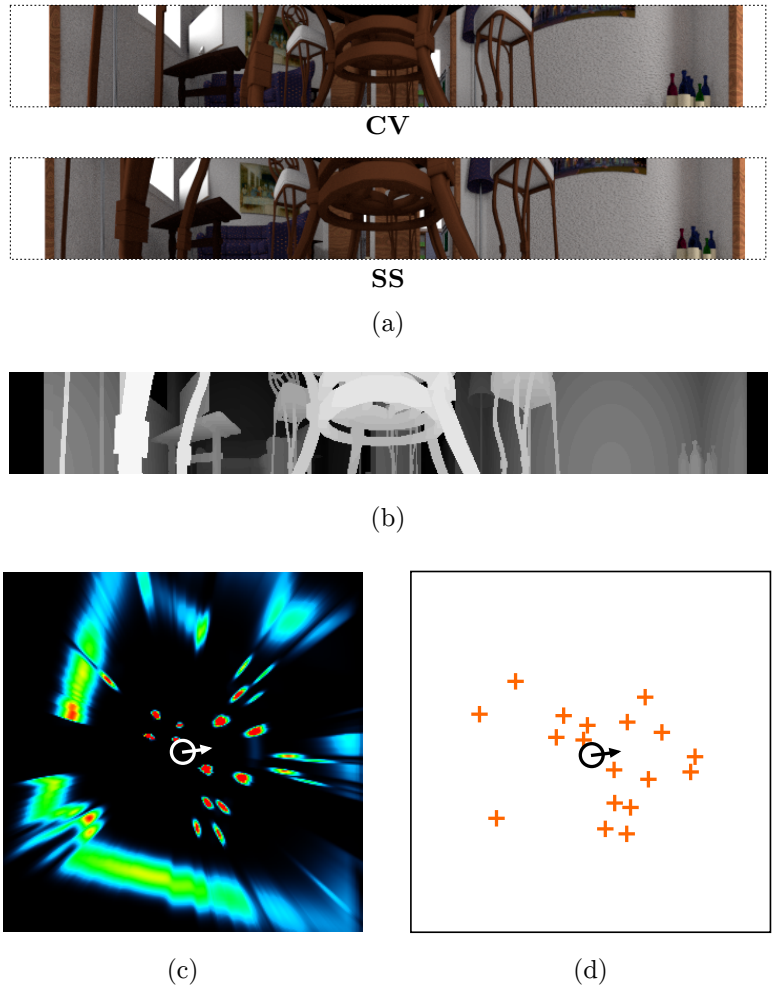


Figure 1.2: Conceptual overview of the stages in the obstacle detection algorithm: (a) A pair of panoramic input images (current view **CV** and snapshot **SS**) is given (the actual images are monochrome). (b) Estimation of a depth map containing the distance for every pixel (image shows the ground truth). (c) The uncertainty distribution of each reconstructed pixel is accumulated in a top-down projected local environment map. (d) Clusters in the map with high density values are returned as obstacle points.

Chapter 2

Algorithm description

This chapter describes the obstacle detection algorithm: First a short description of the input data and the assumptions made throughout the algorithm are given. Then the two major stages of the algorithm are presented: *Depth reconstruction* and the actual *obstacle detection*. While the first stage aims at reconstructing the world position of each pixel by implicitly calculating optical flow between two sequentially captured images, the second stage refines these results by accumulating the uncertainty distributions of the reconstructed landmark points obtained from multiple depth reconstruction steps in a local environment map.

2.1 Prerequisites

This section covers both a description of the input data and a list of assumptions that are (sometimes implicitly) made throughout the following sections.

2.1.1 Input data: motion parameters and panoramic images

The input of the obstacle detection algorithm is given as a pair of monochrome panoramic images, denoted **SS** and **CV** (snapshot and current view), accompanied by a set of motion parameters d, α and ψ.

Motion parameters: As depicted in figure 2.1 the motion parameters d, α and ψ describe a motion vector leading from the snapshot position to the current view position. The Euclidean distance in meters is denoted by d, whereas the motion direction is denoted by the angle α. The change in orientation under which the two images were captured is denoted by ψ.

Panoramic images: The fisheye images as captured from the camera are not directly used in the algorithm. As discussed in the introduction (section

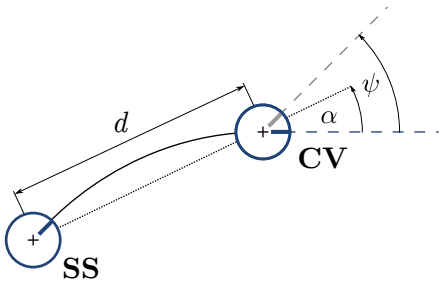


Figure 2.1: Description of the motion parameters d, α, ψ. The two circles depict the robot position and orientation (blue angle marks) under which the snapshot **SS** and current view **CV** were captured. The actual robot trajectory (solid line) is not required as the motion is fully described by the translatory distance d, the movement direction α and the change in orientation expressed by ψ.

1.2), the actual region of interest is too small to justify the usage of the complete input image. Instead the interesting region is mapped (using a lookup table that is generated in a calibration process) to a spherical panoramic image. The source region is chosen larger than the region of interest, as this extension will later (see section 4.2.1) be needed for pixel lookups. Additionally, histogram equalization is applied to the images in order to increase the contrast and to compensate global illumination changes. See figure 2.2 for an example. This and the following chapters assume such a spherical panoramic image and are independent of the actual image source.

Panoramic image coordinate system: The panoramic images contain a 360° view of the surroundings such that the x-, y-pixel coordinates of the image linearly correspond to a pair of angles β, γ which — in conjunction with the yet unknown landmark distance r — describe the position of the projected landmark in spherical coordinates (note that an elevation angle of $\gamma = 0$ corresponds to the image horizon and not the zenith). Let w, h denote the image width and height. It holds:

$$\beta(x) = \frac{2\pi}{w} \cdot x \qquad\qquad \gamma(y) = \frac{\check{\gamma} - \hat{\gamma}}{h - 1} \cdot y + \hat{\gamma}$$

where $\check{\gamma}$ is the so called *depression* describing the elevation angle of the bottom image row and $\hat{\gamma}$ the so called *elevation* describing the elevation angle of the top image row. The angles β and γ respectively correspond to azimuthal component and polar component of spherical coordinates (see figure 2.3 for an illustration).

In contrast to classical spherical coordinates, the distance $r = \tilde{r} \cdot \cos(\gamma)$ (with $\tilde{r}$ being the Euclidean distance) is measured as ground distance, which

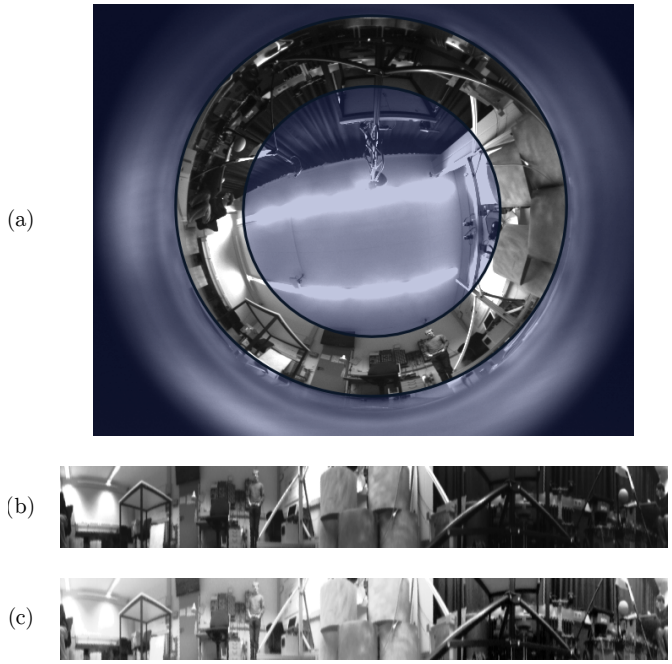


FIGURE 2.2: (a) The original 640×512 pixel frame as recorded by the camera. The region used for the panoramic image is highlighted, with the outer circle being the bottom row in the result images (specified by the *depression* angle $\check{\gamma}$) and the inner circle being the top row in the result images (specified by the *elevation* angle $\hat{\gamma}$). (b) The unrolled 720×96 pixel cylindrical panoramic image. (c) Cylindrical image after applying histogram equalization.

will allow for simplified equations and the separability of the motion field equation, as discussed in section 3.2. As depicted in figure 2.4, the change in the measurement of r only effects the representation of a series of equal-distanced landmarks. Moreover, points with an elevation angle of $\gamma = \pm 90°$ cannot be uniquely described in this coordinate system.

Because β, γ and the pixel coordinates x, y can be used synonymously, the individual intensity values of an image $\mathbf{I}$[1] are subsequently denoted by $\mathbf{I}(\vec{p}) = \mathbf{I}(\beta, \gamma)$. As the panoramic images are cyclic in horizontal direction, arbitrary β values are cyclically mapped onto the interval $[0, 2\pi)$. If the pixel coordinates x, y corresponding to a β, γ pair are non-integral, bilinear interpolation between the neighboring pixels is applied.

[1] A note on notation: Two-dimensional fields $\mathbf{I}$ of scalars or low dimensional vectors (images, depth maps, index maps) are denoted in bold letters. This is not true for "small" matrices (convolution kernels), fields of functions or fields of sets.

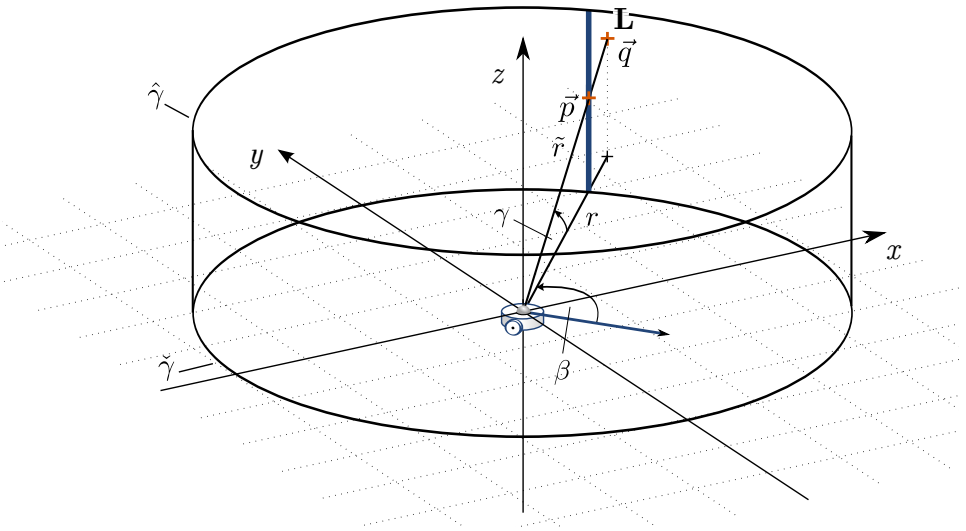


Figure 2.3: Spherical coordinate system used throughout the algorithm. The projected position $\vec{p}$ of the landmark L is given by the azimuthal angle β and the elevation angle γ. The landmark position is given by β, γ and the ground distance r.



(a) (b)

Figure 2.4: Comparison between classical spherical coordinates (a) and the modified spherical coordinates (b). In (a), $\tilde{r}$ is measured as the distance between the coordinate-system origin and the landmark point L (corresponds to the length of the hypotenuse in the above triangle). Landmarks with the same distance $\tilde{r}$ are thus assumed to be curved towards the camera (blue line). In the modified spherical coordinates depicted in (b), the distance r is measured as the ground distance to the landmark. Consequently landmarks with the same distance r shape a straight vertical line. As a side-effect, the elevation angle γ is not of interest when a landmark point is $\vec{q}$ projected onto the x-y-plane. In contrast to classical spherical coordinates, points with $\gamma = \pm 90°$ and $r = 0$ cannot be uniquely described in the modified spherical coordinates.

2.1.2 Assumptions

The algorithm contains several idealizing assumptions, which may not be fully valid in a real-world environment. These assumptions cover both the robot and its environment.

(1) *Perfect path integration:* The robot is capable of perfect path integration and can thus (given an initial offset) determine its current world position and orientation at any time step. There is no mobile robot which completely fulfills this assertion without an external position reference (Siegwart et al., 2011, chapter 3, "Mobile Robot Kinematics", p. 57ff). Yet the relative movement measured by the odometry over a short time window (as needed in the algorithm) and in "normal" operation (no wheel slippage) is assumed to be relatively error-free.

(2) *Limited degrees of freedom:* It is assumed that the robot is only moving in the x-y-plane and only rotates around its z-axis. Neither is there any change in the elevation, nor does the robot tilt around any other axis. Those assumptions are likely to be violated in the real-world. Especially in outdoor environments the ground is never perfectly even and thus both changes in elevation and tilt can occur. In indoor environments e.g. carpets or thresholds are possible problems. We may assume that any tilting is removed from the input images in a preprocessing step, yet this is out of the scope of this thesis.

(3) *Brightness constancy:* Neither are there changes in the global illumination (e.g. lights being switched on/off or the sun being covered by clouds) nor does the surface brightness change between the images. As pointed out by Fleet and Weiss (2006), this "brightness constancy" only holds in environments containing surfaces without specularities (Lambertian surfaces), "a distant point light source, no object rotations and no secondary illumination (shadows or inter-surface reflection)". Yet this brightness constancy assumption (especially the lack of specularities) is used in most optical flow algorithms and seems to work remarkably well in practice.

(4) *Static world:* The world is static, the robot is the only moving entity. This assumption may not be fulfilled in outdoor environments with vegetation and also in crowded indoor scenes. Moving objects may cause wrong depth reconstruction and thus cause false positive obstacles. Therefore the navigation algorithm which processes the obstacle points returned by the algorithm must be capable of discarding obstacles in its internal map that prove to be nonexistent.

The robustness of the algorithm regarding violation of assumption (1) is explicitly tested in section 5.3.6.

2.2 Depth reconstruction

The goal of the *depth reconstruction* stage is to estimate the ground distance r for the landmarks corresponding to each pixel $\vec{p} = (\beta, \gamma)^T$ in the current view image $\mathbf{CV}$. Consequently, the required result is a depth map $\mathbf{R}(\vec{p})$.

The general idea of the depth reconstruction is to exploit the parallax information present in the two images: Objects with a large distance only show a small displacement between the two images, whereas objects close to the camera show a relatively large displacement. This object displacement information is described by the motion field.

First a short introduction to optical flow estimation will be given. In conjunction with the motion parameters, the minimization problem underlying optical flow can then be reduced from a two-dimensional problem in each image point to a one dimensional problem. An approximative heuristic for this one dimensional problem is then given as minimum search in the so called *error stack*.

2.2.1 Short introduction to optical flow estimation

When moving a camera in a static 3D-scene, the projected object points move along certain two-dimensional paths. These paths depend on the camera motion, the object distance and position in the projected image. The velocity of each pixel at position x, y in the image and time t is referred to as *motion field* (Fleet and Weiss, 2006). The motion field is a theoretical construct which can only be calculated if the camera parameters, camera motion and the exact position of each landmark point $\vec{q}$ are known.

Optical flow is an estimation of the motion field for a set of images $\mathbf{I}_1$ and $\mathbf{I}_2$ captured at discrete times t_1 and t_2 during the camera movement (in general optical flow may be estimated from more than two images). By definition, optical flow cannot express the motion of the underlying objects, but of the light intensities created by these objects (Siegwart et al., 2011, chapter 4.2.7 "Motion and optical flow", p. 189). The global optical flow estimation problem can generally be expressed as finding a vector field $\vec{u} : \mathbb{R}^2 \longrightarrow \mathbb{R}^2$ which minimizes the following error function (adapted from Horn and Schunck, 1981):

$$E(\vec{u}) = \int_{\vec{p}} (\|\mathbf{I}_1(\vec{p}) - \mathbf{I}_2(\vec{p} + \vec{u}(\vec{p}))\|_1 + \lambda \|\nabla \vec{u}(\vec{p})\|) \; \mathrm{d}\vec{p} \,. \tag{2.1}$$

The first summand $\|\mathbf{I}_1(\vec{p}) - \mathbf{I}_2(\vec{p} + \vec{u}(\vec{p}))\|$ expresses the residual error after warping the second image according to the vector field. Note that minimization of this term is only valid due to assumptions (3) (brightness constancy) and (4) (static world).

The second term $\lambda \|\nabla \vec{u}(\vec{p})\|$ enforces a certain "smoothness" to the vector field under the regularization parameter λ. This regularization is crucial for

eliminating trivial solutions, such as simply mapping pixels with the same intensity to each other.

Although some approaches try to globally solve the minimization problem stated by equation 2.1 (e.g. Pock, 2008, chapter 5 "Real-time Optical Flow"), the solution presented here only minimizes the error for the local surroundings of each pixel in order to fulfill the real-time constraint on an embedded system.

2.2.2 Motion field equation

Given the motion parameters d, α, ψ, the motion field could actually be calculated if the distance to each landmark point $\mathbf{R}(\vec{p})$ was known:

Let $\vec{q} = (x, y, z)^T$ be the position of a landmark L in robot coordinates (origin at the robot center, x-axis aligned with the robot orientation). The projection $P : \mathbb{R}^3 \longrightarrow \mathbb{R}^2$ that projects $\vec{q}$ onto $\vec{p} = (\beta, \gamma)^T$ in the image plane is given as:

$$\vec{p} = P(\vec{q}) = \begin{pmatrix} \beta \\ \gamma \end{pmatrix} = \begin{pmatrix} \text{arctan2}(y, x) \\ \arctan\left(\frac{z}{\sqrt{x^2+y^2}}\right) \end{pmatrix} \tag{2.2}$$

The inverse $P^{-1} : \mathbb{R}^2 \longrightarrow \mathbb{R}^3$ with free parameter r (ground distance) is given as:

$$\vec{q} = P^{-1}(\vec{p}; r) = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = r \cdot \begin{pmatrix} \cos(\beta) \\ \sin(\beta) \\ \tan(\gamma) \end{pmatrix} \tag{2.3}$$

The motion vector $\vec{u}(\vec{p}; T, r)$ describing the relative movement of a pixel at position $\vec{p}$ between the current view and the snapshot image can now be calculated as

$$\vec{u}(\vec{p}; T, r) = P(T \cdot h(P^{-1}(\vec{p}; r))) - \vec{p} \tag{2.4}$$

where $T \in \mathbb{R}^{3\times4}$ is a homogeneous transformation matrix expressing the robot movement between **CV** and **SS** in world coordinates:

$$T = T(d, \alpha, \psi) = \begin{pmatrix} \cos(\psi) & \sin(\psi) & 0 & d \cdot \cos(\alpha) \\ -\sin(\psi) & \cos(\psi) & 0 & d \cdot \sin(\alpha) \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{2.5}$$

The sole purpose of the mapping $h : \mathbb{R}^3 \longrightarrow \mathbb{R}^4$ is to transform three dimensional points to homogeneous coordinates. h is defined as $(x, y, z)^T \mapsto (x, y, z, 1)^T$. See figure 2.5 for an interpretation of the motion field vectors.

2.2.3 Flow parametrization

By combining equation 2.4 from above and the error function in equation 2.1, the minimum search can be reduced from a two-dimensional problem in each

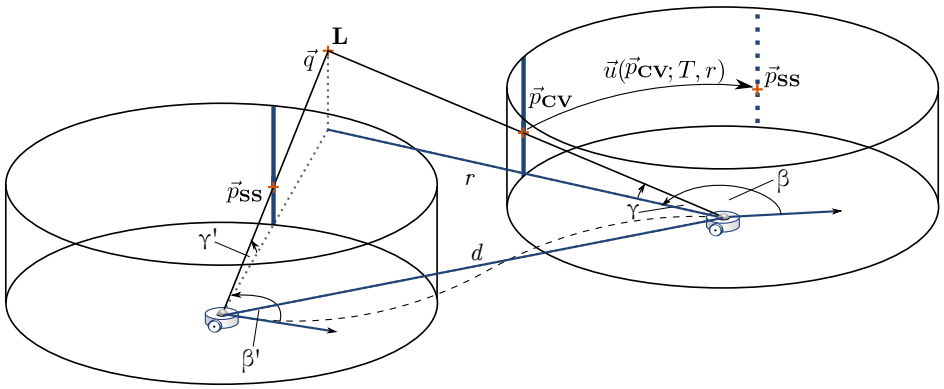


Figure 2.5: The motion field equation can be interpreted as follows: Given a point $\vec{p}_{\mathbf{CV}} = (\beta, \gamma)^T$ in **CV** (right), the motion field vector $\vec{u}(\vec{p}_{\mathbf{CV}}; T, r)$ predicts the position $\vec{p}_{\mathbf{SS}} = (\beta', \gamma') = \vec{u}(\vec{p}_{\mathbf{CV}}; T, r) + \vec{p}_{\mathbf{CV}}$ in which the same landmark point L would be visible in **SS** (right) under the assumption that L has the distance r.

pixel (finding an arbitrary flow vector $\vec{u}$) to a one dimensional optimization problem (finding the best matching depth value r for each pixel):

$$E(\mathbf{R}) = \int_{\vec{p}} (\|\mathbf{I}_1(\vec{p}) - \mathbf{I}_2(\vec{p} + \vec{u}(\vec{p}; T, \mathbf{R}(\vec{p})))\|_1 + \lambda \nabla \|\vec{u}(\vec{p}; T, \mathbf{R}(\vec{p}))\|) \; \mathrm{d}\vec{p} \quad (2.6)$$

Hence the depth field $\mathbf{R}(\vec{p})$ can be seen as an implicit characterization of the optical flow field under the motion parameters d, α and ψ. Note that these parameters are only known if assumption (1) (perfect path integration, see section 2.1.2) is valid — otherwise the motion field cannot be predicted.

2.2.4 Error stack calculation

The heuristic used in this approach searches local minima in the so called *error stack* $\mathbf{E}$ in order to estimate the depth field $\mathbf{R}(\vec{p})$. Each of the n layers $\mathbf{E}_i$ contains the residual error (first summand in equation 2.6) evaluated for a constant depth r_i:

$$\mathbf{E}_i(\vec{p}) = \|\mathbf{CV}(\vec{p}) - \mathbf{SS}(\vec{p} + \vec{u}(\vec{p}; T, r_i))\|_1 \text{ where } 0 \leq i < n \quad (2.7)$$

Figuratively speaking, this equation is equivalent to first calculating a warped snapshot image $\mathbf{SS'}_i$, that predicts "what the current view would look like" under the assumption that all image points had the same distance r_i. The residual error stored in each error stack layer is then given as $\mathbf{E}_i = \|\mathbf{CV} - \mathbf{SS'}_i\|_1$. See figure 2.6 for an example.

Note that some source pixel coordinates may not be in range when calculating the warped snapshot. The error value stored for the target pixel in the error stack layer should then be set to a large value (see figure 2.6).

Conventions for the distance series: The r_i are left open at this point, yet (by convention) any choice has to meet the following conditions:

1. The series r_i is strictly monotonically decreasing.
2. $r_0 = \infty$ (for $i = 0$ the motion vector length should equal zero for all pixels).
3. The step from r_i to r_{i+1} should increase the maximum horizontal flow vector length by one pixel. This way the pixels in $\mathbf{SS'}_i$ are at most shifted by one pixel in horizontal direction between two error stack layers, which allows to exploit the complete horizontal image resolution in the error stack calculation.
4. r_{n-1} should be chosen near the smallest possible distance between an obstacle and the camera.
5. There has to be some strictly monotonically decreasing, continuous function $r(j) : \mathbb{R}^+ \longrightarrow \mathbb{R}^+$, where $r(i) = r_i$ for all i.

A concrete choice for the r_i is presented in section 3.2.2.

Index related error function: Combining the error stack equation (2.7) and the parametrized optical flow equation (2.6) yields a minimization of the *index field* $\mathbf{J}(\vec{p})$, that describes the error stack index chosen for each pixel:

$$\begin{aligned} E(\mathbf{J}) &= \int_{\vec{p}} \left(\mathbf{E}_{\mathbf{J}(\vec{p})}(\vec{p}) + \lambda \|\nabla \vec{u}(\vec{p}; T, r(\mathbf{J}(\vec{p})))\| \right) \, \mathrm{d}\vec{p} \\ &\approx \int_{\vec{p}} \left(\mathbf{E}_{\mathbf{J}(\vec{p})}(\vec{p}) + \lambda \|\nabla \mathbf{J}(\vec{p})\| \right) \, \mathrm{d}\vec{p} \end{aligned} \tag{2.8}$$

At first glance $\nabla \vec{u}(\vec{p}; T, r(\mathbf{J}(\vec{p}))) \approx \nabla \mathbf{J}(\vec{p})$ seems to be a very rough approximation, yet due to the conventions presented above, the error stack index i is approximately proportional to the flow vector length. Consequently a smooth index field $\mathbf{J}(\vec{p})$ implies a smooth flow vector field $\vec{u}(\vec{p}; T, r(\mathbf{J}(\vec{p})))$.

2.2.5 Input image augmentation

Comparing individual pixels from a monochrome image $\mathbf{I}$ (as in the error stack equation 2.7) discards all spatial information that is encoded in the environment of each pixel.

In order to increase the information contained within a single pixel, one can increase its dimensionality. In the resulting feature space, each pixel can be seen as a local feature descriptor. The augmentation proposed here adds the image gradients in horizontal and vertical direction as a second and third component to each pixel, leading to the following equation for the augmented image $\tilde{\mathbf{I}}$:

$$\tilde{\mathbf{I}}(\vec{p}) = \left(\mathbf{I}(\vec{p}), \left.\frac{\partial \mathbf{I}}{\partial x}\right|_{\vec{p}}, \left.\frac{\partial \mathbf{I}}{\partial y}\right|_{\vec{p}} \right)^T \tag{2.9}$$

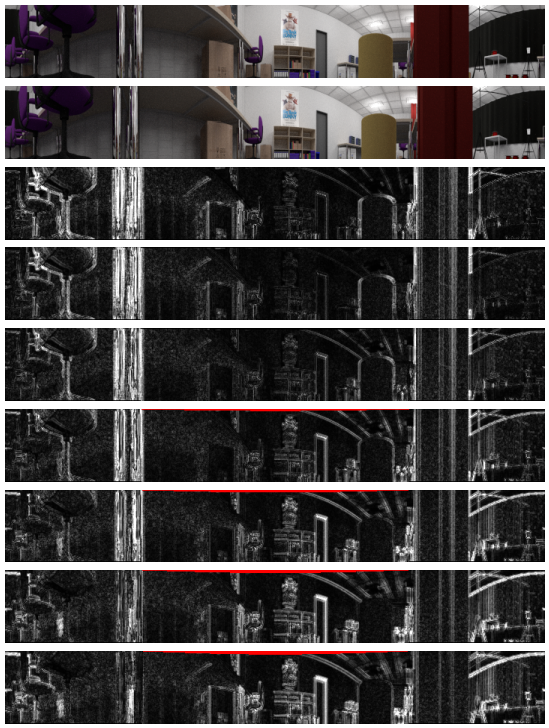
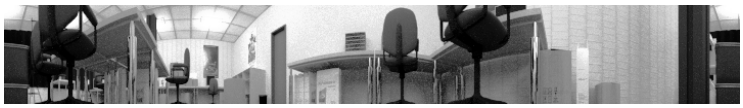


Figure 2.6: Visualization of a series of error stack layers. The error stack layers shown in the images (c) to (i) were calculated for the current view and snapshot shown in (a) and (b) (the original input images are monochrome). The brightness of the pixels in the error stack images corresponds to the error value. Note how certain objects "disappear" (have only a small error) in certain error stack images, like the silhouettes of the yellow cylinder in $\mathbf{E}_5$ or the table legs in $\mathbf{E}_6$, while the error for distant objects like the door frame increases with each layer. The image regions marked red form those for which no prediction can be made and thus have an infinite error.



(a) Monochrome image



(b) Combined pseudocolor image

Figure 2.7: Comparison between the original monochrome image and the pseudocolor image resulting from the transformation described in equation 2.9. The brightness is mapped to the blue channel, x- and y-derivatives to the red and green channels, respectively.

As a drawback, this augmentation reduces the overall spatial resolution: Estimating the derivative on discrete images (e.g. using the 3×3 Sobel kernel) incorporates at least three pixels. Moreover, the derivation increases the image noise (Siegwart et al., 2011, p. 200).

Figure 2.7 shows the result of equation 2.9 being applied to an input image. Note how the image edges (representing the regions with the largest gradient) are prominently marked in the resulting image. Thus "a sense of edges" is added to the algorithm.

2.2.6 Minimum error search

As stated in equation 2.8, estimating depth and optical flow is a matter of minimizing the global error by choosing appropriate error stack indices $\mathbf{J}(\vec{p})$ for each pixel $\vec{p}$. A naïve heuristic to estimating depth for each pixel would be to simply select the error stack layer that holds the global minimum error for each pixel:

$$\mathbf{J}(\vec{p}) = \arg\min_i \mathbf{E}_i(\vec{p}) \tag{2.10}$$

However, while being very simple to compute, this heuristic has a severe disadvantage: Computing the index without regarding neighboring pixels completely disregards the smoothness term. Additionally, as the error stack may not have a single pronounced minimum but multiple local minima with very small difference, the result is prone to noise.

Assuming that most objects cover more than a single pixel in the image and neighboring pixels have approximately the same distance, we can modify equation 2.10 to respect the surroundings of each pixel. Let $W_{\vec{p}}$ be a (possibly space-variant) two-dimensional filter kernel with radius r_W:

$$\mathbf{J}(\vec{p}) = \arg\min_i \left(\mathbf{E}_i * W_{\vec{p}}\right)_{\vec{p}} \tag{2.11}$$

The matrix $W_{\vec{p}}$ is subsequently called *error propagation matrix* as it "propagates" error values from neighboring pixels to its center pixel.

In large homogeneous image regions where multiple error stack layers contain very small error values (see the wall regions in figure 2.6), depth cannot be reconstructed (at least not without using global approaches) and equation 2.11 will very likely produce noise in such regions. In a later step, this noise might produce unwanted false obstacle points. Adding a linear bias $\tau \cdot i$ to the error function allows to classify homogeneous image regions as "very distant", in which case they do not hinder the obstacle detection:

$$\mathbf{J}(\vec{p}) = \arg\min_i \left(\left(\mathbf{E}_i * W_{\vec{p}}\right)_{\vec{p}} + \tau \cdot i\right) \tag{2.12}$$

Choices for $W_{\vec{p}}$: Simple space-invariant choices for $W_{\vec{p}} = W$ include a matrix of ones or a Gaussian filter kernel. A space-variant filter kernel $W_{\vec{p}}$ which takes object boundaries into account and thus only propagates error values for pixels belonging to the same object is presented in appendix B.1.

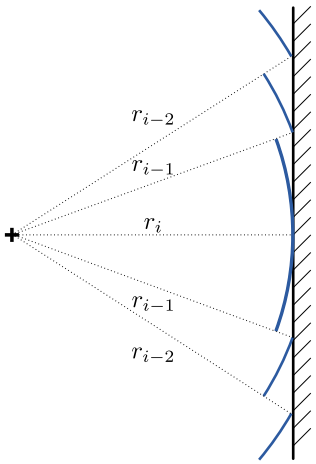


Figure 2.8: Local map (top-down view) showing the artifacts created by approximating a planar surface with a set of circles having discrete radii (blue).

2.2.7 Minimum error refinement

Equation 2.11 produces an index field $\mathbf{J}(\vec{p})$ consisting of discrete values. This is especially problematic if a planar surface needs to be reconstructed (see figure 2.8). Additionally, the discontinuity causes a high penalty in the smoothness term of the global error equation 2.1. In this section the initial index field $\mathbf{J}(\vec{p})$ as defined in equation 2.12 will be refined (allowing non-discrete solutions) and regularized (addressing the smoothness term) which leads to the final index field $\tilde{\mathbf{J}}(\vec{p})$.

Refinement: The discontinuity problem can be solved by approximating the error stack in each layer i and pixel $\vec{p}$ utilizing piecewise polynomials of second degree (quadratic; allows for trivial analytical solution), resulting in a set of functions $\tilde{E}_i(j;\vec{p})$ with the following properties:

$$\tilde{E}_i(j;\vec{p}) := \begin{pmatrix} j^2 & j & 1 \end{pmatrix} \cdot \vec{w}$$
$$\tilde{E}_i(i+j;\vec{p}) \stackrel{!}{=} \mathbf{E}_{i+j}(\vec{p}) + \tau \cdot (i+j) \quad \forall \quad j \in \{-1,0,1\}$$

Solving this set of linear equations yields the following weight vector $\vec{w}$:

$$\vec{w} = \begin{pmatrix} w_2 \\ w_1 \\ w_0 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} & -1 & \frac{1}{2} \\ -i-\frac{1}{2} & 2i & -i+\frac{1}{2} \\ \frac{1}{2}(i^2+i) & -i^2+1 & \frac{1}{2}(i^2-i) \end{pmatrix} \cdot \begin{pmatrix} \mathbf{E}_{i-1}(\vec{p}) + \tau \cdot (i-1) \\ \mathbf{E}_i(\vec{p}) + \tau \cdot i \\ \mathbf{E}_{i+1}(\vec{p}) + \tau \cdot (i+1) \end{pmatrix} \tag{2.13}$$

If one of the error layers $\mathbf{E}_{i-1}$ or $\mathbf{E}_{i+1}$ does not exist, a linear approximation is chosen instead.

Regularization: The regularization term penalizes large variances in the index field. For a single reference point $\vec{q}$, the quadratic regularization function $R(j;\vec{q})$ is given as a polynomial

$$R(j;\vec{q}) = \lambda \cdot (\mathbf{J}(\vec{q}) - j)^2$$

which quadratically increases the penalty according to the difference between the index j in the current point $\vec{p}$ and the index of the reference point $\vec{q}$.

Final minimization formula: Again the error function is not minimized for every individual pixel, but for a weighted region surrounding each pixel as defined by the convolution kernel $W_{\vec{p}}$. For compact notation, let $A(\vec{p}) := \{\vec{p} + (\beta(x), \gamma(y))^T \mid -m \leq x, y \leq m\}$ denote the pixel coordinates beneath the convolution window at position $\vec{p}$ and let $\omega_{\vec{p}}^{\vec{q}}$ for $\vec{q} \in A(\vec{p})$ denote the corresponding value in the convolution matrix $W_{\vec{p}}$. The refinement of the initial (integral) index field $\mathbf{J}$, leading to the refined, real-valued index field $\tilde{\mathbf{J}}$, is then given as

$$\tilde{\mathbf{J}}(\vec{p}) = \underset{\mathbf{J}(\vec{p})-1 \leq j \leq \mathbf{J}(\vec{p})+1}{\operatorname{argmin}} \sum_{\vec{q} \in A(\vec{p})} \omega_{\vec{p}}^{\vec{q}} \cdot \left(\tilde{E}_{\mathbf{J}(\vec{p})}(j;\vec{q}) + R(j;\vec{q}) \right) \tag{2.14}$$

As the sum only contains quadratic polynomials, the minimum can be calculated analytically.

Note that the refinement step can be repeated multiple times. The indices in $\tilde{\mathbf{J}}(\vec{p})$ are then rounded to the nearest integers and treated as the initial index field $\mathbf{J}(\vec{p})$.

Once the final index map $\tilde{\mathbf{J}}(\vec{p})$ has been built, the index-to-depth function $r(j)$ can be used to obtain the desired depth map:

$$\mathbf{R}(\vec{p}) = r(\tilde{\mathbf{J}}(\vec{p}))$$

2.3 Obstacle detection

The simple heuristic used in the depth reconstruction was designed for fast evaluation and does by no means guarantee to find a perfect depth estimation. Additionally, noise in the images or the motion parameters d, α, ψ (and thus a violation of the assumptions presented in section 2.1.2) inevitably leads to noise in the depth map.

For obstacle detection, a local environment map (top-down projection onto the x-y-plane) of the reconstructed landmark points is created. The aim of this map is to improve the depth reconstruction result by combining depth maps resulting from multiple depth reconstruction steps. It then allows the selection of actual obstacle points which form the result of the overall algorithm.

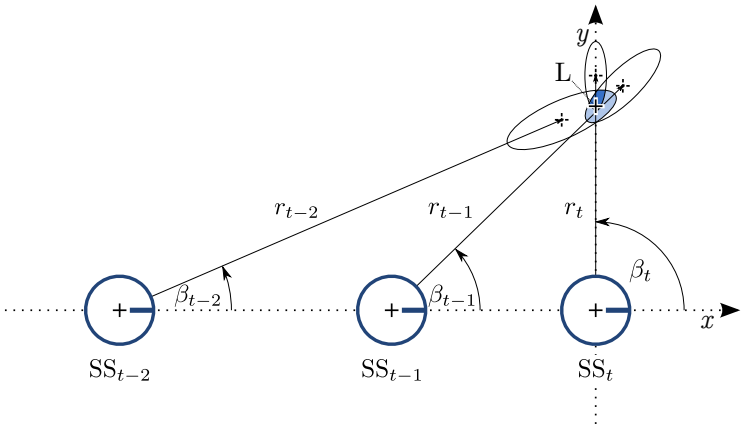


Figure 2.9: The Gaussian distributions for reconstructed landmark positions from multiple time steps can overlap in such a way that their sum forms a distinct maximum at (or at least near) the actual landmark position L. Consequently, whenever a value above a certain threshold appears in the map, the underlying point is emitted as an obstacle. Note that the sum of two overlapping (corresponding to the intersection of certain contour lines) Gaussian distributions does not necessarily yield a maximum at the landmark position as suggested by the drawing. Yet the distributions used in this approach have a relatively high distance variance σ_r in comparison to their direction variance σ_β. Thus overlapping Gaussians implicitly triangulate the landmark and a maximum is probable.

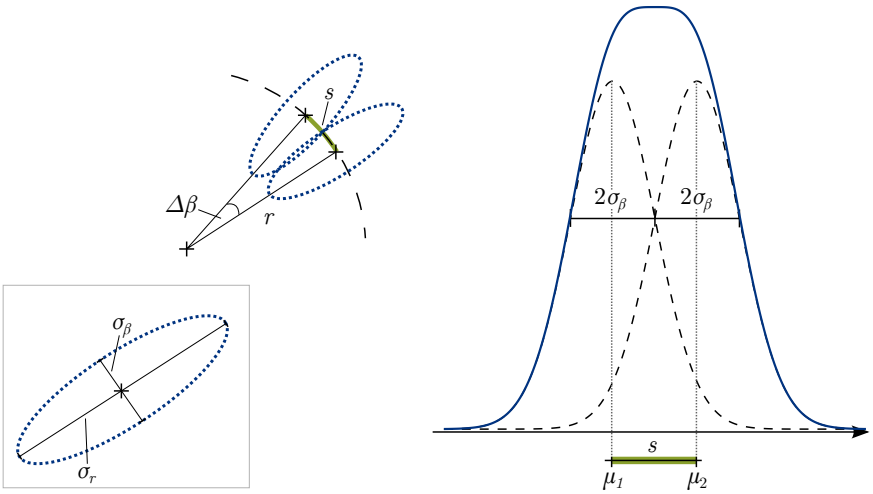


Figure 2.10: Calculation of the variance σ_β. Adjacent Gaussian distributions with the same radius r should form a nearly uniform distribution between μ_1 and μ_2. See also appendix A.1.2.

2.3.1 Probabilistic landmark model

The reconstructed landmark points $(\beta, \gamma, r)^T$ can be projected onto the x-y-plane by just omitting the γ angle, as r was chosen as the ground distance (see section 2.1.1 and the corresponding figure 2.4). The projected landmark point $\vec{q}$ is hence specified by the polar coordinates $\vec{q} = (\beta, r)^T$.

As stated above, the *estimated* landmark position $\vec{q}_E$ as returned from the depth reconstruction is prone to noise. Assume a Gaussian distribution with mean at $\vec{q}_E$ for the probability of the *real* landmark position $\vec{q}_R$:

$$\vec{q}_R \sim \mathcal{N}\left(\vec{q}_E, \Sigma(\vec{q}_E)\right) \text{ where } \Sigma(\beta, r) := \begin{pmatrix} \sigma_\beta^2 & 0 \\ 0 & \sigma_r^2 \end{pmatrix} \tag{2.15}$$

Let $\mathcal{C} : (\beta, r)^T \mapsto r \cdot (\cos(\beta), \sin(\beta))^T$ denote the transformation from polar to Cartesian coordinates. As an approximation, equation 2.15 can be written for Cartesian coordinates as:

$$\mathcal{C}(\vec{q}_R) \sim \mathcal{N}\left(\mathcal{C}(\vec{q}_E), \Sigma_{\mathcal{C}}(\vec{q}_E)\right) \tag{2.16}$$

where $\Sigma_{\mathcal{C}}$ is the corresponding covariance matrix in Cartesian space given as (see A.1.1 for the derivation):

$$\Sigma_{\mathcal{C}}(\beta, r) = \begin{pmatrix} (\sigma_r \cos(\beta))^2 + (r\sigma_\beta \sin(\beta))^2 & \cos(\beta)\sin(\beta)(\sigma_r^2 - (r\sigma_\beta)^2) \\ \cos(\beta)\sin(\beta)(\sigma_r^2 - (r\sigma_\beta)^2) & (\sigma_r \sin(\beta))^2 + (r\sigma_\beta \cos(\beta))^2 \end{pmatrix} \tag{2.17}$$

Derivation of the distribution variances: Until now the variances in direction σ_β and in distance σ_r were left open.

The distance variance σ_r depends on the choice of the yet unknown index-to-depth function $r(j)$ and will be introduced along with the concrete choice in section 3.2.2.

However, σ_β only depends on the horizontal image resolution $\Delta\beta := {}^{2\pi}/_{w}$ and can be derived by demanding that two adjacent Gaussian distributions (with $|\beta_1 - \beta_2| = \Delta\beta$) should overlap in such a way that their sum forms a near-uniform distribution in the region between β_1 and β_2. This choice is motivated by the fact that the probability of an obstacle point being located somewhere between two image columns β_1 and β_2 is the same as the probability of an obstacle point being located exactly on a certain image column β.

One can show (see appendix A.1.2) that σ_β is now given as

$$\sigma_\beta = \frac{\Delta\beta}{2} = \frac{\pi}{w} \,. \tag{2.18}$$

Multiplying σ_β with r produces the Cartesian equivalent of the angular variance, as $\Delta\beta \cdot r$ denotes the length s of the curve segment between β_1 and β_2 (see figure 2.10).

2.3.2 Distribution accumulation and obstacle detection

It is hoped that the accumulated Gaussian distributions (forming the so called *density values*) resulting from subsequent depth reconstruction steps produce pronounced maxima at the real landmark positions. This is possible as the probability distributions have rather small variance in angular direction, but a relatively large variance in distance direction. Thus intersecting Gaussian distributions implicitly triangulate the result (see figure 2.9).

Let $\mathbf{M}$ be a discrete Cartesian map matrix that spans a region of $2 \cdot r_M \times 2 \cdot r_M$ meters (where r_M is radius of the map), and let the origin $\mathbf{M}(0,0)$ correspond to the center of the map matrix. The obstacle detection algorithm is given as follows:

Map position update: Rotate and shift the map matrix such that the camera is always located at the map origin and the angle $\beta = 0$ is always relative to the current translatory motion direction α.

Map decay: In order to ensure the locality of the map, all old density values are multiplied with a decay factor δ depending on the translatory distance d:

$$\mathbf{M}(x,y) \leftarrow \mathbf{M}(x,y) \cdot \delta \;\forall\, x,y \text{ where } \delta = \exp(-\varepsilon \cdot d)\ . \tag{2.19}$$

The free parameter ε is used as a conversion factor between d and δ. This way old density values are gradually discarded from the map as the robot moves through the world.

Distribution accumulation: For each reconstructed and projected landmark point $\vec{q}_k = (\beta_k, r_k) \in Q$ the corresponding probability distributions are accumulated and added to the old density values in the discrete map:

$$\mathbf{M}(x,y) \leftarrow \mathbf{M}(x,y) + \sum_{\vec{q} \in Q} \mathcal{N}\left((x,y)^T \mid \mathcal{C}(\vec{q}), \Sigma_{\mathcal{C}}(\vec{q})\right) \;\forall\, x,y \tag{2.20}$$

Obstacle detection: If multiple distributions overlap in such a way that a distinct maximum is formed in the obstacle map, it is likely for this maximum to correspond to an actual landmark point. As outlined in figure 2.11, these distinct maxima are detected by generating a thresholded map

$$\tilde{\mathbf{M}}(x,y) \leftarrow \begin{cases} 1 & \mathbf{M}(x,y) > \eta \\ 0 & \text{otherwise} \end{cases} \quad \forall\, x,y$$

where η is the threshold value. In a subsequent step, 4-connected components in $\tilde{\mathbf{M}}$ are detected (using the one-pass algorithm presented in Wikipedia, 2013b). If they exceed a certain minimum size their centroid is returned as an obstacle point.



(a) Color coded local environment map

(b) Thresholded map

(c) Resulting obstacle points (orange)

FIGURE 2.11: Overview of the obstacle detection process. (a) shows a color coded environment map $\mathbf{M}$ (black and blue correspond to small, green and yellow to medium and red to high density values), (b) shows the thresholded map $\tilde{\mathbf{M}}$ and (c) the resulting obstacle points which form the result of the algorithm. The arrows in (a) and (c) indicate the robot orientation.

Obstacle point fragmentation and occupancy map: As an extension to the above, large connected components are fragmented into multiple obstacle points. This is accomplished by dividing each connected component into square grid cells, counting the pixels in each cell and — if a threshold is exceeded — returning the centroid of the pixels in each grid cell as an obstacle point.

In order to prevent the algorithm from repeatedly returning the same obstacle points, obstacles are marked in a low resolution *occupancy map* $\mathbf{M_O}$. The size of $\mathbf{M_O}$ is chosen in such a way that each pixel corresponds to one of the square grid cells mentioned above. The map position update is also applied to $\mathbf{M_O}$ and obstacles at positions marked as occupied in $\mathbf{M_O}$ are not returned.

Chapter 3

Algorithm extensions

The abstract algorithm presented in the previous chapter could be naïvely implemented, yet doing so will most likely result in an implementation that does not fulfill the real-time constraint present on a mobile robot.

The extensions introduced in this chapter attempt to lower the computational requirements. They include a *coarse-to-fine refinement* scheme which locally limits both the error stack layer count and the minimum error search space, as well as the calculation of so called *flow templates* which contain precalculated motion field vectors.

3.1 Coarse-to-fine refinement

This section introduces an optimization to the depth reconstruction algorithm which aims at fulfilling two — seemingly contradictory — goals: On the one hand preventing mismatches in the depth reconstruction caused by temporal aliasing and on the other hand lowering the runtime requirements.

First some further motivation and the general idea will be given, followed by remarks regarding the concrete implementation.

3.1.1 Motivation: temporal aliasing

Reconstructing optical flow is prone to *temporal aliasing*[1]: As a consequence of the Nyquist-Shannon sampling theorem (see Wikipedia, 2013c), the motion of a regular pattern with a maximum spatial frequency ξ moving at a certain velocity v can only be uniquely reconstructed from a time series of images, if the sampling time T fulfills the following inequality

$$T \leq \frac{1}{2B} = \frac{1}{2v\xi} \tag{3.1}$$

[1] A well known example for temporal aliasing is often seen in movies, where the so called *wagon-wheel effect* causes wagon wheels to be seemingly spinning into the wrong direction (Wikipedia, 2013a).

where $B = v \cdot \xi$ is the maximum temporal frequency in the system.

In the optical flow case, all three variables T, ξ and v are fixed: T is the maximum time between two snapshots, the spatial frequency ξ is bounded by the image resolution and the maximum landmark velocity v is defined by the minimum landmark distance and the maximum observer velocity.

For high-resolution images, the theoretically required sampling frequency turns out to be impracticably high. Conversely operating with a T lying above the Nyquist-Shannon rate results in a multitude of possible solutions, as soon as a regular pattern becomes visible. Any of these solutions may produce a pronounced minimum in the global error function 2.1.

3.1.2 Working around temporal aliasing: coarse-to-fine refinement

Coarse-to-fine refinement as described by Fleet and Weiss (2006, p. 254f., section "Temporal Aliasing and Coarse-To-Fine Refinement") is a solution to this problem:

1. Start with the coarsest scale of a Gaussian image pyramid — thus the maximum spatial frequency ξ is small and T is expected to be sufficiently large.

2. Perform optical flow-estimation on this resolution level resulting in a vector field $\vec{u}$.

3. Warp the next resolution level in the image pyramid according to $\vec{u}$. Subsequently the velocities v are much smaller in the warped image and will outweigh the higher spatial resolution ξ at this resolution level.

4. Calculate the residual optical flow and refine the last flow vector field.

5. Repeat step 3. until the highest resolution level is reached.

6. Add up the flow vectors from all stages.

A major problem of this approach is that a poor initial estimate cannot be corrected in subsequent steps. Yet from a theoretical point of view, lowering ξ and/or v is the only possible solution for temporal aliasing if a fixed T is given.

3.1.3 Integration of coarse-to-fine refinement into the depth reconstruction framework

Temporal aliasing is also a problem in the depth reconstruction algorithm and its implicit optical flow calculation. The ambiguous minima may result in high velocity flow vectors and falsely classified "near" objects, which in



(a) Without coarse-to-fine refinement

(b) With coarse-to-fine refinement

(c)

FIGURE 3.1: Effects of coarse-to-fine refinement. Notice the mismatches originating from temporal aliasing in the lower left image region of (a) (book shelf). In (b) these mismatches are mostly eliminated by applying the coarse-to-fine refinement. Additionally, the pruning introduced with the coarse-to-fine refinement excludes "uninteresting" image regions from the depth reconstruction. As seen in the book shelf region in (b) this pruning might sometimes be oversensitive. (c) Shows the color-to-distance mapping used in the images.

return cause false-positive obstacle points. See figure 3.1(a) for an example of mismatches and the resulting depth in repetitive regions.

The coarse-to-fine approach described above is applied to the depth reconstruction algorithm as follows:

1. Calculate a Gaussian pyramid for both the snapshot and the current view image. Let the number of pyramid levels be n.

2. Pyramid level $\ell = 1$: Perform a complete depth reconstruction on the coarsest pyramid level resulting in the depth field $\mathbf{R}_1(\vec{p})$.

3. Pyramid level $1 < \ell \leq n$: Perform the depth reconstruction on the next finer pyramid level. For every pixel $\vec{p}$, limit the search range $I_{\vec{p}}$ in the minimum error search to an interval around the last depth as stored in $\mathbf{R}_{\ell-1}(\vec{p})$. This search range (given as error stack indices) could for example be chosen as

$$I_{\vec{p},\ell} = \{\lceil i_\ell(\mathbf{R}_{\ell-1}(\vec{p})) - \kappa \rceil, \ldots, \lfloor i_\ell(\mathbf{R}_{\ell-1}(\vec{p})) + \kappa - 1 \rfloor\} \cap \left\{0, \ldots, i_\ell^{MAX}\right\} . \tag{3.2}$$

The parameter κ defines the breadth of the search range around the last index. The function $i_\ell(r)$ returns the error stack index for a certain depth

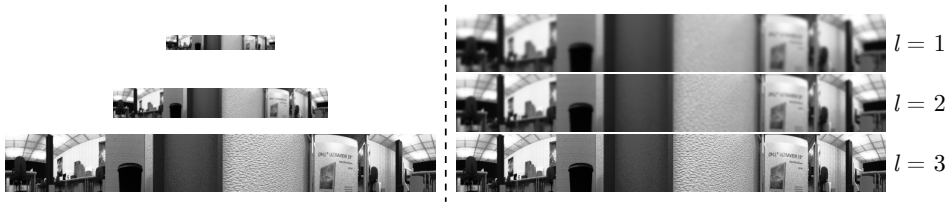


Figure 3.2: Example of the resolution pyramid used in the algorithm. (a) shows the subsampled images for each pyramid level, while (b) contains the low-pass filtered images. Only octaves of the scale space are chosen, resulting in both the low-pass frequency and the image resolution (according to the Nyquist-Shannon sampling theorem) being doubled in each step.

value r and a pyramid level index ℓ. Note that $i_\ell(r)$ varies for all ℓ, as the image resolution changes (due to subsampling) and so does the count of error stack layers i_ℓ^{MAX}. This results from the fact that the maximum horizontal shift between two error stack layers is always set to one pixel (section 2.2.4), but the pixel resolution differs between the pyramid levels.

Note that this approach differs from the original coarse-to-fine refinement scheme: The individual pyramid levels are not warped according to the last flow vector field. This warping was performed in order to reduce the maximum landmark velocity and to conform with the Nyquist-Shannon sampling theorem. Yet limiting the search range in the one dimensional minimum error search — and thus the maximum flow vector length — has the same effect.

3.1.4 Implementation remarks

This section contains a few remarks regarding the actual coarse-to-fine refinement implementation.

Pyramid construction: The first step in the coarse-to-fine refinement is to construct a Gaussian resolution pyramid for both the snapshot and the current view image. In contrast to the complete scale space pyramid used in SIFT (Lowe, 2004), only octaves are selected from the scale space, as depicted in figure 3.2. The scaled images are constructed by low-pass filtering the input images with a 5×5 binomial convolution kernel and then omitting even rows and columns. As a result, a subsampled image with halved width and height is generated[2]. This process is repeated until the smallest resolution level is reached. In the current implementation three pyramid levels are generated, reaching from an input resolution of 720×96 pixels down to 180×24 pixels.

[2]This behavior is implemented in the OpenCV `pyrDown` function.

It is important to note that the first order derivatives of the images used for input dimensionality augmentation (see section 2.2.5) have to be separately calculated for each resolution level and must not be downsampled along with the input image. This approach allows to reveal large scale gradients in the image and hence improves the matching between **SS** and **CV**.

Simplified index range calculation: Due to the convention in section 2.2.4 to choose a maximum horizontal shift per error stack layer of one pixel, halving the width/height between two resolution pyramid levels also halves the count of error stack layers. An error stack index $i_{\ell-1}$ in pyramid level $\ell - 1$ corresponds to the same distance r as the error stack index $2 \cdot i_{\ell-1}$ in pyramid level ℓ. Consequently the index range equation 3.2 can be reformulated as

$$I_{\vec{p}} = \{2 \cdot \mathbf{J}_{\ell-1}(\vec{p}) - \kappa, \ldots, 2 \cdot \mathbf{J}_{\ell-1}(\vec{p}) + \kappa - 1\} \cap \{0, \ldots, i_\ell^{MAX}\} \; . \tag{3.3}$$

Note that the unrefined index field **J** is used, as the rounding in equation 3.2 would undo the effects of the refinement. Consequently the index refinement and the index-to-depth steps can be skipped for all resolution layers $\ell < n$.

The concrete implementation additionally guarantees $|I_{\vec{p}}| = 2\kappa$. This is ensured in a correction step performed after clipping to $\{0, \ldots, i_\ell^{MAX}\}$ (not represented in the equation for sake of simplicity, see figure 3.3 instead).

Index range representation: Locally limiting the search range of each pixel turns out to be problematic from a runtime point of view. Besides the memory footprint needed for storing the index ranges it would also hinder a SIMD implementation of the minimum error search. By definition, SIMD instructions can only be used if the same operation is performed on blocks of data — which is certainly not true if each pixel potentially needs to be compared to a different range of reference values. Even in a non-SIMD implementation, the additional branching and memory access could potentially outweigh the benefit of being capable to skip some (relatively cheap) comparisons.

A solution is to store the index range column- instead of pixelwise. The index range I_β for column β is then calculated as

$$\breve{I}_\beta = \bigcup_\gamma I_{(\beta,\gamma)} \; , \tag{3.4}$$

$$I_\beta = \{\min(\breve{I}_\beta), \ldots, \max(\breve{I}_\beta)\} \; . \tag{3.5}$$

Using this index range for all pixels in a column may lead to problems if the indices are subject to high variance (see figure 3.3 for an example). In this case, there is no restriction in the flow vector length and the column is open to the problems caused by temporal aliasing.

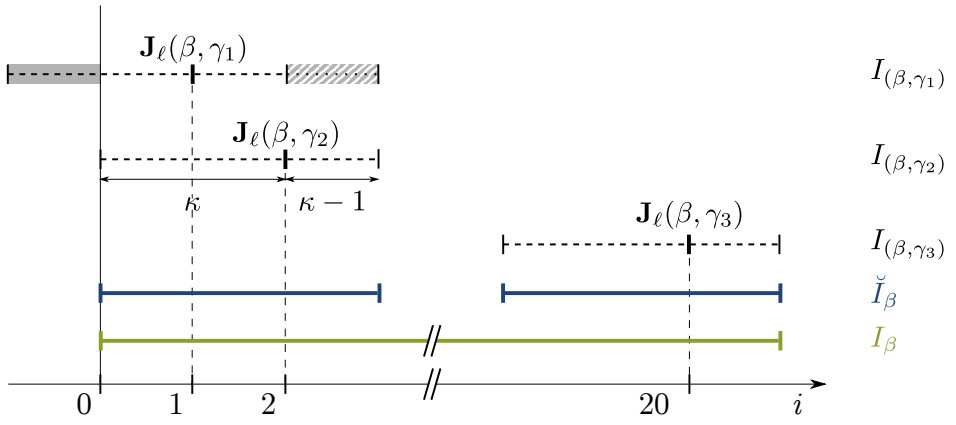


Figure 3.3: The index range calculation as stated in equations 3.4 and 3.5. The individual pixel index intervals from column β are joined, resulting in the auxiliary set $\breve{I}_\beta$. The minimum and maximum values of $\breve{I}_\beta$ are then used to build the final column index range I_β. The gray areas surrounding the interval at pixel (β, γ_1) visualize the "breadth constancy" employed in the implementation (for sake of simplicity not present in the equations). Note how high variances in the indices can cause very large index ranges.

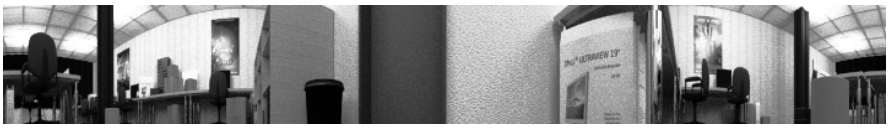
Runtime improvements and pruning: The coarse-to-fine refinement leads to a significant reduction of the runtime as the error stack layers only need to be calculated in the regions actually required for higher resolution levels.

An additional runtime optimization is given by pruning "uninteresting" parts of the image from the error stack calculation and the minimum error search (see also figure 3.4). "Uninteresting" hereby refers to image regions containing no close obstacles. This can be easily accomplished by excluding all image columns β from the calculation where $\mathbf{J}_\ell(\beta, \gamma) = 0 \; \forall \; \gamma$.

Possibly undesired effects of this pruning can be seen in figure 3.1(b), where on the one hand noise is greatly reduced, but on the other hand interesting parts (although with a relatively high distance) are also pruned.

3.2 Flow template calculation

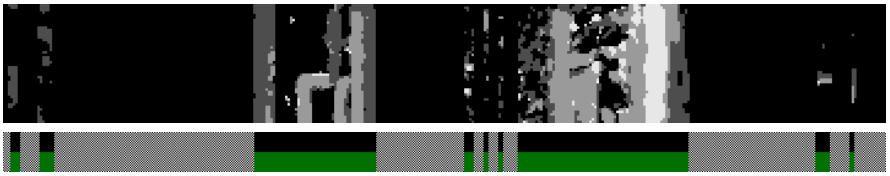
The most important concept in the optimized implementation of the abstract depth reconstruction algorithm is the computation of so-called "flow templates". In essence these contain the precomputed motion field vectors which are needed to warp the snapshot images.



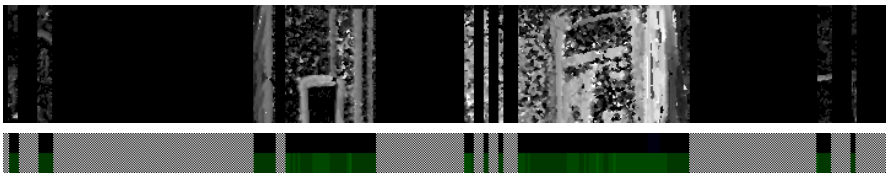
(a) Currentview image **CV**



(b) Indices for $\ell = 1$, 180×24 px, 8 error stack layers



(c) Indices for $\ell = 2$, 360×48 px, 16 error stack layers

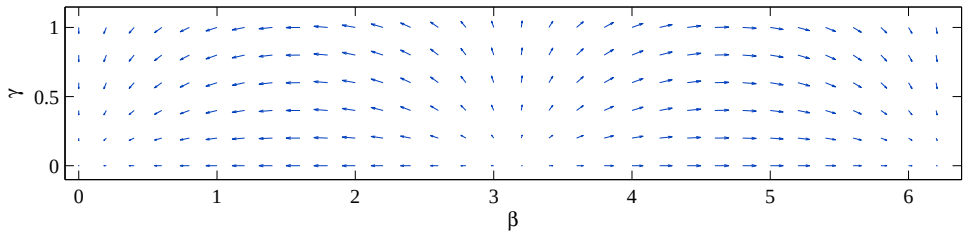


(d) Refined indices for $\ell = 3$, 720×96 px, 32 error stack layers

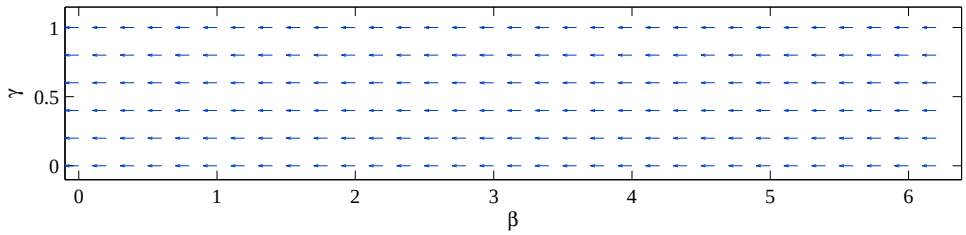
FIGURE 3.4: Exemplary visualization of the coarse-to-fine refinement. (a) shows the reference current view image. (b)-(d) show the indices resulting from the depth reconstruction algorithm in varying resolutions. In contrast to the last resolution level in (d), the index fields in (b) and (c) have no index refinement applied to them. The small images visualize the index range of each individual column. The first (in this case black) row depicts the start index, the second (green) row the end index encoded as brightness relative to the total count of error stack layers. Note how the relative percentage of used error stack layers decreases with each resolution level. The gray/checkered regions depict pruned columns.

3.2.1 Motion field parameter space reduction

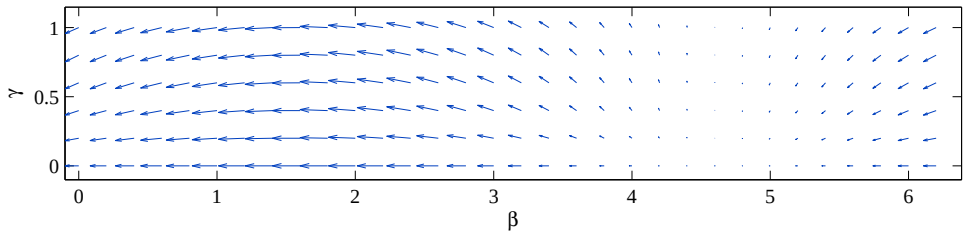
Calculating the motion field $\vec{u}(\vec{p}; T(d, \alpha, \psi), r)$ for each set of motion parameters d, α, ψ, depth step r_i and pixel $\vec{p} = (\beta, \gamma)$ as stated in the abstract algorithm is not reconcilable with the real-time constraint, as the motion field equation contains numerous trigonometric functions which are relatively slow to evaluate due to their transcendental nature. However, precalculating the motion field vectors in the given form is infeasible as a six-dimensional parameter space $(\beta, \gamma, d, \alpha, \psi, r)$ has to be covered. Consequently the parameter



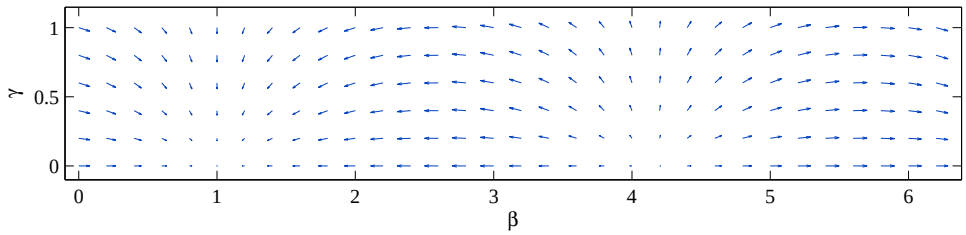
(a) Plot of $\vec{u}(\vec{p}; T(d, \alpha, \psi), r)$ for $d = 0.1$ m, $\alpha = 0.0$, $\psi = 0.0$, $r = 1.0$ m



(b) Plot of $\vec{u}(\vec{p}; T(d, \alpha, \psi), r)$ for $d = 0.0$ m, $\alpha = 0.0$, $\psi = 0.1$, $r = 1.0$ m



(c) Plot of $\vec{u}(\vec{p}; T(d, \alpha, \psi), r)$ for $d = 0.1$ m, $\alpha = 0.0$, $\psi = 0.1$, $r = 1.0$ m



(d) Plot of $\vec{u}(\vec{p}; T(d, \alpha, \psi), r)$ for $d = 0.1$ m, $\alpha = 1.0$, $\psi = 0.0$, $r = 1.0$ m

Figure 3.5: Plots of the motion field $\vec{u}(\vec{p}; T(d, \alpha, \psi), r)$ for varying parameters. (a) Shows a pure translatory movement, with focus of expansion (FOE) at $\beta = 0$ and focus of contraction (FOC) at $\beta = \pi$. (b) Shows a pure rotational movement. (c) Shows the combination of both the translatory and the rotational movement. Note how the to motion fields from (a) and (b) are just additively combined. (d) Shows a pure translatory movement with $\alpha \neq 0$. Note how the FOE at $\beta = \alpha$ and FOC at $\beta = \alpha + \pi$ correspond to the movement direction defined by α. This is equivalent to a coordinate transformation in β.

space dimensionality needs to be reduced.

The parameters d and r are not independent. In fact, the flow vector field only depends on the ratio of the two parameters. A graphical motivation can be seen in figure 2.5, where the flow vector depends on the triangle formed by r and d. Scaling this triangle does not change the flow vector. A proof is given in appendix A.2.1. Thus a new parameter ρ can be introduced which expresses the ratio between the two values:

$$\vec{u}(\vec{p}; T(d, \alpha, \psi), r) = \vec{u}(\vec{p}; \alpha, \psi, \rho) \text{ where } \rho := \frac{d}{r} . \tag{3.6}$$

This substitution was originally presented in the related warping algorithm for visual homing (Franz et al., 1998).

Eliminating the rotational movement: The motion parameters α and ψ describe the rotational movement that was executed by the robot. It can be shown (see A.2.2 for a proof), that varying α applies an affine coordinate transformation to the β-component and ψ is an additive term on the β component:

$$\vec{u}(\vec{p}; \alpha, \psi, \rho) = \vec{u}(\vec{p} - (\psi + \alpha, 0)^T; \rho) - (\psi, 0)^T . \tag{3.7}$$

See figure 3.5 for an example. This equation can be interpreted as elimination of the rotational movement by panoramic image alignment:

- ψ expresses the change in orientation between **SS** and **CV**. Due to the nature of the panoramic images, this change in orientation can be eliminated by a simple horizontal translation of the snapshot image **SS**.
- α denotes the movement direction and by implication the position of the focus of expansion (FOE) in the image. Accordingly, α can be set to zero, if both the snapshot **SS** and current view **CV** are shifted by α in such a way that the observed FOE and FOC (focus of contraction) positions in **CV** match their positions in the non-rotated motion field.

Hence using correct image alignment allows to reduce the parameter space by setting $\alpha = \psi = 0$. The resulting motion field equation becomes:

$$\vec{u}((\beta, \gamma)^T; \rho) = \begin{pmatrix} \arctan2\left(\sin(\beta), \cos(\beta) + \rho\right) - \beta \\ \arctan\left(\frac{\tan(\gamma)}{\sqrt{(\cos(\beta)+\rho)^2 + \sin(\beta)^2}}\right) - \gamma \end{pmatrix} = \begin{pmatrix} \beta' - \beta \\ \gamma' - \gamma \end{pmatrix} . \tag{3.8}$$

It should be noted that the β-component of $\vec{p}_{\mathbf{SS}} = (\beta', \gamma')^T$ does not depend on γ: For a constant ρ, all pixels in a column in **SS** are mapped to a single column in **CV**, yet a non-linear scaling in γ-direction is applied. This property will allow the linear interpolation needed for calculating the warped $\mathbf{SS'}_i$ to be separated into a horizontal and a vertical interpolation step (see section 4.2.2).

3.2.2 Parameter selection, index to depth conversion and distance variance

Flow template definition: In the last two steps the parameter space was reduced to three parameters β, γ and ρ. This allows precalculating the motion field vectors for each pixel $\vec{p} = (\beta, \gamma)^T$ and a set of n values of ρ_i where $0 \leq i \leq n-1$. The snapshot images can then be warped according to these motion vectors, subsequently called *flow templates*. A single flow template at index i, $\mathbf{U}_i$, is a $h \times w$ matrix containing two-dimensional vectors as its entries:

$$\mathbf{U}_i(\vec{p}) = \vec{u}(\vec{p}; \rho_i) \; . \tag{3.9}$$

Selection of ρ_i: In section 2.2.4 it was stated that the r_i should be chosen in such a way that the maximum horizontal shift between two consecutive motion fields is about one pixel. This was motivated by the ability to exploit the complete horizontal resolution that is available in the images.

In the new motion field equation, the motion vector length in pixels solely depends on the parameter ρ. Thus the ρ_i have to be selected with the same consideration. As shown in appendix A.2.3 choosing, ρ_i as a multiple of the horizontal angular image resolution (with w being the image width in pixels)

$$\rho_i = \frac{2\pi}{w} \cdot i \tag{3.10}$$

approximately fulfills this requirement for small i. Note that for large i this means that the "one pixel per step" condition might by violated. In practice d should always be significantly smaller than r, as the algorithm is designed for short distances between consecutive snapshot images. Large d would imply near landmarks to be viewed from two completely different directions in the **SS**, **CV** pair.

Index to depth conversion: The distance function $r(j)$ needed for conversion from continuous index- to world-space is now given as

$$r(j; d) = \frac{d}{\rho(j)} = \frac{d}{j} \cdot \frac{w}{2\pi} \; . \tag{3.11}$$

Calculation of σ_r: The choice of σ_r was left open in section 2.3.1. Now that the index-to-depth function $r(j)$ has been selected, the corresponding variance σ_r can be calculated, which is proportional to the error $\Delta r(j)$. Assuming an index-space error Δj and eliminating j yields:

$$\Delta r(j) = \left| \frac{\partial r(j)}{\partial j} \right| \cdot \Delta j = \frac{d}{j^2} \cdot \Delta j \cdot \frac{w}{2\pi} = \frac{r(j)^2}{d} \cdot \Delta j \cdot \frac{2\pi}{w} \tag{3.12}$$

Thus the variance σ_r quadratically depends on r:

$$\sigma_r \propto \frac{r^2}{d} \tag{3.13}$$

Chapter 4

Implementation

This chapter focuses on the actual implementation of the algorithm and the *implementation-level* optimizations, which — along with the *algorithm-level* optimizations presented in the previous chapter — allow for the real-time execution of the algorithm.

First a short introduction to the utilized SSE instruction set extension is given, followed by the techniques employed in the acceleration of the error stack calculation, which include a rearrangement of the snapshot memory along with a separation of the required bilinear interpolation. The next section focuses on the minimum error search and the index space refinement, where a coordinate transformation is applied in order to reduce the required bit depth and to simplify the calculation of the quadratic approximations. The last section describes a fast version of the local environment map for which the Gaussian probability distributions are precalculated for a discrete parameter set.

4.1 Short introduction to SSE

SSE (Streaming SIMD Extensions) is one of the families of SIMD instruction set extensions available on x86- and x64-processors. The original SSE instructions were expanded by incremental updates, namely SSE 2, SSE 3, SSSE 3, SSE 4.1 and SSE 4.2. The target platform supports all extensions up to SSSE 3. This implementation will mostly make use of the integer instructions provided by SSE 2 and SSSE 3.

SSE adds sixteen (eight on 32-bit platforms) new 128-bit registers to the processor, which may either hold two double- or four single-precision floats or 4×32-bit, 8×16-bit or 16×8-bit integers, thus allowing up to $16\times$ parallel execution on a single processor core. The data layout solely depends on the instructions used on these registers. A 128-bit data block is hereinafter referred to as *SSE word*.

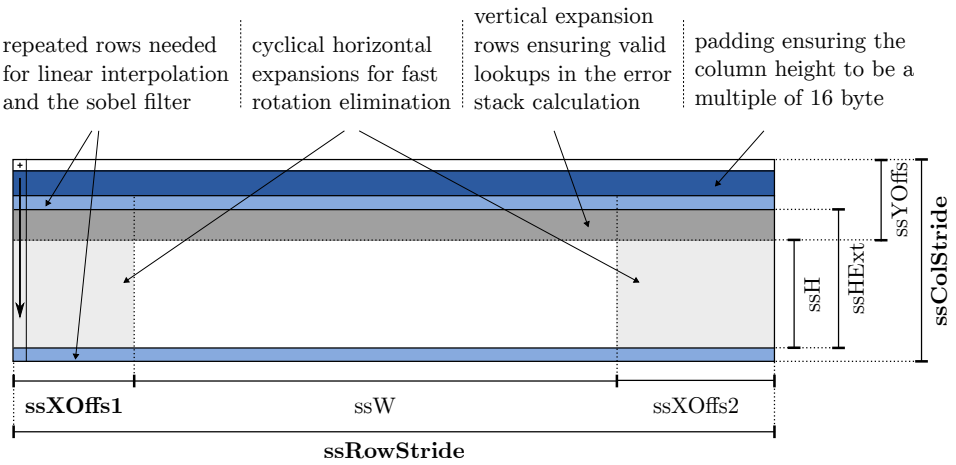


Figure 4.1: Snapshot memory layout. Each resolution level and each snapshot component is stored according to the scheme shown above. The cross in the top left corner marks the origin, the arrow indicates the column-major memory layout. Bold labels represent values which are chosen as multiples of sixteen. The actual data used for the error stack calculation is stored in the white area in the middle of the image.

The SSE instructions can be accessed in high level programming languages using so called *intrinsics*, which behave like normal functions operating on a 128-bit data type (`__mm128i` for integers, `__mm128d` for doubles and `__mm128` for singles) but receive special handling from the compiler and are (most of the time) translated into a single SSE instruction.

See the *Intel 64 and IA-32 Architectures Optimization Reference Manual* (Int, 2013) for more information on this topic.

4.2 Error stack calculation

This section focuses on the optimizations that allow a very fast calculation of the error stack layers $\mathbf{E}_i$. These optimizations include a special snapshot memory layout suitable for SSE as well as exploiting the separability of the motion field equation.

4.2.1 Snapshot memory layout

Incoming panoramic snapshot images are rearranged in memory according to a special memory layout scheme which meets the demands of the SSE implementation. This rearrangement is employed after the resolution pyramid has been calculated (see section 3.1.4). Once the first stage of the rearrangement has been performed (except for the transpose and horizontal expansion step, see below) the first derivatives in x- and y-direction are approximated using

a 3×3-Sobel filter. The resulting image components (intensity, x-derivative, y-derivative) are stored individually as independent images. This corresponds to the so called *structure of arrays* (SoA) paradigm, which in conjunction with SSE performs better for most applications than the classical *array of structures* (AoS) (Int, 2013, chapter 4.5.1, "data structure layout").

Column-major layout: The first design decision regarding rearrangement includes a conversion from the standard row-major to a column-major memory format (hence the image is stored transposed). A column-wise processing enables the efficient interpolation scheme presented below (see section 4.2.2). In order to save memory (and thus expensive memory access), the bit depth of the input images remains unchanged and each pixel is stored as an 8-bit unsigned integer.

By adding padding-rows, it is ensured that the column height is a multiple of 16 bytes. This is important, as SSE load instructions work most efficiently if the memory addresses are aligned to 16-byte boundaries (Int, 2013, chapter 4.4, "stack and data alignment"). Additionally, this allows each column to be handled with an integral count of SSE words.

Horizontal rotation elimination borders: In order to facilitate the rotational motion elimination described in equation 3.7, the snapshot image is cyclically extended in x-direction. As the panoramic images are cyclic in horizontal direction this allows the rotational motion elimination to be reduced to two horizontal offsets for the current view and the snapshot images.

However, the maximum rotational angle that can be eliminated is now limited by the width of the cyclic extension, which is currently chosen as $\frac{w}{4}$ in both directions, limiting the maximum rotation angle to $\pm 90°$ (which should be sufficient). It should also be noted that the use of discrete pixel offsets introduces a quantization error with respect to the rotation angle, which is especially large for lower resolution levels.

Vertical lookup extension: As noted in section 2.1.1, only a portion of the polar range available in the snapshot is effectively used for the actual depth reconstruction. The preprocessing step only copies this reduced portion, yet as warping the snapshot requires lookups which lie outside this reduced polar range (see figure 4.3), some additional vertical extension rows allowing these lookups are added to the top of the snapshot. The height of this extension region is chosen to be as large as possible (only rows that are present in the input image are copied) and as small as needed (only rows that are referenced by the longest flow vector are copied).

If the polar range covered by the input image is too small, the vertical lookup region does not eliminate the need of adding some mechanism to the error stack calculation that allows to cope with such invalid lookups.

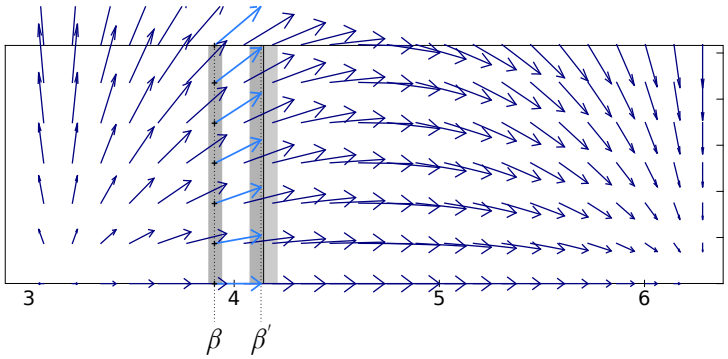


Figure 4.2: Section of a flow template $\mathbf{U}_i$. The highlighted arrows point from a target column β (corresponding to a discrete pixel) to the source column β'. As all arrows point to the same source row, the bilinear interpolation employed when calculating the warped image $\mathbf{SS'}_i$ can be split into a *horizontal* interpolation step (interpolating between the two referenced source rows) and a *vertical* interpolation step.

A copy of the top row is added to the top of the snapshot data and the same procedure is analogously performed at the bottom. This allows to prevent branching in the border regions for the linear interpolation and the Sobel–kernel application.

Memory layout descriptors: A schematic showing the complete memory layout can be seen in figure 4.1. The annotated widths and heights fully describe the memory layout and together form a *memory layout descriptor*. These descriptors (also including the flow template and error stack memory layout data not described here) are calculated once for each pyramid level. A pointer to the memory descriptor is then passed to all parts of the implementation.

4.2.2 Separability of the motion field equation

The motion field equation 3.8 as given in section 3.2 describes the mapping of a target pixel (β, γ) in $\mathbf{SS'}_i$ (needed for the error stack calculation, see section 2.2.4) to a source pixel (β', γ') in $\mathbf{SS}$. Thus the $\mathbf{SS'}_i$ are calculated by warping the snapshot image $\mathbf{SS}$ according to the flow template $\mathbf{U}_i$ using bilinear interpolation. As noted above, the source column β' solely depends on the target column β and not on the target row γ.

Column-wise calculation of each discrete target column β allows for separation of the bilinear interpolation into a *horizontal* and a *vertical* interpolation step. For the horizontal interpolation, the two neighboring source columns referenced by β' are interpolated, resulting in an intermediate source column.

In the vertical interpolation step for each row γ in β the two source pixels referenced by γ' in the intermediate source column are interpolated (see figure 4.2).

4.2.3 Flow template calculation

The actual flow template data for all $\mathbf{U}_i$ and all resolution levels are precomputed for each pixel as defined in equation 3.9. In order to simplify the code the depression angle $\check{\gamma}$ is set to a fixed value of 0°, while the elevation angle $\hat{\gamma}$ remains a free parameter.

Instead of storing a field of two-dimensional floating point vectors for each target pixel, the flow template is encoded in memory using a set of five integer arrays:

- A one-dimensional 16-bit array `tarX` (size w) holds the index of the left-most of the two discrete source columns corresponding to each target column β.
- A one-dimensional array `facX` (size w) holds the horizontal interpolation factors for the two source columns. The factors are multiplied by 2^6 and stored as 2×8-bit tuples. The full 8-bit range is not used in order to prevent overflows in subsequent steps.
- A two-dimensional 8-bit array `offsY` (size $w \times \frac{h}{8}$) holds the index of the first source pixel (clamped to zero) in the intermediate column for each block of eight pixels in each target column.
- For each pixel the two-dimensional 8-bit array `shuffleMaskY` (size $w \times 2 \cdot h$) holds the two vertical source pixel indices relative to the first source pixel offset stored in `offsY`. Inaccessible pixels — those with an index smaller than zero or larger than fifteen — are marked with 0xFF.
- A two-dimensional array `facY` (size $w \times h$) holds the vertical interpolation factors for the two source pixels corresponding to each target pixel. The values are encoded as described for `facX`.

The two-dimensional arrays are stored column-major and with corresponding padding ensuring the column size to be a multiple of 16. The offsets stored in `tarX` and `offsY` directly address the pixels as they are stored in the snapshot memory layout.

4.2.4 Snapshot warping and error stack calculation

The calculation of a single error stack layer $\mathbf{E}_i$ consists of the following steps: For each snapshot component, a warped version of that component is calculated according to the flow template using bilinear interpolation. Then the

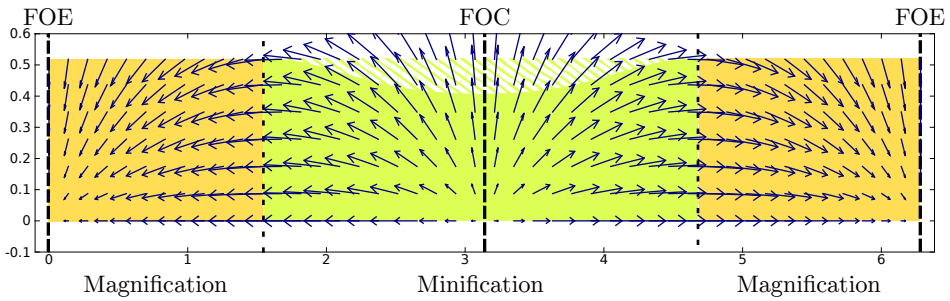


Figure 4.3: Regions in each flow template. The π-interval surrounding the focus of contraction (FOC) describes a *minification* in which the source pixel region is always larger or equal in size to any region of target pixels (many pixels are mapped to few pixels). The π-interval surrounding the focus of expansion (FOE), however, describes a *magnification* in which the source pixel region is always smaller or equal in size to any region of target pixels (few pixels are mapped to many pixels). The hatched region in the image contains the flow vectors pointing to invalid (out of range) source pixels.

absolute difference between the warped image component and the corresponding current view image component is calculated. This value is subsequently added to the resulting error stack layer value (remember that the optical flow as defined in equation 2.1 uses the L^1-norm).

Image warping and interpolation: Given the precalculated flow template, the horizontal and vertical warping and interpolation steps become equally trivial, although the vertical interpolation requires additional attention.

Of special interest in both steps is the SSSE 3 `_mm_maddubs_epi16` intrinsic, which takes two 16×8-bit SSE words (a and b) as input and calculates a 8×16-bit output vector c according to the following expression ($[i]$ corresponds to the ith cell in the SSE word):

$$c[i] = a[2\cdot i]\cdot b[2\cdot i] + a[2\cdot i+1]\cdot b[2\cdot i+1] \quad \forall\, 0 \le i < 8\,.$$

Interpolation between pixel pairs can be achieved by placing each pair in adjacent cells in a and the corresponding interpolation factors in b, allowing eight result pixels to be computed in a single instruction.

The interpolation in horizontal direction is performed by interlacing two 16×8-bit SSE words read from the source rows into two pixel pair vectors aLO and aHI and interpolating those according to the factors stored in the flow template `facX` array. The two result vectors are then packed into the 8-bit intermediate column (see appendix C.1.1 for the complete code).

Interpolation in vertical direction is done by reading a 16×8-bit SSE word from the intermediate row starting with the pixel specified in `offsY`

and building a single pixel pair vector by shuffling this 16-pixel input word according to the indices specified in `shuffleMaskY`. The pixel pairs are then interpolated according to the interpolation factors in `facY`. Note that this technique is only capable of mapping a maximum of 16 consecutive source pixels to 8 target pixels, so the minification factor should never exceed two (see figure 4.3). Yet for the parameters used in practice this is never the case.

For invalid pixel lookups — those marked with 0xFF in the shuffle mask — a "maximum error mask" is created from the shuffle mask and the corresponding error values are set to this maximum error value. As three image components are processed and the maximum pixel intensity difference is 255 per component, this maximum error value is chosen as 0x3FF (see C.1.2 for the complete code).

Error stack bias: The error bias $\tau \cdot i$ introduced for the minimum error search in section 2.2.6 is directly added to the error values in each error stack layer $\mathbf{E}_i$ and will thus not be regarded in the following sections.

4.3 Index field calculation

The next step in the algorithm is to find the minimum error in the error stack layers and to generate the index field $\mathbf{J}$, which is then refined to the index field $\tilde{\mathbf{J}}$. Yet before these steps can be discussed in detail, an error propagation matrix $W_{\vec{p}}$ (left open in section 2.2.6) needs to be chosen.

4.3.1 Propagation matrix and minimum error search

The most influential parameter from an efficiency point of view is the propagation matrix $W_{\vec{p}}$. The space and data variant $n \times n$ error propagation matrix (where $n = 2 \cdot r_W + 1$) introduced in section B.1 is generally non-separable and thus needs n^2 calculations in every pixel. Additionally, creating and storing the filter kernels requires additional time and storage.

Preceding experiments (see section B.2 for the results and chapter 5 for the test set-up) have shown that choosing $W_{\vec{p}}$ as a space-invariant 3×3 binomial convolution kernel provides an acceptable trade-off between speed and quality. The propagation matrix W is subsequently defined as:

$$W := W_{\vec{p}} = \frac{1}{16} \cdot \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix} .$$

The filter is separable into a horizontal and vertical convolution

$$W := \frac{1}{16} \cdot \begin{pmatrix} 1 & 2 & 1 \end{pmatrix} * \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}$$

which can both be trivially implemented using SSE.

Minimum error search: The minimum error search leading to the index field $\mathbf{J}(\vec{p})$ is implemented exactly as defined in equation 2.12: First $\mathbf{E}_i * W$ is calculated for all error stack layers (which as noted above already contain the bias $\tau \cdot i$), then for each pixel $\vec{p}$ the layer index with minimum error is searched. Using SSE, eight pixels can be processed at a time. In order to allow for a refinement of the index field, the indices are stored as 16-bit fixed point numbers with the scale factor 2^8.

4.3.2 Index field refinement

In equation 2.14 the refined index field $\tilde{\mathbf{J}}$ was effectively defined as

$$\tilde{\mathbf{J}}(\vec{p}) = \underset{\mathbf{J}(\vec{p})-1 \leq j \leq \mathbf{J}(\vec{p})+1}{\operatorname{argmin}} \left(\check{E}_{\vec{p}}(j) + \check{R}_{\vec{p}}(j) \right) , \tag{4.1}$$

where $\check{E}(J)$ refers to the convoluted quadratic error stack approximation functions and $\check{R}(j)$ to the convoluted regularization functions:

$$\check{E}_{\vec{p}}(j) = \sum_{\vec{q} \in A(\vec{p})} \omega_{\vec{p}}^{\vec{q}} \cdot \tilde{E}_{\mathbf{J}(\vec{p})}(j; \vec{q}) , \qquad \check{R}_{\vec{p}}(j) = \lambda \cdot \sum_{\vec{q} \in A(\vec{p})} \omega_{\vec{p}}^{\vec{q}} \cdot (\mathbf{J}(\vec{q}) - j)^2 .$$

Simplification of the index field refinement equation: The convolution over a set of quadratic polynomials is impractical for a fast implementation. It would be of advantage if the convoluted error stack layers $\mathbf{E}_i * W$ calculated in the previous step could be reused.

In the case of $\check{E}_{\vec{p}}(j)$, the convolution and the calculation of the quadratic coefficients are commutative: For $i \in \{-1, 0, 1\}$ and $j = \mathbf{J}(\vec{p})$ the following relation holds

$$\check{E}_{\vec{p}}(j+i) \overset{\text{Def. } \tilde{E}}{=} \sum_{\vec{q} \in A(\vec{p})} \omega_{\vec{p}}^{\vec{q}} \cdot \mathbf{E}_{\mathbf{J}(\vec{p})+i}(\vec{q}) \overset{\text{Def. } \omega_{\vec{p}}^{\vec{q}}}{=} \left(\mathbf{E}_{\mathbf{J}(\vec{p})+i} * W \right)_{\vec{p}}$$

and — as each quadratic polynomial is uniquely defined by three points — proves that fitting the coefficients according to the values in the convoluted error stack layers is equivalent to convolving the polynomials themselves.

Unfortunately the regularization function $\check{R}_{\vec{p}}(j)$ is not commutative regarding the convolution of the index field $\mathbf{J}$. Expanding $\check{R}_{\vec{p}}(j)$ yields

$$\check{R}_{\vec{p}}(j) = \lambda \cdot \left(j^2 - 2 \cdot (\mathbf{J} * W)_{\vec{p}} \cdot j + \sum_{\vec{q} \in A(\vec{p})} \omega_{\vec{p}}^{\vec{q}} \cdot \mathbf{J}(\vec{q})^2 \right)$$

where the constant term plays no role for finding the minimal j. Thus an equivalent and commutative regularization function is given by

$$\tilde{R}'(j; \vec{p}) = \lambda \cdot (j^2 - 2 \cdot (\mathbf{J} * W)_{\vec{p}} \cdot j) \tag{4.2}$$

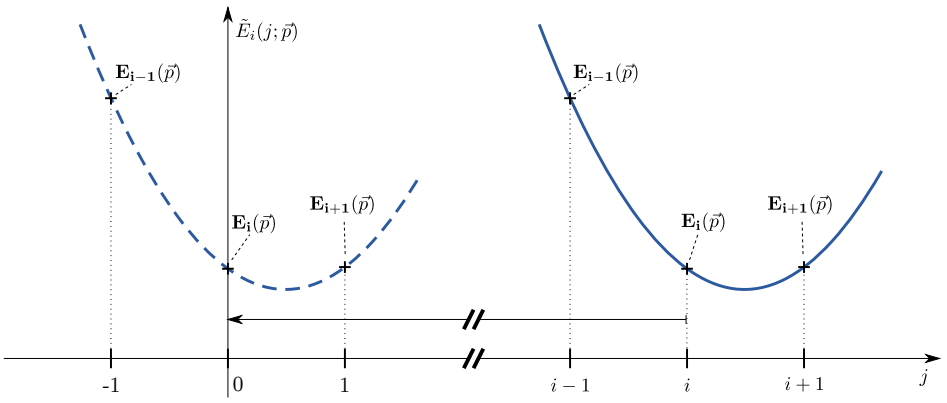


FIGURE 4.4: Translating the center index i of each quadratic approximation polynomial $\tilde{E}_i(j;\vec{p})$ (solid blue line) to the origin at 0 (dashed blue line) allows for simplified calculation of the polynomial coefficients $\vec{w}$ and lowers the required bit depth needed to store those.

Quadratic approximation calculation: The calculation of the quadratic polynomials $\tilde{E}_i(j;\vec{p}) = (j^2, j, 1) \cdot \vec{w}$ as given in equation 2.13 can be simplified by a coordinate transformation which translates the center index i to the origin at $j = 0$ (see figure 4.4). The adapted refinement equation 4.1 is then given as

$$\tilde{\mathbf{J}}(\vec{p}) = \underset{-1 \le j \le 1}{\operatorname{argmin}} \left(\check{E}_{\vec{p}}(j) + \check{R}_{\vec{p}}(j) \right) + \mathbf{J}(\vec{p}) \; . \tag{4.3}$$

The coefficients $\vec{w}$ of $\check{E}_{\vec{p}}(j)$ can now be trivially implemented in SSE:

$$\vec{w} = \begin{pmatrix} w_2 \\ w_1 \\ w_0 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 & -2 & 1 \\ -1 & 0 & 1 \\ 0 & 2 & 0 \end{pmatrix} \cdot \begin{pmatrix} \mathbf{E}_{i-1}(\vec{p}) * W \\ \mathbf{E}_i(\vec{p}) * W \\ \mathbf{E}_{i+1}(\vec{p}) * W \end{pmatrix} \; . \tag{4.4}$$

Additionally, this equation has the advantage of yielding a much smaller coefficient vector, which can (using the SoA paradigm) be stored in three 8×16-bit SSE words `w0`, `w1`, `w2`, allowing the refinement of eight pixels at the same time.

For the regularization term $\tilde{R}'(j;\vec{p})$ the coordinate transformation yields the following equation

$$\tilde{R}'(j;\vec{p}) = \lambda \cdot (j^2 - 2 \cdot ((\mathbf{J} * W)_{\vec{p}} - \mathbf{J}(\vec{p})) \cdot j) \; .$$

Note that for the search range $-1 \le j \le 1$ the quadratic term j^2 is negligible and the factor two can be condensed into λ. Thus the regularization is reduced to subtracting $\lambda \cdot ((\mathbf{J} * W)_{\vec{p}} - \mathbf{J}(\vec{p}))$ from the linear coefficient vector `w1`.

All coefficients are stored as fixed point numbers (scale factor 2^3) allowing a slightly higher resolution for λ.

Refined index calculation: The extremum of a quadratic polynomial $w_2 j^2 + w_1 j + w_0$ is given by $\check{\jmath} = -\frac{w_1}{2w_2}$. In order to eliminate the need for special handling (which is hardly possible when using SIMD instructions) in the case of $w_2 = 0$, w_2 is always set to one in that case. The resulting $\check{\jmath}$ is clamped to the interval $[-1, 1]$ and the corresponding error value is then compared to the error values at $j = -1$ and $j = 1$. The actual refined index field entry is then chosen as the index belonging to the smallest of the three error values.

As SSE does not provide integer division instructions, the term $-\frac{w_1}{2w_2}$ is evaluated using floating point numbers. The approximative but significantly faster reciprocal instruction `_mm_rcp_ps` in conjunction with a multiplication is used here, which has a combined precision in the range of $\approx 2^{-11}$ (Int, 2013, chapter 11.12 "divide and square root operations", p. 417ff.). This is not a problem since the resolution of the fixed-point 16-bit indices j was chosen as 2^{-8}.

4.4 Local environment map

The local environment map is especially optimized in the position update and the distribution accumulation as introduced in section 2.3.2. These optimizations are roughly sketched in the two following sections.

4.4.1 General optimizations

The local environment map is stored in memory as a 16-bit two-dimensional matrix. SSE trivially allows to speed up the map decay step (equation 2.19), as the SSSE 3 intrinsic `_mm_mulhrs_epi16` allows 8× parallel multiplication of a 8×16-bit word with a scalar $\delta \in (0, 1]$.

Furthermore the need of the image rotation in the map update step was eliminated by integrating the change in orientation ψ and using this current orientation and the current translatory motion direction α as an angular offset when inserting the Gaussian distributions. This allows to reduce the map update step to a discrete, pixel wise translation. The error introduced by rounding to discrete pixel offsets is accounted for by feeding the rounding error into the next map update step (comparable to error-diffusion dithering).

Although the calculation of the thresholded map image can be parallelized using SSE, no trivial optimization of the connected component calculation was found.

4.4.2 Distribution accumulation optimization

In order to avoid the costly evaluation of the Gaussian probability distributions whenever a landmark point $\vec{q} = (\beta, r)$ is inserted into the map, the distributions are precalculated in Cartesian space for discrete $\beta_i \in [0, 2\pi]$ and a discrete range of $r_i \in [r_{\mathrm{MIN}}, r_{\mathrm{MAX}}]$. Note that the variance σ_r as derived in

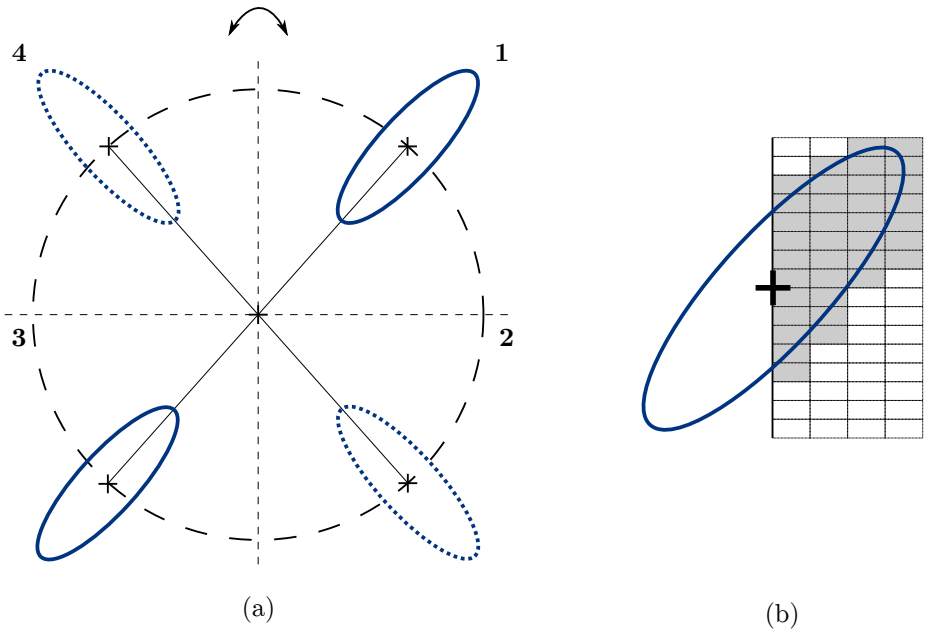


FIGURE 4.5: Symmetries of a Gaussian distribution. (a) A Gaussian distribution calculated for quadrant 1 can be used in quadrant 3 without modification. Mirroring along the x-axis allows to use the distribution in quadrant 2 and 4. Consequently precalculating the distributions for one quadrant only allows to reduce the space requirement by one quarter. (b) As each Gaussian distribution is point-symmetric, only one half of the distribution needs to be stored in memory — the memory footprint can be reduced even more by skipping all SSE words which only contain pixel values below a certain threshold (white).

equation 3.13 also depends on the translatory distance d between **SS** and **CV**, yet precalculating the distributions for a three-dimensional parameter space β, r, d is impractical. Therefore the robot is implicitly assumed to move with constant speed.

As depicted in figure 4.5, the distributions are stored as fixed point 8×16-bit SSE words for a rectangular hull encompassing a contour line at a certain threshold level. To lower the memory requirements symmetries in the distributions are exploited and only SSE words containing values above the threshold level are stored in memory, along with a descriptor that contains the number of words skipped and the number of words in a row. Preceding experiments have shown that the reduced memory footprint and better cache locality outweighs the overhead introduced by this packing scheme.

The map array size is extended by the half width/height of the largest Gaussian distribution bounding rectangle in order to eliminate the need of border handling (and thus branching) when plotting the map pixels.

The SSE code used for accumulating a precalculated distributions onto the local environment map is given in appendix C.2.

Chapter 5

Experiments and results

This chapter describes the experiments which have been conducted in order to measure the performance of the obstacle detection algorithm. These experiments include tests in a computer-generated environment as well as real-world tests on the robot platform. For the computer-generated environment quantitative measures describing the performance of the algorithm have been developed.

5.1 Environments

In this section the environments used for testing the algorithm are described. Those environments cover two computer-generated scenes and a real-world test in a living room inspired test room.

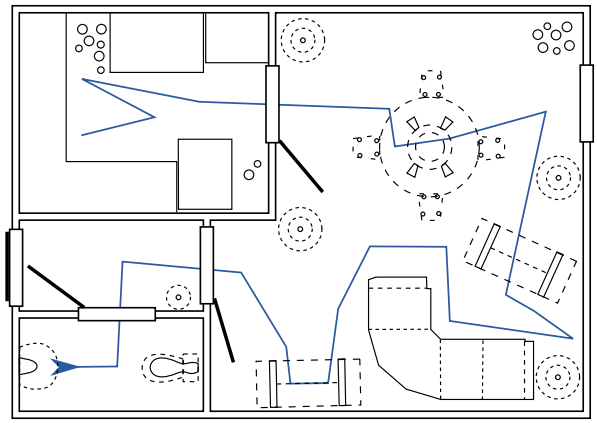
5.1.1 Computer-generated environments

Measuring a perfect ground truth for the depth reconstruction stage of the algorithm in the real-world would be a very tedious task. In computer-generated environments, the ground truth can be simply extracted by storing the so-called z-buffer, which contains the distance of each pixel in the scene and is created during the rendering process. Moreover, the computer-generated scenes allow to test the algorithm under a perfect path integration assumption (section 2.1.2) as the position and orientation of the robot are known at all times.

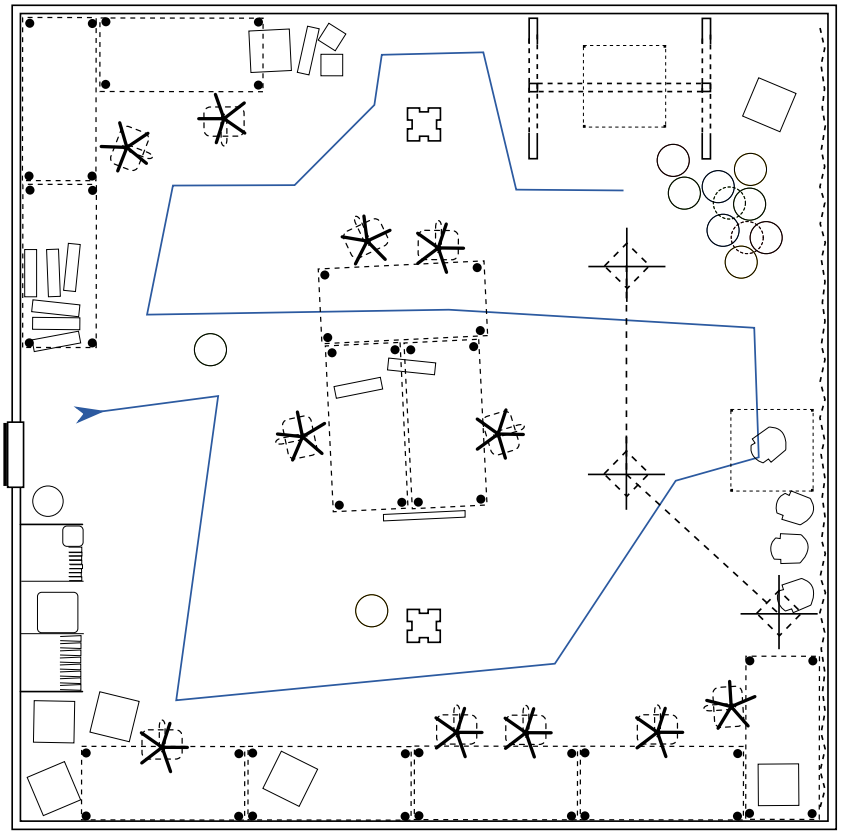
The algorithm is given a sequence of prerendered panoramic snapshot images. These snapshots are generated from the perspective of a virtual robot which moves with constant velocity along a predefined trajectory. The trajectories were laid out by hand, no navigation or obstacle evasion was simulated.

The scenes were modeled and rendered using the open-source software *Blender* and its path-tracing renderer *Cycles*[1], which was slightly modified in

[1] See `http://www.blender.org/` for more information.



(a) Apartment scene (7×5 m)



(b) Laboratory scene (10×10 m)

Figure 5.1: Overview of the computer-generated scenes. Solid lines indicate obstacles relevant to the robot (as they are placed directly on the floor), dashed lines are only for orientation and indicate objects which are too high to be detected as obstacles. The blue line depicts the robot trajectory.

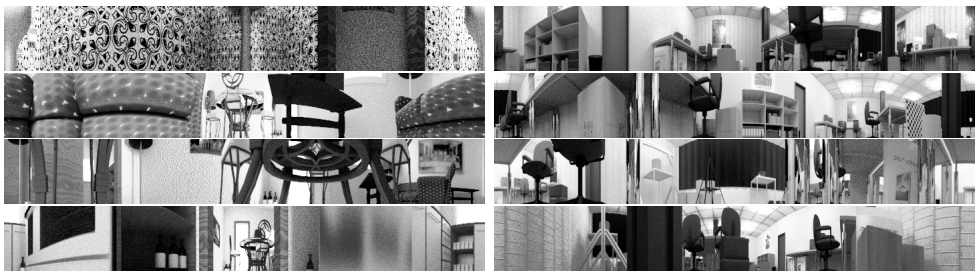


FIGURE 5.2: Some exemplary snapshots from the apartment scene (left) and the laboratory scene (right) with applied histogram equalization. The pictures are used as input for the test runs exactly as shown here.

order to produce the required spherical panoramic images. The scenes were rendered under full global illumination. The snapshot resolution is set to 720×96 pixels and the depression and elevation angles are set to $\check{\gamma} = 0°$ and $\hat{\gamma} = 30°$, respectively. The robot travels along the trajectory at $0.2\,\frac{\mathrm{m}}{\mathrm{s}}$. A new frame is created every 200 ms. The virtual camera moves at an elevation of 0.1 m above the floor. Note that the images contain sampling noise produced by the path-tracer.

Two environments (see figure 5.2 for example images) were created: A large-scale single-room laboratory scene and a relatively small apartment scene, consisting of bathroom, hallway, living room and a kitchen. See figure 5.1 for the scene layouts.

The laboratory scene primarily features two types of table legs (cylindrical brushed metal legs and rectangular dull-white powder-coated legs), chairs, printed cardboard boxes, tripod legs and featureless cylinders as obstacles. Two of the walls (left and right) feature a very fine wallpaper-like structure, the other two walls feature a brick texture. The scene is lit by a total of nine ceiling lights. In order to simulate a realistic computer scientist working environment, the window openings are barred with a light-tight black curtain. The average distance between the robot and the obstacles is relatively high (typically around $\approx 0.5 - 1\,\mathrm{m}$). The prerecorded trajectory consists of 873 snapshot images.

The apartment scene features floor lamps, a couch, tables, chairs, bathroom and kitchen interior as well as empty glass bottles in the corners of the room[2]. The bathroom is tiled with patterned tiles, while the other walls in the apartment are covered with relatively rough interior wall plaster. The scene is lit through various window openings. The average distance between the camera and the obstacles is relatively small (typically around $\approx 0.2 - 0.6\,\mathrm{m}$). The prerecorded trajectory consists of 624 snapshot images.

[2]The author likes to stress that the apartment scene in no way reflects his own living environment.

While the resulting snapshots are not completely photo-realistic, they are believed to be a reasonable approximation.

5.1.2 Real-world tests

Real-world tests are performed on the actual mobile robot platform which runs the fully optimized version of the obstacle detection algorithm. The robot performs a cleaning run in which it seeks to drive parallel lanes in order to cover the complete room. The detected obstacle points are directly fed into the navigation algorithm and influence the robot trajectory. The robot captures a snapshot image approximately every 200 ms and moves at a translatory velocity of about 0.2 $\frac{\mathrm{m}}{\mathrm{s}}$.

The obstacle detection is activated whenever the robot drives straight lanes. It fully relies on odometry data as input for the local motion parameters. In the current implementation, the motion parameters are not integrated between the snapshots. The wheel encoder values are read after an image has been captured and are assumed to be piecewise linear. The odometry data and the snapshot images are recorded for later analysis.

The test room features a living room suite (four chairs, one table, carpet on the floor), a couch, floor lamp, curtain and a sideboard. The room is lit by a single fluorescent ceiling lamp. The floor of the room is covered with wooden planks leading to a slightly uneven floor.

5.2 Quality measures

The simulated test runs are analyzed using quantitative quality measures. These include the count of *true* and *false positive* obstacle points as well as the average obstacle point detection error and the average depth reconstruction error.

5.2.1 True and false positive obstacle points

Let $\mathbf{M_R}$ denote the local environment map as generated by the algorithm containing the reconstructed obstacle points and let $\mathbf{M_T}$ be a reference local environment map, which — instead of using the reconstructed depth map — is generated from the ground truth depth map. Note that the depth variance σ_r used in the generation of $\mathbf{M_T}$ has to be chosen as a constant value describing the quantization error of the reference depth map (the depth is sampled at 8 bit with a resolution of 0.078 $^{\mathrm{m}}/_{\mathrm{dig}}$). Additionally, the following sets need to be defined:

$$P_R := \{\vec{p} = (x, y)^T \mid \vec{p} \text{ is reconstructed obstacle point in } \mathbf{M_R}\}$$
$$P_T := \{\vec{q} = (x, y)^T \mid \vec{q} \text{ is true obstacle point in } \mathbf{M_T}\}$$

where $\vec{p}$ and $\vec{q}$ denote the obstacle points in Cartesian world coordinates. Furthermore the function

$$\mathcal{D}(\vec{p}; A) := \min \{ \|\vec{p} - \vec{q}\| \mid \vec{q} \in A \} \tag{5.1}$$

denotes the distance between $\vec{p}$ and the point $\vec{q} \in A$ closest to $\vec{p}$.

Given a threshold value χ, define the following sets, which describe *true positive* obstacle points (reconstructed obstacle points near a true obstacle point) and *false positive* obstacle points (reconstructed obstacle points having no near true obstacle point).

$$\begin{aligned} T_{\text{POS}} &:= \{ \vec{p} \in P_R \mid \mathcal{D}(\vec{p}; P_T) < \chi \} \, , \\ F_{\text{POS}} &:= \{ \vec{p} \in P_R \mid \mathcal{D}(\vec{p}; P_T) \geq \chi \} \, . \end{aligned} \tag{5.2}$$

For the experiments χ is arbitrarily chosen as the radius of the robot platform ($\chi = 0.15\,\text{m}$). Note that smaller values (for example $\chi = 0.05\,\text{m}$) might allow for a better evaluation of the algorithm's performance, yet due to limited time, the evaluation of the experiments presented here was not repeated. However, the expressiveness of the results suffices for qualitative statements regarding false and true positive obstacle count.

The two measures describing the obstacle detection performance are given as

$$\nu_F = \frac{|F_{\text{POS}}|}{|P_R|} \, , \quad \nu_T = \frac{|T_{\text{POS}}|}{|P_T|} \, , \tag{5.3}$$

where ν_F corresponds to the relative count of falsely detected obstacle points and ν_T corresponds to the relative count of correctly detected obstacle points.

For the target application, a low false positive count ν_F is of highest importance: Falsely detected obstacles points can hinder the planning process and lead to a poor navigation performance. While the detection of obstacles and thus a high true positive count ν_T was the motivation for the research topic presented in this thesis, undetected obstacle points in the worst case lead to the robot detecting the obstacle upon collision — which greatly reduces the navigation performance too, but is probably more excusable from a user perspective, than a cleaning robot stopping in the middle of a lane for no obvious reason.

5.2.2 Obstacle detection error

The mean obstacle detection error μ is calculated as the average distance between reconstructed and true obstacle points.

$$\mu := \sum_{\vec{p} \in P_R} \frac{\mathcal{D}(\vec{p}; P_T)}{|P_R|} \, . \tag{5.4}$$

For some results shown below, the median distance M of $\mathcal{D}(\vec{p}; P_T)$ (assuming an underlying uniform distribution) is calculated. Together the pair μ and M gives some indication of the distribution variance.

5.2.3 Depth reconstruction error

The depth reconstruction error μ_R is calculated from the average deviation (L^1-norm) between the ground truth depth map $\mathbf{R_T}$ and the reconstructed depth map $\mathbf{R}$. Only pixels with reconstructed depth values smaller than the radius of the local environment map r_M are counted (this clamping is required as $\mathbf{R}$ may also contain infinite depth values). Let $\mathbf{R}^i$ and $\mathbf{R}^i_\mathbf{T}$ describe the ith of N depth reconstruction results. μ_R is then defined as:

$$\begin{aligned} \vec{P}^i &:= \left\{ \vec{p} \mid \mathbf{R}^i(\vec{p}) \leq r_M \right\} , \\ \mu_R &:= \sum_{i,\vec{p}\in\vec{P}^i} \frac{|\mathbf{R}^i_\mathbf{T}(\vec{p}) - \mathbf{R}^i(\vec{p})|}{N \cdot |\vec{P}^i|} . \end{aligned} \tag{5.5}$$

As above the median depth reconstruction error M_D (assuming an underlying uniform distribution) is calculated for some results.

5.3 Results

This section features a set of experiments in the environments described above and the corresponding results. First the overall results for both simulation and real-world tests are presented. Subsequent experiments seek to explore the impact of the error stack bias parameter τ, the effects of coarse-to-fine refinement and the robustness against noise.

5.3.1 Default parameters

Unless noted differently, the following default parameters are used for both simulation and real-world tests:

Input image: 720×96 pixels, depression angle $\check{\gamma} = 0°$, elevation angle $\hat{\gamma} = 30°$, reduced elevation angle (region in which the depth is reconstructed) $\hat{\gamma}' = 10°$.

Error stack calculation: 32 error stack layers (at $d = 0.04\,\mathrm{m}$ r_i ranges from $r_0 = \infty$ to $r_{31} \approx 0.15\mathrm{m}$), error stack bias $\tau = 1$.

Coarse-to-fine refinement: Search range $K = 2\kappa + 1 = 8$; 3 coarse-to-fine refinement steps.

Minimum error search: $W_{\vec{p}}$ is a 3×3 binomial filter kernel, $\lambda = 1$, one refinement iteration.

Local environment map: The standard deviation in depth direction is chosen as $\sigma_r = 0.01 \cdot (r^2 + 1\,\mathrm{m}^2)$, the environment map density value threshold as $\eta = 40$.

5.3.2 Simulated test run

Experiment description: In this experiment a test run with the default parameters (as given above) is conducted for each simulated environment. Runtime measurements are performed on an Intel® Atom™ N2600 dual core CPU running at 1.60 GHz (the same model controlling the target robot) and covers both the depth reconstruction and the obstacle detection stage.

Results: The results of this test run can be seen in figure 5.3 (apartment scene) and 5.4 (laboratory scene), where *false negative* obstacle points are defined as reference points $\vec{q} \in P_T$ with $\mathcal{D}(\vec{q}; P_R) > \chi$.

In the apartment scene, the algorithm creates multiple false positive obstacle points in the couch region and a single false positive point in the table region. On the other hand, all major obstacles are detected, including the table and chair legs. Most walls are detected. Relatively few obstacle points are detected for the brushed stainless steel refrigerator and the glass front of the stove (bottom right and top middle of the kitchen).

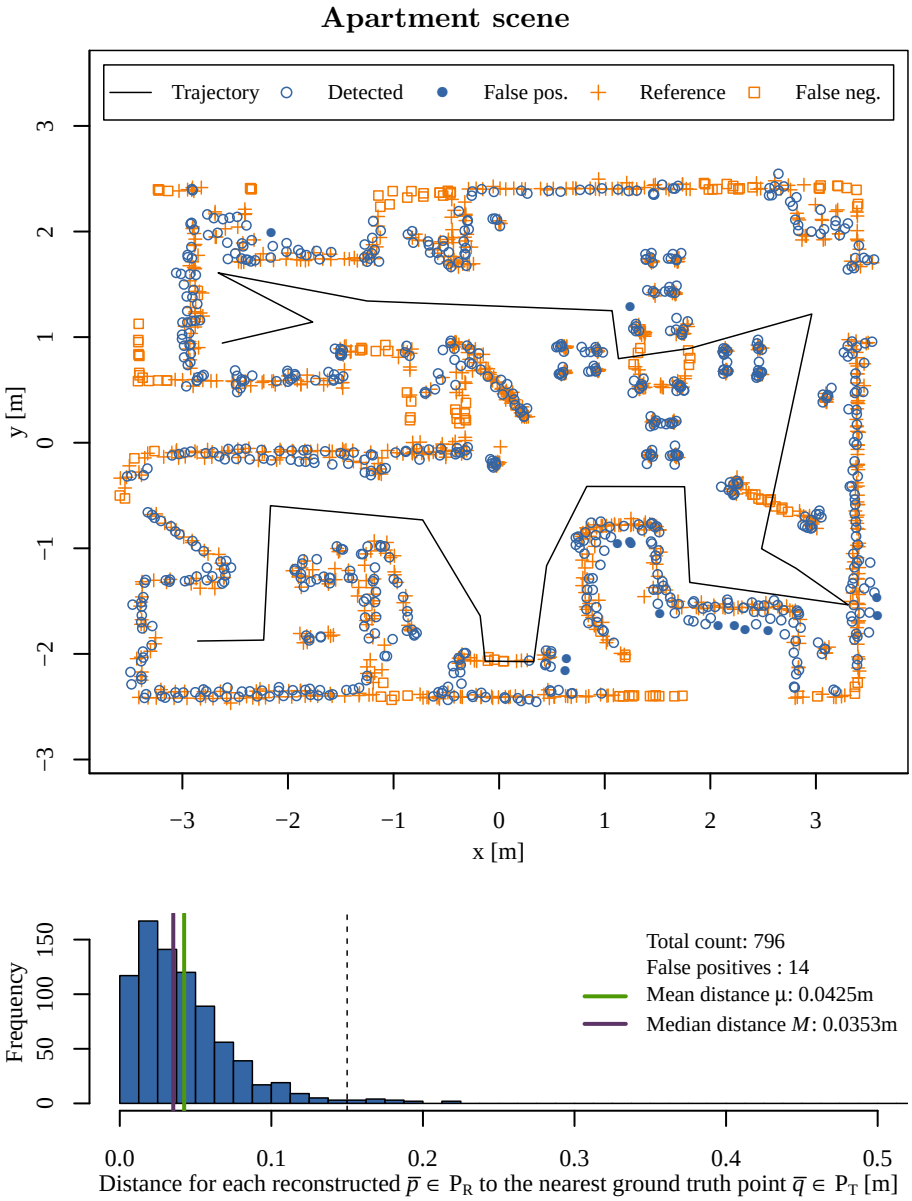
In the laboratory scene a single false positive obstacle point is found. Generally fewer obstacle points are detected (likely due to the size and higher sparsity of the room), yet all obstacles surrounding the trajectory in a distance of up to 1 to 1.5 m are detected. A notable exception is the checkerboard in the middle of the room.

During the simulation run it can be observed that the detection range is smaller for obstacles which lie in the direction of the focus of expansion or the focus of contraction, unless the corresponding objects have a prominent texture. For example at the end of the trajectory in the laboratory scene one can see that the (untextured) cylinders lying directly in front of the robot are not detected. With an average execution time of 24.28 ms and 16.39 ms, respectively, real-time execution is possible on the target processor.

5.3.3 Real-world test run

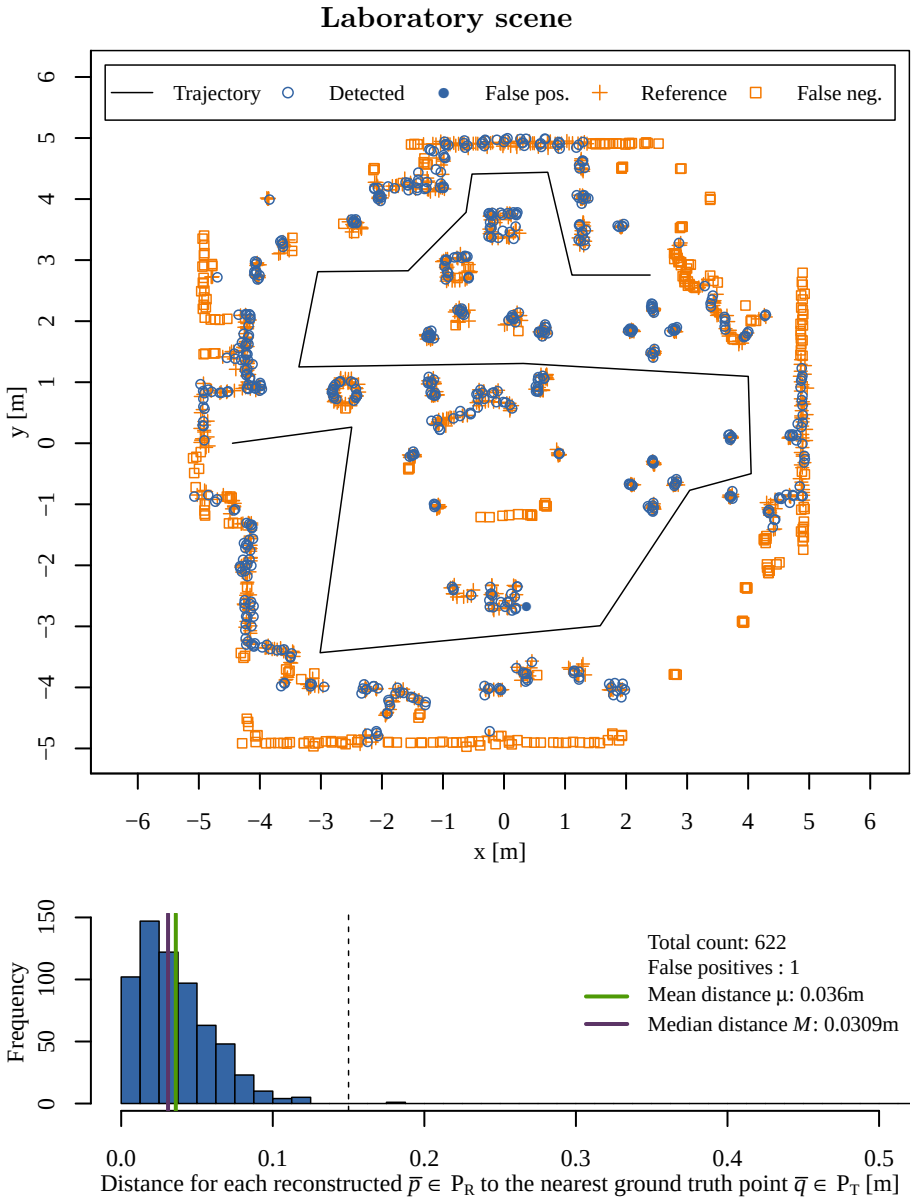
The results of two test runs in the test room environment described above can be seen in the figures 5.5 and 5.6. Note that the environment map threshold η (see section 2.3.2) is doubled from $\eta = 40$ to $\eta = 80$ in order to increase the robustness against noise in exchange for a reduced detection range.

In the first test run, the robot drives a straight trajectory between the legs of the living room suite (see figure 5.5(a)). The legs of chairs and tables are detected, although superimposed by relatively severe noise. In the depth reconstruction, the legs are correctly classified (figure 5.5(c)), yet the depth of each leg spills onto background pixels. Correspondingly the probability distributions belonging to the table legs occupy relatively large regions in the local environment map (figure 5.5(b)).



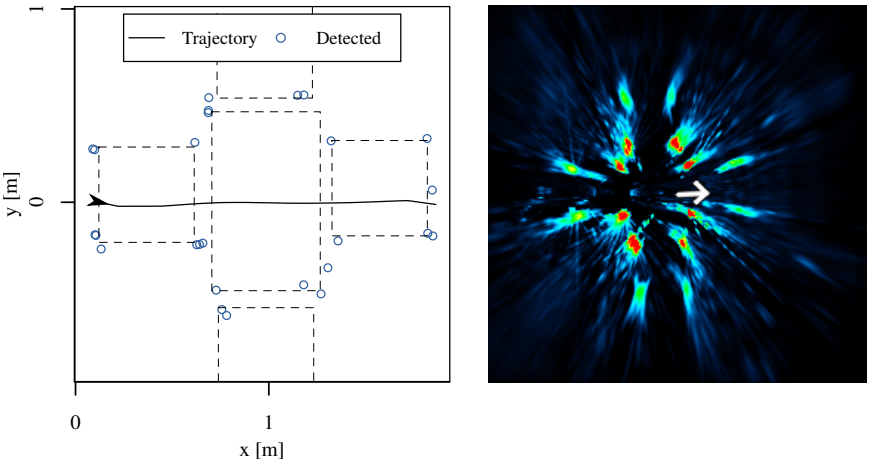
Reconstructed count:	796	Ground truth count:	1150
False positives:	14	True positive:	782
ν_F:	1.76%	ν_T:	68%
μ:	0.0425 m	M:	0.0353 m
μ_D:	0.594 m	M_D:	0.218 m
Average time:	24.28 ms	Max. time:	35.40 ms

Figure 5.3: Overall results for a simulated run in the apartment scene using the default parameters described above. See figure 5.1 for the room layout.



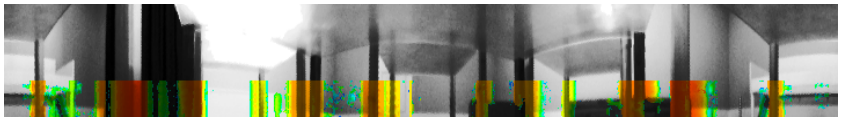
Reconstructed count:	622	Ground truth count:	1225
False positives:	1	True positive:	621
ν_F:	0.16%	ν_T:	51%
μ:	0.036 m	M:	0.0309 m
μ_D:	0.769 m	M_D:	0.307 m
Average time:	16.39 ms	Max. time:	27.2 ms

FIGURE 5.4: Overall results for a simulated run in the laboratory scene using the default parameters described above. See figure 5.1 for the room layout.

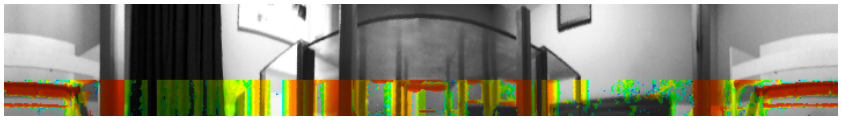


(a) Reconstructed obstacle points

(b) Local environment map



(c) Depth reconstruction



(d) Artifacts introduced by tilting

Figure 5.5: Test room dinner table results. (a) Obstacle points emitted during the test run and the corresponding trajectory (odometry data integration). The dashed lines depict the approximate position of the table and the four chairs (only shown for visualization and orientation purposes). (b) Local environment map. The arrow depicts the current robot position and orientation. (c) A snapshot of the depth reconstruction at the arrow position. Notice how the table legs are colored in yellow, orange and red depending on their distance. See figure 5.8 for details on the used color coding. (d) Artifacts in the depth reconstruction (note the red/near regions at the side and in the middle of the image) introduced by tilting the robot.

A false obstacle point is emitted at the end of the trajectory. At this position, the robot tilts/changes its elevation while driving over the edge of a carpet. Figure 5.5(d) shows the corresponding artifacts in the depth map, which are responsible for the false positive obstacle point.

In the second test run, the legs of the couch (bottom region in figure 5.6) as well as the floor lamp to it's right are correctly detected. Correct obstacle points are also detected for the left wall which is partially covered by a curtain and two legs of the bottom chair. However, a large amount of false positive

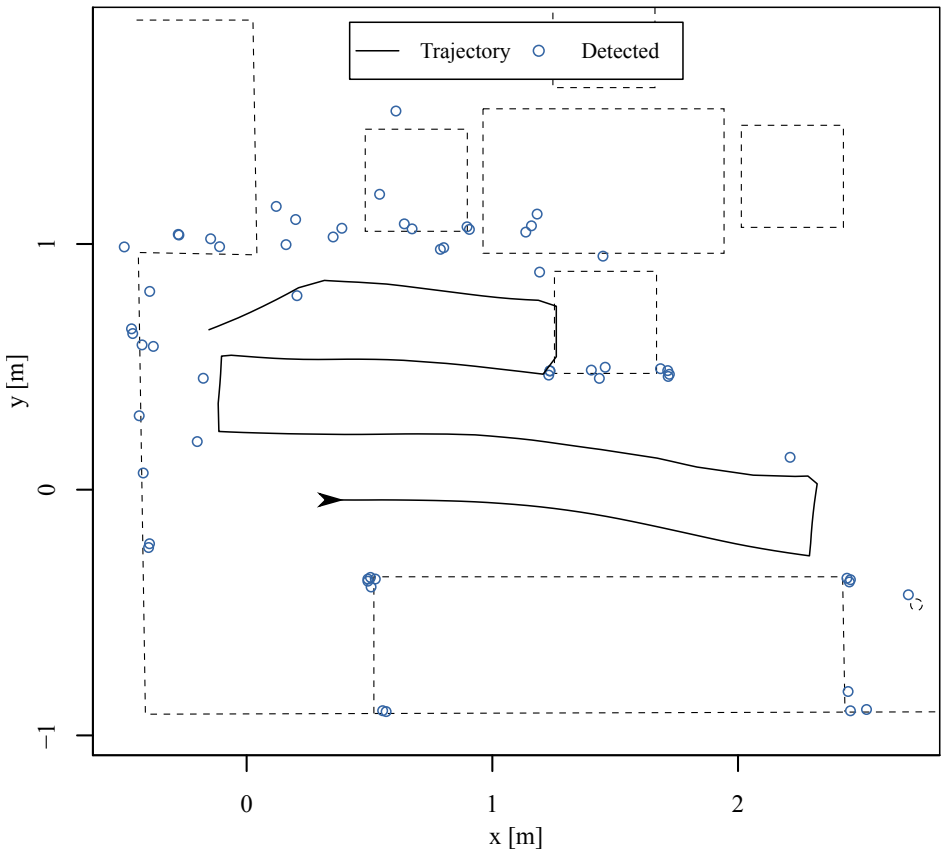


FIGURE 5.6: Test room results. Blue circles show obstacle points which were emitted during the test run, the straight line depicts the corresponding robot trajectory (odometry data integration). The dashed lines depict the approximate position of the furniture (table, four chairs, couch, sideboard) and the walls (only shown for visualization and orientation purposes). Note that the left wall is partially covered by a curtain.

obstacle points is emitted in the top-left region of the trajectory.

5.3.4 Error stack bias

Experiment description: This test run measures the effect of the error stack bias parameter τ in the simulated environments (introduced in section 2.2.6). The parameter τ is varied between 0 and 5. Due to restrictions in the optimized implementation only integral values are tested.

Results: As shown in figure 5.7, nearly $1/5$ of the reconstructed points are false positive for $\tau = 0$, in both laboratory scene and apartment scene. This

number rapidly decreases as soon as $\tau \geq 1$. Yet a further increase in τ continuously decreases the count of reconstructed obstacle points as well as (at least in the apartment scene) the depth reconstruction error. Note how in the large laboratory room the depth reconstruction error decreases in the step from $\tau = 0$ to $\tau = 1$. The depth reconstruction result shown in figure 5.8 shows false classifications in unstructured areas of the input image for $\tau = 0$, which are eliminated for $\tau = 1$.

5.3.5 Coarse-to-fine search

Experiment description: In this simulated test run the effects of the coarse-to-fine refinement on runtime and quality are to be measured.

The algorithm is executed in the apartment and laboratory test environments under different search range breadths $K \in \{1, \ldots, 8\}$. A reference run is performed in which coarse-to-fine refinement is disabled.

Results: The result of the test runs is shown in figure 5.9: One can see, that in comparison to the reference run, the depth reconstruction error μ_D is reduced for $K \geq 3$. The same is true for the false positive count ν_F, yet a search range of $K = 6$ is required in order to perform considerably better in the apartment scene. For large search ranges the true positive count ν_T is slightly reduced compared to the reference value.

The runtime rises with increasing search range, yet even for $K = 8$ the total runtime is reduced by nearly 50% compared to the reference run.

Figure 5.10 shows detected and reference obstacle points as a global top-down map in the laboratory environment without coarse-to-fine refinement. In contrast to the result shown in figure 5.4 (which had coarse-to-fine refinement enabled), false positive obstacle points are detected in the lower left region. This region features repeating patterns including a bookshelf and a chess board, see second laboratory snapshot example image in figure 5.2. Figure 5.11 shows a snapshot of the depth reconstruction for that region, with both enabled and disabled coarse-to-fine refinement. One can see, that the coarse-to-fine refinement eliminates the artifacts in the depth map, which are suspected to cause the false positive obstacle points.

5.3.6 Noise robustness

Experiment description: In this test, the impact of noisy input data is measured. The noise leads to violations of the assumptions made in section 2.1.2. The input data (motion parameters d, α and ψ, as well as the input image $\mathbf{I}$) are superimposed by an additive Gaussian noise term with standard deviation σ. The noise is sampled independently at each data point/pixel.

The following (arbitrarily chosen) standard deviations are used: For d (traveled distance) σ is varied from $\sigma = 0\,\mathrm{m}$ to $\sigma = 0.025\,\mathrm{m}$ (the relative error

at $d = 0.04$ ranges from 0% to 62.5%). For α (motion direction) σ is varied from $\sigma = 0$ to $\sigma = 0.2$ radians (in degrees: 0° to 11.46°). For ψ (change in orientation) σ is varied from $\sigma = 0$ to $\sigma = 0.1$ (in degrees: 0° to 5.73°). For the image $\mathbf{I}$ the standard deviation is varied from $\sigma = 0$ to $\sigma = 40$ (relative deviation ranges from 0% to 16% for 8-bit intensity values). In all experiments σ is varied in 20 steps.

Results: The results presented in figure 5.12 show that independent noise superimposed on the input image and the motion parameters α and d only has a very subtle impact on the count of false positive obstacles until certain thresholds are reached: Image noise starts to effect the output noticeably at $\sigma \approx 20$ (8%), noise in d is is tolerable up to $\sigma \approx 0.005\,\mathrm{m}$ (12.5%) and noise in α has only little effect on the false positive count for deviations up to $\sigma \approx 0.1$ radians (5.73°). However, even small deviations in the orientation change ψ result in a high false positive count: Even a deviation of 0.02 radians ($\approx 1^\circ$) causes almost every second reconstructed obstacle point to be false positive. See figure 5.13 for the impact of noisy ψ on the depth reconstruction.

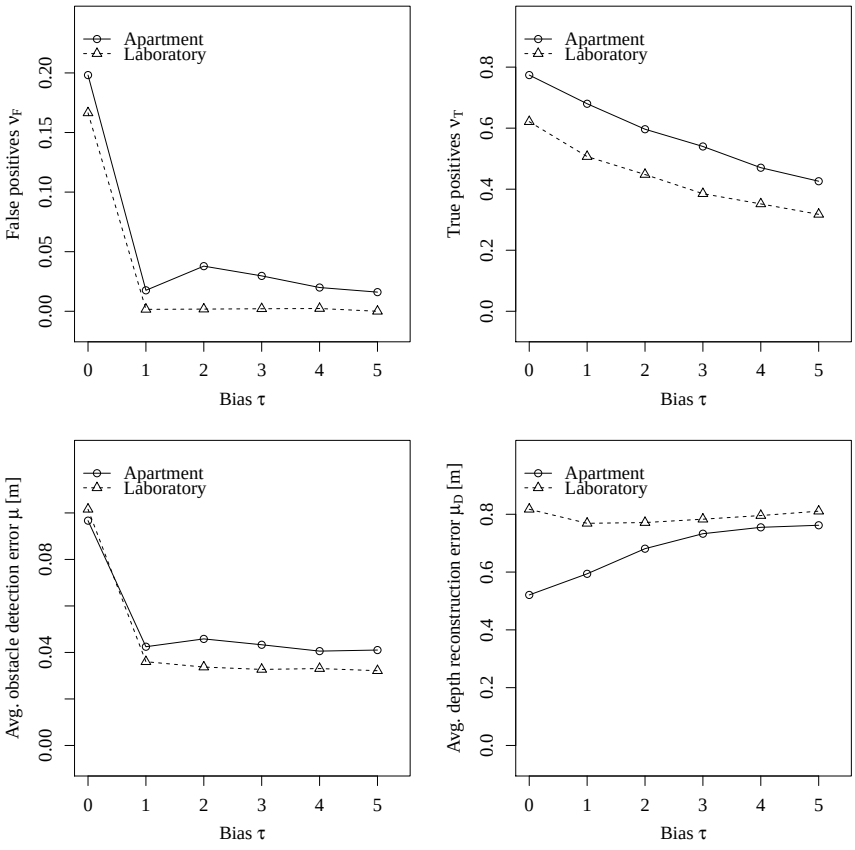
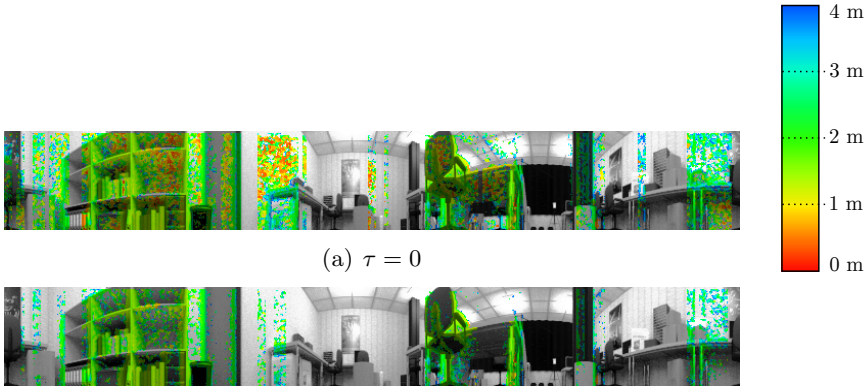


Figure 5.7: Results for varying the error stack bias τ for both the laboratory and the apartment scene.



(a) $\tau = 0$

(b) $\tau = 1$

FIGURE 5.8: Two depth reconstruction results for the laboratory scene for $\tau = 0$ and $\tau = 1$. For these particular images, the reduced elevation angle $\hat{\gamma}'$ was set to $\hat{\gamma} = 30°$. Uncolored image regions either have a reconstructed distance larger than 4 m or were pruned in the coarse-to-fine refinement.

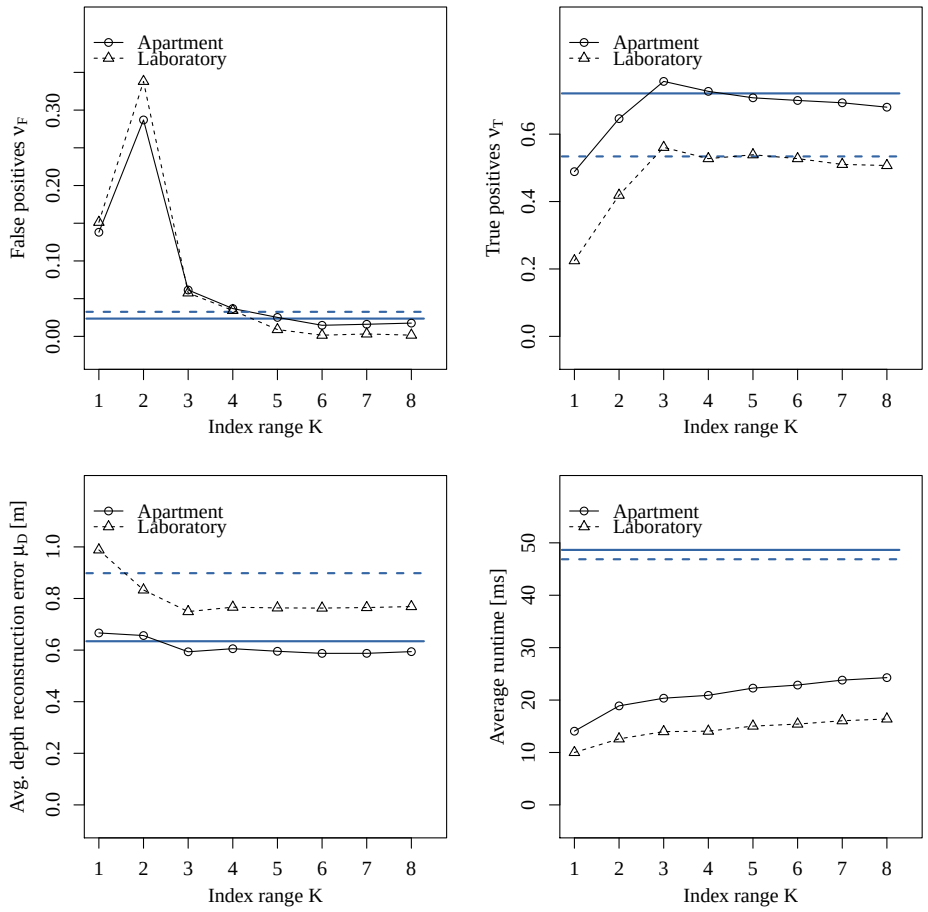


Figure 5.9: Results for varying index search range K in both the apartment and the laboratory environment. The horizontal lines (blue) correspond to the result without coarse-to-fine refinement.

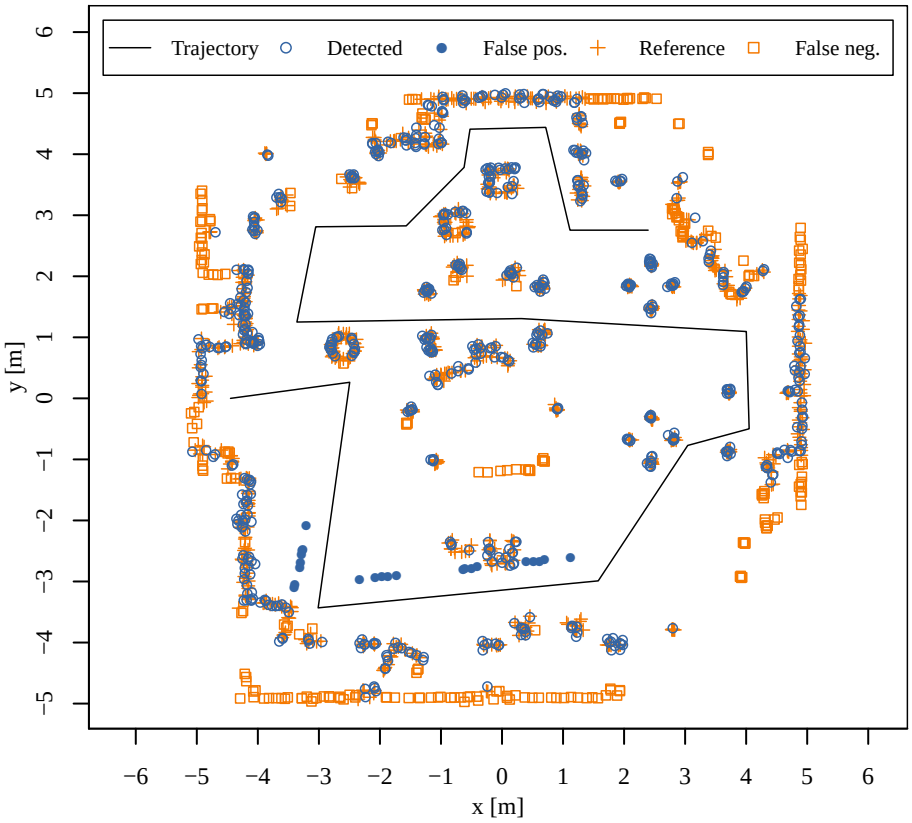
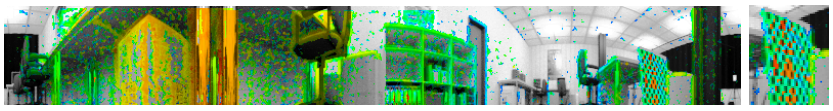
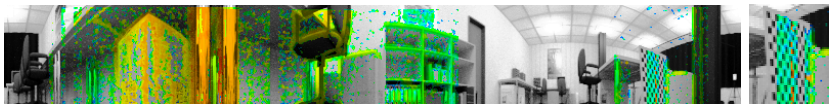


FIGURE 5.10: Obstacle detection performance in the simulated laboratory environment without coarse-to-fine refinement. Note the false positive points. See figure 5.1 for a room plan and see figure 5.4 for the run with coarse-to-fine refinement.



(a) Without coarse-to-fine refinement



(b) Coarse-to-fine search, $K = 8$

FIGURE 5.11: Two depth reconstruction results for the laboratory scene for the reference run (without coarse-to-fine refinement) (a) and with coarse-to-fine-search, $K = 8$ (b). The right images show the enlarged chess-board area. See figure 5.8 for details on the used color coding.

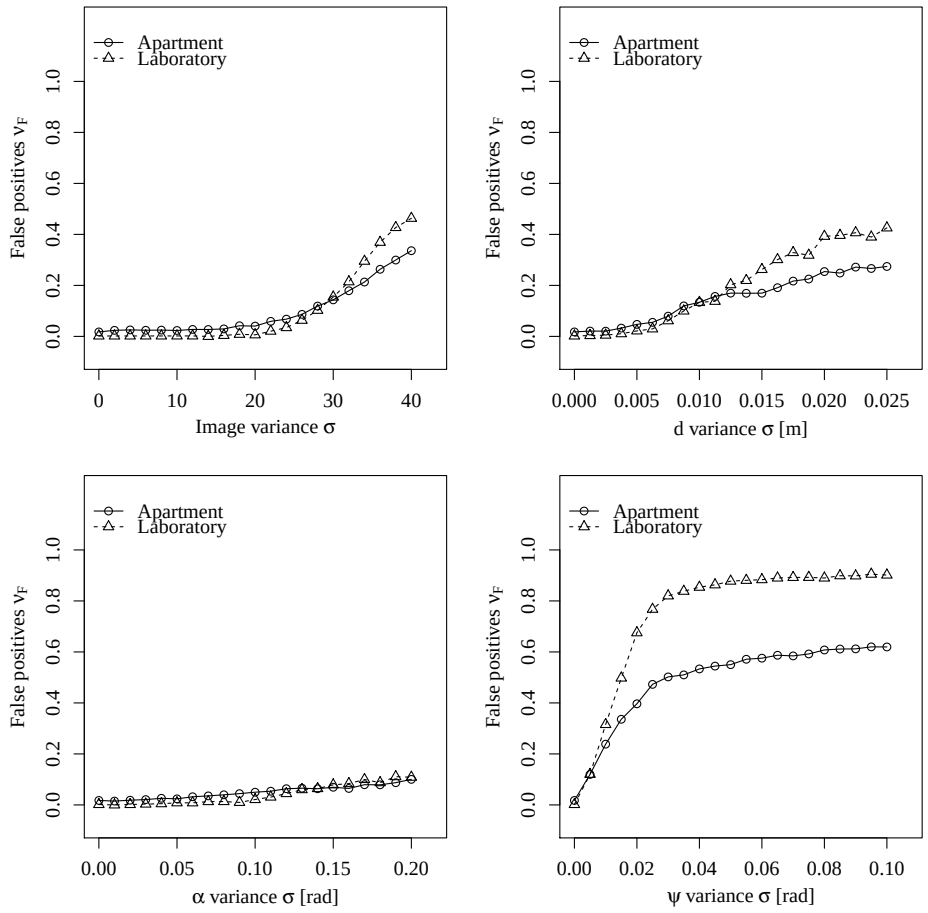


Figure 5.12: Impact of noise superimposed on the input images and the motion parameters on the relative false positive count ν_F for both the laboratory and the apartment scene.

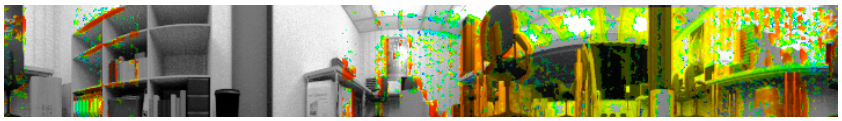


Figure 5.13: Depth reconstruction result for the laboratory scene showing artifacts introduced by a 0.0287 radians (1.64°) deviation of ψ. See figure 5.8 for details on the used color coding and the reference result.

Chapter 6

Discussion and conclusion

In this chapter the results from the previous chapter are discussed, followed by a brief conclusion and an outlook concerning future work.

6.1 Discussion

6.1.1 Simulated test runs

The results of the simulated test runs show that the algorithm presented in this thesis is capable of reconstructing obstacles from a time-series of of spherical panoramic images and fully fulfills the real-time constraints, typically requiring less than 30 ms per snapshot on the target platform, an Intel® Atom™ N2600 dual core CPU running at 1.60 GHz.

As expected, the heuristic used in the depth reconstruction stage produces very coarse results: The mean depth reconstruction error ranges from 0.5 to 0.8 m. However, the post-processing of the reconstructed landmark points in the local environment map increases the obstacle detection precision by an order of magnitude, allowing the algorithm to detect obstacle points with a median error of about 3 cm and mean error of about 6 cm if perfect odometry data is available.

The algorithm is unexpectedly robust concerning glossy surfaces, although these were explicitly excluded from the assumptions made in section 2.1.2: The stainless steel table legs in the laboratory scene are correctly detected, and while obstacle points are detected at the edges of the stainless steel refrigerator only, its glossiness does not cause any false positive obstacle points.

The false positive obstacle points in the apartment scene are believed to be caused on the one hand by the virtual camera moving too close to the obstacles (< 0.15 m, which is smaller than the distance corresponding to the last flow template) and on the other hand by the error stack bias, which causes local minima belonging to larger distances to be preferred.

The algorithm works best if the obstacles are structured, allowing for a distinct global minimum in the error stack. Unstructured objects, for example

the cylinders in the laboratory scene, are only detected at their edges. The algorithm is even capable of precisely reconstructing walls, as soon as those have some visible structure (like the tiled walls in the bathroom).

6.1.2 Real-world test

Earlier tests of the robot under laboratory conditions (results not shown here) have shown that the obstacle detection algorithm is capable of correctly detecting obstacles with no or only few false positives. The relatively poor performance under realistic test conditions is believed to be caused by the following factors:

Noisy orientation change parameter ψ: This may be caused by the odometry data not being captured synchronously with the snapshots. Especially when the robot is rotating, large deviations in ψ become apparent.

Tilting and change in elevation: In the test room, the edges of the carpet are especially problematic. While small tilt could be avoided by equipping the robot with some kind of mechanical suspension, tilt in critical quantity needs to be detected.

Unstructured environment: The room is relatively sterile featuring only uniformly colored objects.

The spreading of the reconstructed depth seen in figure 5.5(c) is partially assumed to be caused by occlusions and by the use of small space-invariant local error propagation matrices: There is no way for the depth reconstruction to decide which parts of the image are connected. Assuming that the uniformly colored background belongs to one of the chair/table legs is perfectly valid. This problem could only be solved by using global optical flow estimates (Pock, 2008; Fleet and Weiss, 2006) which are most likely not suitable for real-time execution on the robot.

However, the extreme impact of noise in ψ as shown in the corresponding experiment provides evidence that hardening the algorithm against this problem will also increase its real-world performance.

6.1.3 Detailed experiments

Error stack bias: The error stack bias τ proves to be mandatory for the obstacle-detection algorithm and behaves as intended: The bias promotes landmarks in regions with no visible structure to be classified as far-distance landmarks, which will then be ignored in later stages. The choice of τ turns out to be uncritical as long as chosen larger than zero.

Coarse to fine search: The results show that the coarse-to-fine refinement eliminates most artifacts resulting from temporal aliasing in regions with repeating patterns. Depending on the environment this can lead to a significant decrease in the count of false-positive obstacle points. Moreover, the pruning and the limited search range introduced along with the coarse-to-fine refinement reduce the total runtime by almost 50%. When comparing depth-reconstruction results, the pruning sometimes appears to be oversensitive, yet the data show that the decrease in the count of true positive obstacle points is negligible.

Noise: Noise in the orientation change parameter ψ proves to render the depth reconstruction result useless. This can partially be explained by the two following effects:

Reciprocal shift-to-depth function: As derived in appendix A.2.3, a horizontal pixel shift Δx between two projected landmark points $\vec{p}_1$ and $\vec{p}_2$ can approximately be transformed into a landmark distance:

$$r \approx \frac{w}{2\pi} \cdot \frac{d}{\Delta x} \,.$$

If additive noise is superimposed on small Δx, this results in a large change in r.

Forward search along flow lines: Another problem concerns the fact that the minimum error search along a flow line — as performed in the error stack calculation — is always a forward search. Noise in ψ can cause **CV** and **SS** to be aligned in such a way that the actual global minimum has already passed (at least in one half of **CV**). Due to the forward search only the next local minimum can be used.

6.2 Conclusion

The optimizations presented in this thesis allow to perform obstacle detection based on a time-series of spherical panoramic images while fulfilling the real-time constraint present on the used robot. The results from the simulated test environments prove that the general algorithm is working as intended and the introduced measures allow to verify how changes in the algorithm will effect the overall performance. Yet in order to be usable in arbitrary domestic in-door environments, the reliability of the algorithm needs to be increased.

6.3 Outlook

The following compilation enumerates some future research topics which are either hoped to resolve current issues, to further optimize the runtime requirements, or to be interesting from a scientific point of view.

Hardening against noise in ψ: As the odometry data obtained from the robot turned out to be far from perfect, the algorithm needs to be hardened against noise, especially in the orientation change parameter ψ. This problem could be solved in software by either using an image compass or by repeating the depth reconstruction algorithm (up to the minimum error search) for varying ψ and selecting the "best" result. The criteria for this choice need to be developed.

Optimization of the landmark selection: Currently all reconstructed landmarks originating from a single column are inserted into the local obstacle map with equal weight. A further optimization would be to select the prevalent distances from the column and to insert fewer, but weighted landmark points into the obstacle map. The selection could for example be performed by one-dimensional k-means.

Combination of multiple depth maps: Another possible extension would be to reconstruct the depth for multiple snapshots relative to a single current view. The resulting depth maps can then be combined into a single result by calculating the mean distance and pruning regions with high variance in the results. Yet this approach might prove to be difficult due to accumulating odometry errors.

Comparison to existing approaches: The results of this algorithm need to be compared to competing approaches, including feature based sparse optical flow as introduced in section 1.3.

Navigation: It would also be interesting to explore whether the local environment maps and the reconstructed obstacle points can be used for navigation.

GPU implementation: As the depth reconstruction is intrinsically parallel in every pixel (each pixel only depends on its direct neighbors), a GPU implementation using either the compute languages OpenCL or CUDA, or a GPGPU implementation using OpenGL (or to some degree even OpenGL ES) is believed to be viable.

Color images: It would be interesting to test the algorithm with color images as input. The availability of color increases the input dimensionality and is expected to allow for fewer ambiguities in the minimum error search.

Appendix A

Derivations and proofs

This appendix contains derivations and proofs for some of the statements made throughout the thesis. A note on notation: In order to shorten some equations, $\cos(\alpha)$ and $\sin(\alpha)$ are replaced by the shorter c_α and s_α.

A.1 Probabilistic landmark model

A.1.1 Covariance matrix transformation

The covariance matrix of the Gaussian distribution for polar coordinates $\vec{q} = (\beta, r)^T$ was given in equation 2.15 as

$$\vec{q}_R \sim \mathcal{N}\left(\vec{q}_E, \Sigma(\vec{q}_E)\right) \text{ where } \Sigma(\beta, r) := \begin{pmatrix} \sigma_\beta^2 & 0 \\ 0 & \sigma_r^2 \end{pmatrix} .$$

This distribution can be estimated by a Gaussian distribution in Cartesian space (see figure A.1 for an overview). Note that the estimation introduces a small bias towards smaller distances. The magnitude of this bias depends on the ratio of σ_β to σ_r. The estimation is performed by choosing a covariance matrix $\Sigma_\mathcal{C}$ with Eigenvectors pointing in radial direction and the corresponding orthogonal direction. The Eigenvectors $\vec{v}_r, \vec{v}_\beta$ of $\Sigma_\mathcal{C}$ should hence have the following form:

$$\vec{v}_r = \begin{pmatrix} c_\beta \\ s_\beta \end{pmatrix} \qquad\qquad \vec{v}_\beta = \begin{pmatrix} -s_\beta \\ c_\beta \end{pmatrix}^T$$

The Eigenvalues of a Covariance matrix correspond to the squared standard deviations of the elementary distributions:

$$\lambda_r = \sigma_r^2 \qquad\qquad \lambda_\beta = (r \cdot \sigma_\beta)^2$$

The angular standard deviation needs to be multiplied with r, as in Cartesian coordinates the length of the underlying circle segment scales with the radius.

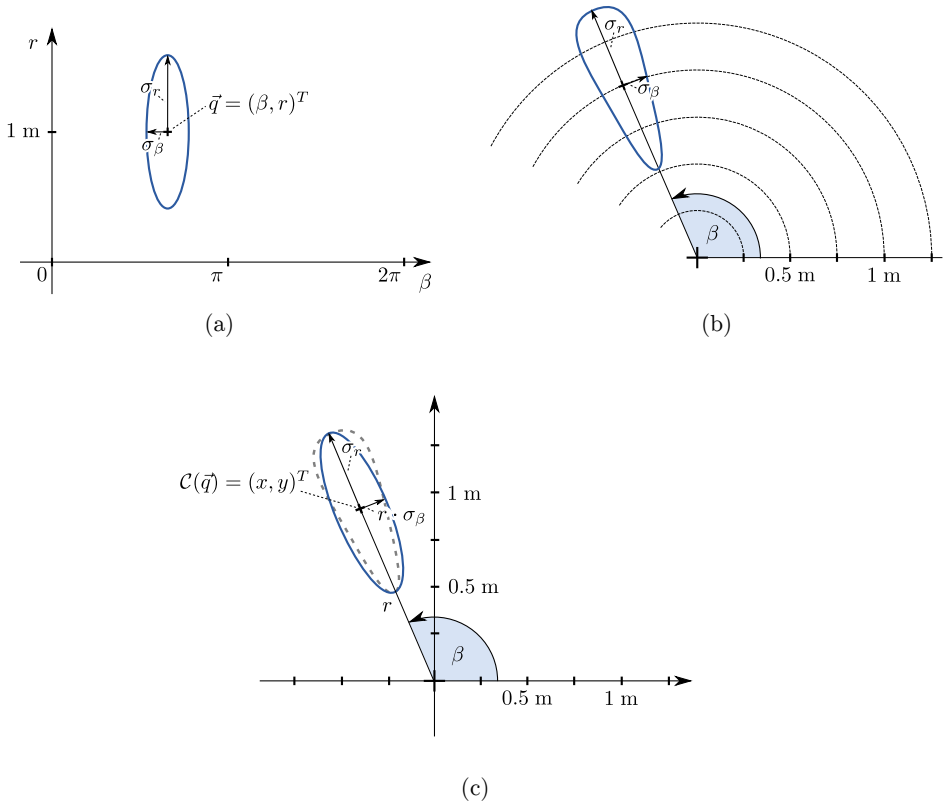


Figure A.1: Idea of the covariance matrix transformation from polar coordinates to Cartesian coordinates. (a) Shows a contour of the original Gaussian distribution in polar space. (b) Shows the same distribution drawn in a polar coordinate system. Note how the distribution is distorted. (c) Shows the approximated Gaussian distribution in a Cartesian coordinate system. The distortion present in (b) is discarded. The angular variance σ_β is transformed to Cartesian coordinates by multiplication with the radius r. The covariance matrix of the Cartesian distribution is then calculated by rotating the Eigenraum of the original covariance matrix.

The desired $\Sigma_\mathcal{C} = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ can now be described by the following equations:

$$\Sigma_\mathcal{C} \cdot \vec{v}_r = \begin{pmatrix} a & b \\ b & c \end{pmatrix} \begin{pmatrix} c_\beta \\ s_\beta \end{pmatrix} = \lambda_r \begin{pmatrix} c_\beta \\ s_\beta \end{pmatrix}$$
$$\Sigma_\mathcal{C} \cdot \vec{v}_\beta = \begin{pmatrix} a & b \\ b & c \end{pmatrix} \begin{pmatrix} -s_\beta \\ c_\beta \end{pmatrix} = \lambda_\beta \begin{pmatrix} -s_\beta \\ c_\beta \end{pmatrix}$$

Solving the equations for b yields:

$$b = \frac{(\lambda_r - a) \cdot c_\beta}{s_\beta} = \frac{(\lambda_r - c) \cdot s_\beta}{c_\beta} = \frac{(a - \lambda_\beta) \cdot s_\beta}{c_\beta} = \frac{(c - \lambda_\beta) \cdot c_\beta}{s_\beta}$$

Solve these equalities for a, c and b:

$$
\begin{aligned}
a &= \lambda_r c_\beta^2 + \lambda_\beta s_\beta^2 \\
c &= \lambda_r s_\beta^2 + \lambda_\beta c_\beta^2 \\
b &= \left(\lambda_r(1 - c_\beta^2) + \lambda_\beta s_\beta^2\right) \frac{c_\beta}{s_\beta} = \left(\lambda_r c_\beta^2 + \lambda_\beta(s_\beta^2 - 1)\right) \frac{s_\beta}{c_\beta} = c_\beta s_\beta(\lambda_r - \lambda_\beta)
\end{aligned}
$$

The Cartesian approximate of the polar covariance matrix is thus given as:

$$
\Sigma_{\mathcal{C}}(\beta, r) = \begin{pmatrix} (\sigma_r c_\beta)^2 + (r\sigma_\beta s_\beta)^2 & c_\beta s_\beta(\sigma_r^2 - (r\sigma_\beta)^2) \\ c_\beta s_\beta(\sigma_r^2 - (r\sigma_\beta)^2) & (\sigma_r s_\beta)^2 + (r\sigma_\beta c_\beta)^2 \end{pmatrix}
$$

A.1.2 Derivation of σ_β for adjacent Gaussian distributions

The angular variance σ_β used in equation 2.18 is derived by demanding that the sum two adjacent Gaussian distributions ($\mu_1 = \mu_2 + \Delta\beta$) nearly forms a uniform distribution in the interval $[\mu_1, \mu_2]$ as shown in figure 2.10. The variance can be derived as a solution to the following more general problem:

Given the sum of two (one dimensional) Gaussian distributions with means μ_1 and μ_2 (where $\zeta := \mu_2 - \mu_1$) and a common variance σ:

$$
f(x) = \mathcal{N}(x \mid \mu_1, \sigma) + \mathcal{N}(x \mid \mu_2, \sigma)
$$

Goal: Find σ such that second derivative of $f(x)$ at $x = \mu_1 + \zeta/2$ equals zero:

$$
\left.\frac{\partial f}{\partial x^2}\right|_{x=\mu_1+\zeta/2} \stackrel{!}{=} 0
$$

By shifting $\mu_1 + \zeta/2$ to the origin, the equation can be rewritten as:

$$
\left.\frac{\partial f\,(x - \mu_1 - \zeta/2)}{\partial x^2}\right|_{x=0} = \frac{\partial}{\partial x^2}\ \left(\mathcal{N}\left(x \mid -\zeta/2, \sigma\right) + \mathcal{N}\left(x \mid \zeta/2, \sigma\right)\right)\Big|_{x=0} \stackrel{!}{=} 0
$$

Derivation and evaluation at $x = 0$ yield (α represents all omitted normalization and constant factors):

$$
\frac{\partial}{\partial x^2}\ \left(\mathcal{N}\left(x \mid -\zeta/2, \sigma\right) + \mathcal{N}\left(x \mid \zeta/2, \sigma\right)\right)\Big|_{x=0} = \alpha \cdot \frac{\exp\left(-\frac{\zeta^2}{8\,\sigma^2}\right)}{\sigma^5}\left(-4\,\sigma^2 + \zeta^2\right) \stackrel{!}{=} 0
$$

The solution for σ is then given as

$$
\sigma = \pm\frac{\zeta}{2}\,.
$$

Setting $\zeta = \Delta\beta$ yields the term used in equation 2.18.

A.2 Motion field equation

A.2.1 Substitution of r and d

The parameters r and d of equation 2.4 are redundant and can be substituted by a single parameter $\rho := \frac{d}{r}$.

Proof. Equation 2.4 was given as

$$\vec{u}(\vec{p}; T(d, \psi, \alpha), r) = \begin{pmatrix} \beta' - \beta \\ \gamma' - \gamma \end{pmatrix} .$$

Expand the equation as defined in 2.2, 2.3 and 2.5. As $\arctan2(y, x)$ only selects the sign of the result depending on the quadrant of the input vector, $\arctan2(y, x)$ is substituted by $\arctan(y/x)$ in this proof.

$$\begin{aligned}
\beta' &= \arctan\left(\frac{-rs_\psi c_\beta + rc_\psi s_\beta + ds_\alpha}{rc_\psi c_\beta + rs_\psi s_\beta + dc_\alpha}\right) \\
&= \arctan\left(\frac{r \cdot (-s_\psi c_\beta + c_\psi s_\beta + \rho s_\alpha)}{r \cdot (c_\psi c_\beta + s_\psi s_\beta + \rho c_\alpha)}\right) \\
&= \arctan\left(\frac{-s_\psi c_\beta + c_\psi s_\beta + \rho s_\alpha}{c_\psi c_\beta + s_\psi s_\beta + \rho c_\alpha}\right) \\
\gamma' &= \arctan\left(\frac{r \cdot \tan(\gamma)}{\sqrt{(rc_\psi c_\beta + rs_\psi s_\beta + dc_\alpha)^2 + (-rs_\psi c_\beta + rc_\psi s_\beta + ds_\alpha)^2}}\right) \\
&= \arctan\left(\frac{r \cdot \tan(\gamma)}{r \cdot \sqrt{(c_\psi c_\beta + s_\psi s_\beta + \rho c_\alpha)^2 + (-s_\psi c_\beta + c_\psi s_\beta + \rho s_\alpha)^2}}\right) \\
&= \arctan\left(\frac{\tan(\gamma)}{\sqrt{(c_\psi c_\beta + s_\psi s_\beta + \rho c_\alpha)^2 + (-s_\psi c_\beta + c_\psi s_\beta + \rho s_\alpha)^2}}\right)
\end{aligned}$$

□

A.2.2 Rotational motion elimination

It was stated in equation 3.7:

$$\vec{u}(\vec{p}; \psi, \alpha, \rho) = \vec{u}(\vec{p} - (\psi + \alpha, 0)^T; 0, 0, \rho) - (\psi, 0)^T$$

Proof. Prove the equation in two steps using the results derived in the previous section A.2.1:

1. It holds $\vec{u}(\vec{p};\alpha,\psi,\rho) = \vec{u}(\vec{p}-(\psi,0)^T;\alpha,0,\rho)-(\psi,0)^T$. Apply the addition theorems:

$$\begin{aligned}
\beta'(\beta,\psi) &= \arctan\left(\frac{-s_\psi c_\beta + c_\psi s_\beta + \rho s_\alpha}{c_\psi c_\beta + s_\psi s_\beta + \rho c_\alpha}\right)\\
&= \arctan\left(\frac{\sin(\beta-\psi)+\rho s_\alpha}{\cos(\beta-\psi)+\rho c_\alpha}\right)\\
&= \beta'(\beta-\psi,0)\\
\gamma'(\beta,\psi) &= \arctan\left(\frac{\tan(\gamma)}{\sqrt{(c_\psi c_\beta + s_\psi s_\beta + \rho c_\alpha)^2 + (-s_\psi c_\beta + c_\psi s_\beta + \rho s_\alpha)^2}}\right)\\
&= \arctan\left(\frac{r\cdot\tan(\gamma)}{r\cdot\sqrt{(\cos(\beta-\psi)+\rho c_\alpha)^2 + (\sin(\beta-\psi)+\rho s_\alpha)^2}}\right)\\
&= \gamma'(\beta-\psi,0)
\end{aligned}$$

   Plug these results into the definition of $\vec{u}$:

$$\begin{aligned}
\vec{u}(\vec{p};\psi,\alpha,\rho) &= \begin{pmatrix}\beta'(\beta,\psi)-\beta\\ \gamma'(\beta,\psi)-\gamma\end{pmatrix}\\
&= \begin{pmatrix}\beta'(\beta-\psi,0)-\beta+\psi\\ \gamma'(\beta-\psi,0)-\gamma\end{pmatrix}\\
&= \vec{u}(\vec{p}-(\psi,0)^T;\alpha,0,\rho)-(\psi,0)^T
\end{aligned} \tag{A.1}$$

2. It holds $\vec{u}(\vec{p};\alpha,0,\rho) = \vec{u}(\vec{p}-(\alpha,0)^T;0,0,\rho)$. Expand the argument of the arctangent in $\beta'(\beta+\alpha,\alpha)$ using the addition theorems and reorder:

$$\begin{aligned}
\beta'(\beta-\alpha,\alpha) &= \arctan\left(\frac{\sin(\beta-\alpha)+\rho s_\alpha}{\cos(\beta-\alpha)+\rho c_\alpha}\right)\\
&= \arctan\left(\frac{c_\alpha s_\beta - s_\alpha(c_\beta+\rho)}{s_\alpha s_\beta + c_\alpha(c_\beta+\rho)}\right)\\
&\overset{\text{Rot. Mat.}}{=} \arctan\left(\frac{s_\beta}{c_\beta+\rho}\right)+\alpha\\
&= \beta'(\beta,0)-\alpha
\end{aligned}$$

   Analogously it holds for $\gamma'(\beta-\alpha,\alpha)$:

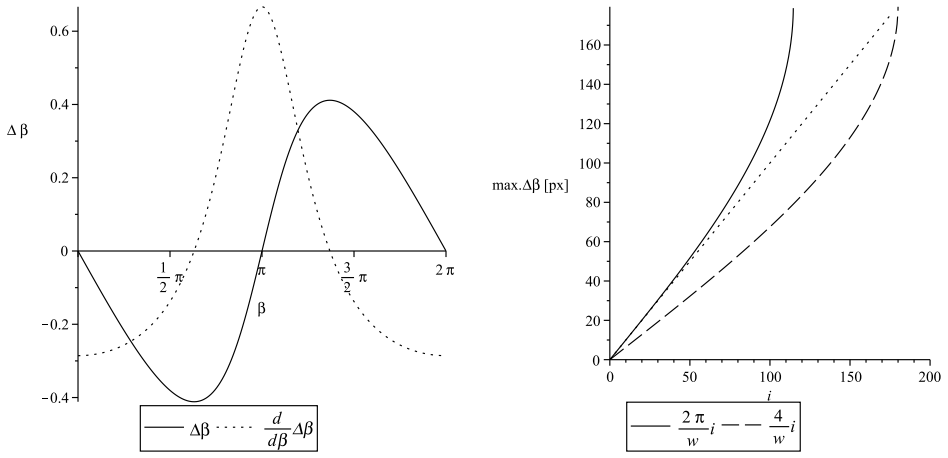
$$\begin{aligned}
\gamma'(\beta-\alpha,\alpha) &= \arctan\left(\frac{r\cdot\tan(\gamma)}{r\cdot\sqrt{(\cos(\beta-\alpha)+\rho c_\alpha)^2+(\sin(\beta-\alpha)+\rho s_\alpha)^2}}\right)\\
&= \arctan\left(\frac{r\cdot\tan(\gamma)}{r\cdot\sqrt{(s_\alpha s_\beta + c_\alpha(c_\beta+\rho))^2+(c_\alpha s_\beta - s_\alpha(c_\beta+\rho))^2}}\right)\\
&= \arctan\left(\frac{r\cdot\tan(\gamma)}{r\cdot\sqrt{(c_\beta+\rho)^2+(s_\beta)^2}}\right)\\
&= \gamma'(\beta,0)
\end{aligned}$$

Plug these results into the definition of $\vec{u}$:

$$\begin{aligned}\vec{u}(\vec{p}-(\alpha,0)^T;0,0,\rho) &= \begin{pmatrix}\beta'(\beta-\alpha,\alpha)-\beta+\alpha\\ \gamma'(\beta-\alpha,\alpha)\end{pmatrix}\\ &= \begin{pmatrix}\beta'(\beta,0)-\beta\\ \gamma'(\beta,0)\end{pmatrix}\\ &= \vec{u}(\vec{p};\alpha,0,\rho)\end{aligned} \tag{A.2}$$

Combining the results from equations A.1 and A.2 yields the desired equation. □

A.2.3 Maximum horizontal shift



(a) (b)

Figure A.2: (a) Plot of $\Delta\beta(\beta)$ and its derivative for $\rho = 0.4$. (b) A plot of the horizontal shift in pixels at $\hat{\beta}$ for both approximations presented below. The image width was set to $w = 720$. The dotted line represents the aimed-at behavior as achieved by the analytical solution.

In section 3.2.2 the formula $\rho_i = \frac{2\pi}{w} \cdot i$ was introduced, which approximately ensures the horizontal motion field shift $\Delta\beta(\beta) = \beta'(\beta) - \beta$ to be smaller than one pixel per step for $0 \leq \rho \ll 1$.

Derivation. The shift in horizontal direction $\Delta\beta(\beta)$ is given as:

$$\Delta\beta(\beta) = \beta'(\beta) - \beta = \operatorname{arctan2}\left(\sin(\beta), \cos(\beta) + \rho\right) - \beta$$

The extrema of this function can be found by calculating the first derivative and equating it with zero:

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}\beta}\Delta\beta(\beta) &= \frac{\mathrm{d}}{\mathrm{d}\beta}\left(\arctan2\left(\sin(\beta),\cos(\beta)+\rho\right)-\beta\right) \\
&= \frac{\cos(\beta)^2+\rho\cos(\beta)+\sin(\beta)^2}{(\cos(\beta)+\rho)^2}\cdot\frac{(\cos(\beta)+\rho)^2}{(\cos(\beta)+\rho)^2+\sin(\beta)^2}-1 \\
&= \frac{(\cos(\beta)+\rho)^2+\sin(\beta)^2-\cos(\beta)\rho-\rho^2}{(\cos(\beta)+\rho)^2+\sin(\beta)^2}-1 \\
&= -\frac{\rho\cdot(\cos(\beta)+\rho)}{(\cos(\beta)+\rho)^2+\sin(\beta)^2}\overset{!}{=}0 \\
\Leftrightarrow 0 &= \cos(\beta)+\rho \\
\Leftrightarrow \beta &= \underbrace{\pm\arccos(-\rho)}_{=:\hat{\beta}}
\end{aligned}$$

Note that this solution is only valid for $0 \leq \rho \leq 1$. Plugging this result into $\Delta\beta$ and equating $\Delta\beta(\hat{\beta})$ for multiples of the horizontal image resolution $\frac{2\pi}{w}$ (with w being the image width in pixels) results in the following equation:

$$\begin{aligned}
\Delta\beta(\hat{\beta}) &= \arctan2\left(\sin(\pm\arccos(-\rho_i)),\cos(\arccos(-\rho_i))+\rho_i\right)\mp\arccos(-\rho_i) \\
&= \arctan2\left(\pm\sqrt{1-\rho_i^2},0\right)\mp\arccos(-\rho_i) \\
&= \pm\frac{\pi}{2}\mp\arccos(-\rho_i)\overset{!}{\leq}\frac{2\pi}{w}\cdot i
\end{aligned}$$

An analytical solution that ensures equality is achieved for

$$\rho_i = -\cos\left(\frac{2\pi}{w}\cdot i+\frac{\pi}{2}\right)=\sin\left(\frac{2\pi}{w}\cdot i\right)\ .$$

For small $\frac{2\pi}{w}\cdot i$ the analytical solution can be approximated as:

$$\rho_i \approx \frac{2\pi}{w}\cdot i$$

As can be seen in figure A.2(b) the approximation seems to be acceptable for $i < 40$ and $w = 720$ px. In the current implementation only 32 error stack layers ($0 \leq i \leq 31$) are used.

A solution which ensures that the inequality stated above is always fulfilled can be obtained by linearly estimating the arctangent. It holds:

$$-\frac{\pi}{2}+\arccos(-\rho_i)\leq\frac{\pi}{2}\cdot\rho_i\overset{!}{\leq}\frac{2\pi}{w}\cdot i\ .$$

Solving the right part of the inequality for ρ_i results in

$$\rho_i \leq \frac{4}{w}\cdot i$$

which allows to fulfill the requirement for all $0 \leq \rho_i \leq 1$ (see figure A.2(b) for a comparison to the approximation).

Appendix B

Error propagation matrices

B.1 Space variant propagation matrix

When using a space-invariant convolution kernel $W = W_{\vec{p}}$, the minimum error search presented in section 2.2.6 implies that it is possible for pixels belonging to objects other than the object at the center pixel $\vec{p}$ to influence the accumulated error value. A mechanism that restricts the error propagation to regions belonging to the same object is desirable.

The assumption made here is that different objects (with possibly different depths) in the scene are separated by visible edges. This assumption appears to be reasonable, as the absence of edges in the input image means that the scene consists of homogeneously colored surfaces. In that case, it would not be possible to reconstruct depth at all.

Note that the inverse of the assumption does not hold: Objects might have inner edges that do not imply any change in depth. Yet this should be unproblematic as the presence of structured surfaces allows for distinct minimums in the error stack and thus a higher quality depth reconstruction.

Let $\mathbf{CV_E}(\vec{p})$ denote the edge amplitude of the current view image where $\vec{p} = (x, y)^T$:

$$\mathbf{CV_E}(\vec{p}) = \sqrt{\left(\left.\frac{\partial \mathbf{CV}}{\partial x}\right|_{\vec{p}}\right)^2 + \left(\left.\frac{\partial \mathbf{CV}}{\partial y}\right|_{\vec{p}}\right)^2}$$

The entries w_{ij} in the $2r_W + 1 \times 2r_W + 1$ filter kernels $W_{\vec{p}}$ (with $i, j \in \{-r_W, \ldots, r_W\}$) specify how many edges are located between the center pixel $\vec{p}$ and the position referenced by i, j. This way only the error values of pixels that are not separated from the center pixel by edges will be propagated. Let w'_{ij} denote the unnormalized error propagation matrix entires:

$$w'_{ij} = \prod_{\tilde{p} \in \mathrm{Path}(\vec{p}, \vec{p}+(i,j)^T)} (1 - \mathbf{CV_E}(\tilde{p})) \tag{B.1}$$

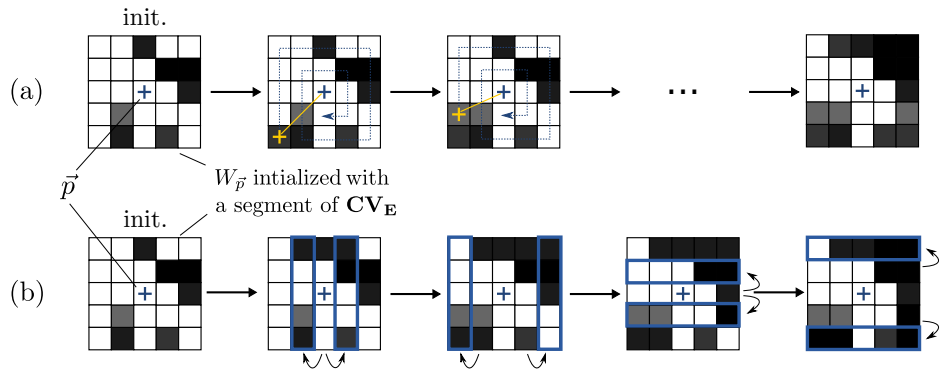


Figure B.1: Step-by-step visualization of the error propagation matrix calculation. The top figure (a) visualizes the calculation as given in equation B.1, (b) shows the calculation resulting from the approximation in equation B.2. In (a) the values from the edge image (black pixels correspond to edges) are multiplied along a line starting from the center pixel $\vec{p}$ (blue) to the target pixel (yellow). Choosing the target pixels along a spiral path allows the matrix to be updated in-place. The approximation in (b) can be calculated by in-place multiplication of neighboring columns and rows respectively starting from the center.

Where $\text{Path}(\vec{p}_1, \vec{p}_2) : \mathbb{R}^2 \times \mathbb{R}^2 \longrightarrow \mathcal{P}(\mathbb{R}^2)$ returns a set of points that lie on the line between $\vec{p}_1$ and $\vec{p}_2$. Such a set can for example be calculated using Bresenham's line algorithm (see Bresenham (1965)).

A less sophisticated approximation is given by following the principal axes in the matrix instead of straight paths. This results in the following formula:

$$w'_{ij} = \left(\prod_{\tilde{\imath} \in \{0,\dots,i\}} (1 - \mathbf{CV_E}(x + \tilde{\imath}, y + j)) \right) \cdot \left(\prod_{\tilde{\jmath} \in \{0,\dots,j\}} (1 - \mathbf{CV_E}(x + i, y + \tilde{\jmath})) \right) \tag{B.2}$$

A visualization of the propagation matrix calculation process can be found in figure B.1. In the experiments this significantly faster, second method is used.

The entries of the normalized propagation matrix are given as

$$w_{ij} = \frac{w'_{ij}}{\sum_{i,j} w'_{ij}}$$

B.2 Propagation matrix performance

Experiment description: The tests in this section are performed in order to determine the best type and radius r_W of the error propagation matrix $W_{\vec{p}}$. Note that this test is performed using the (non-realtime) author's personal reference implementation (not described in this thesis), as support for different

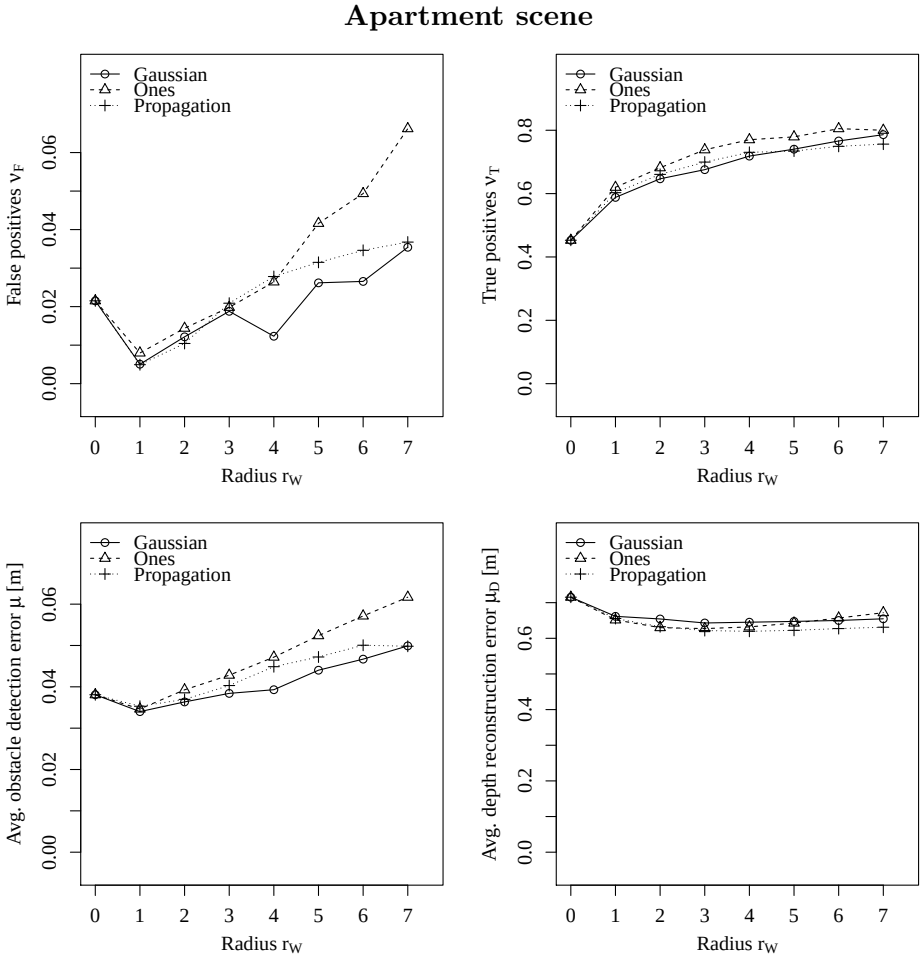


FIGURE B.2: Results for varying propagation matrices $W_{\vec{p}}$ in the apartment scene.

types of error propagation matrices and varying radii was removed from the optimized implementation.

The experiment is conducted in the simulated environments described in section 5. In the test both the propagation matrix radius $m \in \{0, \ldots, 7\}$ and its type are varied. The types include a discrete Gaussian filter kernel (approximated using a binomial filter kernel, dubbed "Gaussian"), a box filter (matrix of ones, dubbed "Ones"), and the error propagation matrix as described above (dubbed "Propagation").

Results: The results of the this experiment can be seen in figures B.2 and B.3. One can see that for $r_W > 0$ the relative count of false positives ν_F first decreases but increases again for larger r_W. The same is true for the average

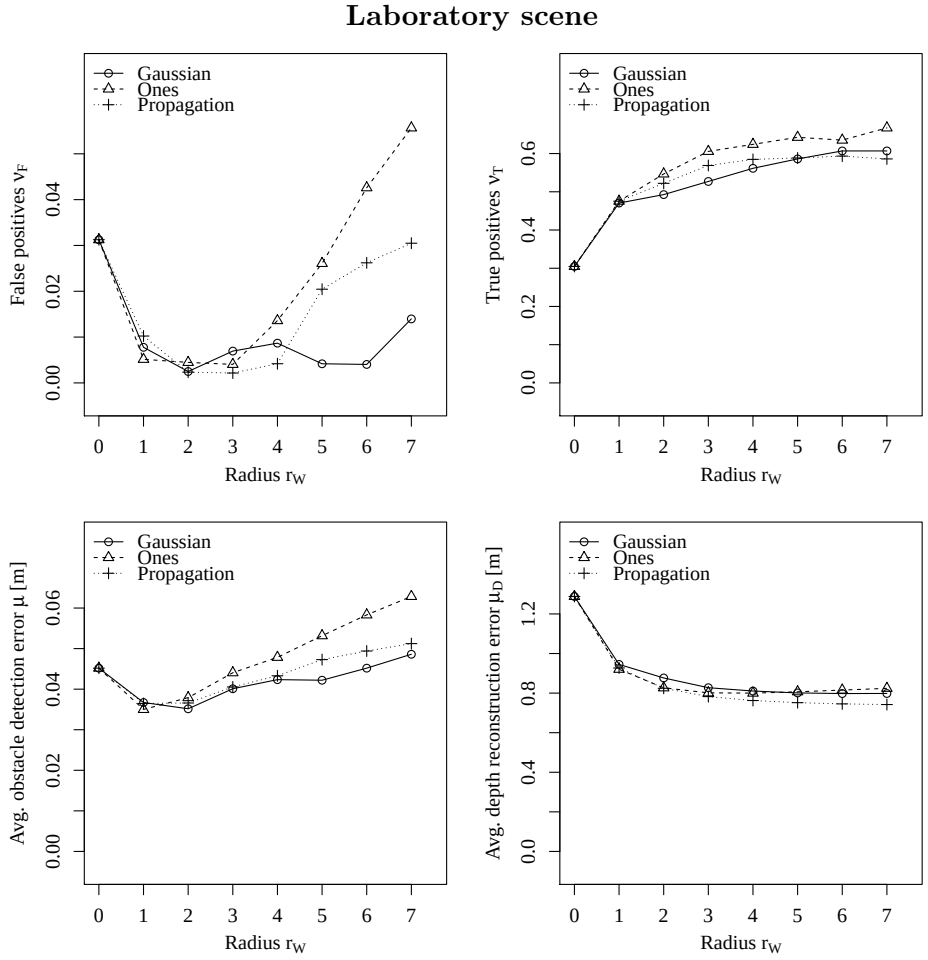


Figure B.3: Results for varying propagation matrices $W_{\vec{p}}$ in the laboratory scene.

obstacle detection error μ. The relative count of true positives ν_T increases with increasing r_W. An extraordinarily high increase in ν_T and (along with a large drop in ν_F and μ_D) can be seen in the step from $r_W = 0$ to $r_W = 1$).

Generally speaking there is no large difference in the performance of the different propagation matrix types. Yet the "Ones"-matrix tendentially shows the worst performance and the "Propagation"-matrices have a slightly superior depth reconstruction performance for large r_W.

A 3×3 (radius $r_W = 1$) binomial filter kernel was chosen for the optimized implementation: Larger radii increase the run time and do not necessarily yield a higher obstacle detection quality. The propagation matrices described above were not used, as they introduce an additional calculation step and (in contrast to the binomial kernel) are not separable and not space-invariant.

Appendix C

Source code

This appendix contains some excerpts of the source code at key points in the algorithm. Note that the code has been slightly simplified for illustration purposes. Thus each code snippet is accompanied by a reference to its location in the "real source code".

C.1 Error stack calculation

This section contains the code used for the error stack calculation as described in section 4.2.

C.1.1 Horizontal interpolation

Code located in `FlowSIMD-1-0/src/ErrorStack.C::_ErrorstackInterpolateHorz`

```
// Shuffle masks used for interlacing
const __m128i shuffleLo1 = _mm_set_epi8(
    0, 0x80, 1, 0x80, 2, 0x80, 3, 0x80,
    4, 0x80, 5, 0x80, 6, 0x80, 7, 0x80);
const __m128i shuffleLo2 = _mm_set_epi8(
    0x80, 0, 0x80, 1, 0x80, 2, 0x80, 3,
    0x80, 4, 0x80, 5, 0x80, 6, 0x80, 7};
const __m128i shuffleHi1 = _mm_set_epi8(
    8, 0x80, 9, 0x80, 10, 0x80, 11, 0x80,
    12, 0x80, 13, 0x80, 14, 0x80, 15, 0x80};
const __m128i shuffleHi2 = _mm_set_epi8(
    0x80, 8, 0x80, 9, 0x80, 10, 0x80, 11,
    0x80, 12, 0x80, 13, 0x80, 14, 0x80, 15};

// Load the source pointers
__m128i *src1 = (__m128i*)snapshot.components[c].ptr(
    tarX(d, x) + 0 + ssXOffs);
__m128i *src2 = (__m128i*)snapshot.components[c].ptr(
    tarX(d, x) + 1 + ssXOffs);
```

```
// Load the factors
__m128i fac = _mm_set_epi8(
    facX(x)[0], facX(x)[1],
    facX(x)[0], facX(x)[1],
    [...]);

for (int y = 0; y < h16; y++) {
  // Load the two source words (16x8-bit)
  __m128i in1 = _mm_load_si128(src1++);
  __m128i in2 = _mm_load_si128(src2++);

  // Interlace the two input words and process the
  // LO part
  __m128i in1Lo = _mm_shuffle_epi8(in1, shuffleLo1);
  __m128i in2Lo = _mm_shuffle_epi8(in2, shuffleLo2);
  __m128i inLo = _mm_or_si128(in1Lo, in2Lo);

  // Calculate a 16-bit intermediate result
  __m128i resLo = _mm_maddubs_epi16(inLo, fac);

  // Scale the result down to a 8-bit range
  resLo = _mm_srai_epi16(resLo, TEMPL_SCALE_FACTOR_LOG);

  // Interlace the two input words and process the
  // HI part
  __m128i in1Hi = _mm_shuffle_epi8(in1, shuffleHi1);
  __m128i in2Hi = _mm_shuffle_epi8(in2, shuffleHi2);
  __m128i inHi = _mm_or_si128(in1Hi, in2Hi);

  // Calculate a 16-bit intermediate result
  __m128i resHi = _mm_maddubs_epi16(inHi, fac);

  // Interlace the 8x16-bit words and store the
  // result
  __m128i res = _mm_packus_epi16(resLo, resHi);
  _mm_store_si128(tar++, res);
}
```

C.1.2 Vertical interpolation

Code located in `FlowSIMD-1-0/src/ErrorStack.C::_ErrorstackInterpolateVert`

```
// Iterate over 8 pixel blocks in the intermediate
// column
for (int y = 0; y < h8; y++) {
  // Load the interpolation factors from facY
  __m128i fac = _mm_load_si128(facYPtr++);
```

```
  // Load the shuffle mask from shuffleMaskY
  __m128i shuffle = _mm_load_si128(shuffleMaskYPtr++);

  // Load the input data from the intermediate row
  // and apply the offset offsYPtr
  __m128i in = _mm_loadu_si128(
      (__m128i*)(src + *(offsYPtr++)));

  // Shuffle the input word according to the
  // shuffle mask
  in = _mm_shuffle_epi8(in, shuffle);

  // Perform the actual interpolation, scale
  // the result down to a 8-bit range
  __m128i ss = _mm_maddubs_epi16(in, fac);
  ss = _mm_srli_epi16(ss, TEMPL_SCALE_FACTOR_LOG);

  // Calculate the absolute difference between
  // SS and CV
  __m128i cv = _mm_load_si128(cvPtr++);
  __m128i err = _mm_sub_epi16(ss, cv);
  err = _mm_abs_epi16(err);

  // Mask invalid results with maximal error
  __m128i mask = _mm_cmpeq_epi16(shuffle, cmpMask);
  mask = _mm_srli_epi16(mask, MAX_ERR_SHIFT);
  err = _mm_or_si128(err, mask);

  // Store the result in the error stack
  _mm_store_si128(tar++, err);
}
```

C.2 Gauss template accumulation

Code located in `FlowSIMD-1-0/src/ObstacleMap.C::additiveBlitInline`

This code is used for accumulating the gauss templates at a certain x-y-position in the local environment map. The attribute `flip` specifies whether the template is to be placed in quadrants 2 or 4 (see figure 4.5).

```
// Load the mask used for reversing 16-bit
// SSE words.
const __mm128i reverseMask = _mm_set_epi8(
    14, 15, 12, 13, 10, 11, 8,
    9, 6, 7, 4, 5, 2, 3, 0, 1);

// Iterate over each row in the template
for (int yt = -h2; yt <= h2; yt++) {
  // Fetch the pointer to the row descriptor
```

```
  // containing the actual length of the row (len)
  // as well as a the count of words that have to
  // be skipped.
  FastGaussTemplateRowDescr rowDescr = *(rowPtr++);

  // Fetch the target pointers
  __m128i *tarPtr1, *tarPtr2;
  if (flip) {
    tarPtr1 = ((__m128i*)&(tar(y - yt, x)))
        + rowDescr.skip + 0;
    tarPtr2 = ((__m128i*)&(tar(y + yt, x)))
        - rowDescr.skip - 1;
  } else {
    tarPtr1 = ((__m128i*)&(tar(y + yt, x)))
        + rowDescr.skip + 0;
    tarPtr2 = ((__m128i*)&(tar(y - yt, x)))
        - rowDescr.skip - 1;
  }

  // Iterate over each SSE word in the row
  for (int i = 0; i < rowDescr.len; i++) {
    // Load gauss template and calculate its
    // reverse
    __m128i block1 = _mm_load_si128(blockPtr++);
    __m128i block2 = _mm_shuffle_epi8(block1,
        reverseMask);

    // Load the target blocks, add the gauss
    // template data and store the result
    __m128i tar1 = _mm_loadu_si128(tarPtr1);
    tar1 = _mm_adds_epi16(tar1, block1);
    _mm_storeu_si128(tarPtr1++, tar1);

    __m128i tar2 = _mm_loadu_si128(tarPtr2);
    tar2 = _mm_adds_epi16(tar2, block2);
    _mm_storeu_si128(tarPtr2--, tar2);
  }
}
```

Bibliography

M. Bertozzi and A. Broggi. Gold: a parallel real-time stereo vision system for generic obstacle and lane detection. *Image Processing, IEEE Transactions on*, 7(1):62–81, 1998. ISSN 1057-7149.

J. Bresenham. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1):25–30, 1965.

H. Bülthoff and A. Yuille. Shape-from-x: Psychophysics and computation. In *Fibers' 91, Boston, MA*, pages 235–246. International Society for Optics and Photonics, 1991.

D. Coombs and K. Roberts. 'Bee-bot': using peripheral optical flow to avoid obstacles. In *Proc. SPIE, Intelligent Robots and Computer Vision XI: Algorithms, Techniques, and Active Vision*, volume 1825, pages 714–721, 1992.

M. Egelhaaf. Insect motion vision. *Scholarpedia*, 4(11), 2009. URL `http://www.scholarpedia.org/article/Insect_motion_vision`.

D. Fleet and Y. Weiss. *Handbook of Mathematical Models in Computer Vision*, chapter Optical Flow Estimation, pages 237–258. Springer, 2006.

M. Franz, B. Schölkopf, A. Mallot, and H. Bülthoff. Where did i take that snapshot? scene-based homing by image matching. *Biological Cybernetics*, 79(3):191–202, 1998.

L. Gerstmayr-Hillen, F. Röben, M. Krzykawski, S. Kreft, D. Venjakob, and R. Möller. Dense topological maps and partial pose estimation for visual control of an autonomous cleaning robot. *Robotics and Autonomous Systems*, 61(5):497–516, 2013.

B. Horn and B. Schunck. Determining optical flow. *Artificial Intelligence*, 17: 185–203, 1981.

Intel 64 and IA-32 Architectures Optimization Reference Manual. Intel Corporation, July 2013. URL `http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-optimization-manual.pdf`.

R. Kawanishi, A. Yamashita, T. Kaneko, and H. Asama. Parallel line-based structure from motion by using omnidirectional camera in textureless scene. *Advanced Robotics*, 27(1):19–32, 2013.

J. Kim and Y. Suga. An omnidirectional vision-based moving obstacle detection in mobile robot. *International Journal of Control Automation and Systems*, 5(6):663–673, 2007.

T. Low and G. Wyeth. Obstacle detection using optical flow. In Claude Sammut, editor, *Australasian Conference on Robotics and Automation 2005*, Sydney, N.S.W, 2005. Australian Robotics and Automation Association Inc.

D. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60:91–110, 2004.

R. Möller, M. Krzykawski, and L. Gerstmayr. Three 2D–warping schemes for visual robot navigation. *Autonomous Robots*, 29(3):253–291, 2010.

T. Pock. *Fast Total Variation for Computer Vision*. PhD thesis, Graz University of Technology, 2008.

A. Saxena, S. Chung, and A. Ng. 3-d depth reconstruction from a single still image. *International Journal of Computer Vision*, 76(1):53–69, 2008. ISSN 0920-5691.

R. Siegwart, I. Nourbakhsh, and D. Scaramuzza. *Introduction to autonomous mobile robots*. Intelligent robotics and autonomous agents. MIT Press, Cambridge, Mass., 2. ed. edition, 2011.

Wikipedia. Wagon-wheel effect — Wikipedia, the free encyclopedia, 2013a. URL `http://en.wikipedia.org/w/index.php?title=Wagon-wheel_effect&oldid=547507805`. [Online; accessed 5-August-2013].

Wikipedia. Connected-component labeling — Wikipedia, the free encyclopedia, 2013b. URL `http://en.wikipedia.org/w/index.php?title=Connected-component_labeling&oldid=570782455`. [Online; accessed 4-September-2013].

Wikipedia. Nyquist–Shannon sampling theorem — Wikipedia, the free encyclopedia, 2013c. URL `http://en.wikipedia.org/w/index.php?title=Nyquist%E2%80%93Shannon_sampling_theorem&oldid=564238539`. [Online; accessed 15-August-2013].

Y. Yasushi. Omnidirectional sensing and its applications. *IEICE Transactions on Information and Systems*, 82(3):568–579, 1999.

W. Yoshizaki, Y. Mochizuki, N. Ohnishi, and A. Imiya. Free Space Detection from Catadioptric Omnidirectional Images for Visual Navigation using Optical Flow. In *8th Workshop on Omnidirectional Vision, Camera Networks and Non-classical Cameras - OMNIVIS*, Marseille, France, 2008. Rahul Swaminathan and Vincenzo Caglioti and Antonis Argyros.

Hiermit versichere ich, dass ich diese Bachelorarbeit selbständig bearbeitet habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt und entsprechende Zitate kenntlich gemacht.

Bielefeld, 5. September 2013

Datum	Andreas Stöckel